

claude-windows / c47f0851-d75f-45b6-9126-e48544381426

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c47f0851-d75f-45b6-9126-e48544381426\subagents\workflows\wf_5af9a132-4ce\agent-a24f5a83838336f4f.jsonl`
- SHA-256: `06ce72546bb7d77fcf2036fc983ba2f14e0adb75f40a1a839287b0ffd8e3c358`
- Source modified: `2026-06-20T09:02:05+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf_5af9a132-4ce`
- session_id: `c47f0851-d75f-45b6-9126-e48544381426`
```

Transcript

1. user (2026-06-20T08:57:11.749Z)

Repo: badminton-highlight-indexer (Python). Branch feat/native-wasb-runner off origin/master. The change builds the M3.5 Linux-native WASB detector runner + wires real extraction into the worker.

Files changed/added (read them in the WORKING TREE ? changes are uncommitted):

```
- backend/pipeline/detectors/native_wasb_runner.py (NEW ? the native runner)
- backend/pipeline/detectors/wasb_infer.py (added --device, threaded device into
_build_cfg/run/run_video_streaming/main; device-keyed the resumable manifest)
- backend/pipeline/segmenters/trajectory_hybrid.py (_resolve_runner: new 'native' branch +
_build_native_runner base hook; generalized 'WSL env' log lines)
- backend/pipeline/segmenters/wasb_hybrid.py (_build_native_runner override)
- backend/pipeline/segmenters/fusion_hybrid.py (_build_native_runner override; WASB
substrate only)
- backend/config/models.py (WasbIndexConfig.device + .python_bin;
detector_impl doc mentions 'native')
- backend/worker/__main__.py (cmd_train: --trajectory-source
{synth,detector} + --segmenter; _resolve_train_detector; detector path pulls videos + runs
runner)
- tests/test_native_wasb_runner.py, tests/test_worker.py (new tests)
```

DESIGN INTENT / CONTRACTS (judge against these):

```
- The EXISTING WSL runner is backend/pipeline/detectors/wasb_runner.py (WasbRunner,
WslCommandMixin). The native runner must be a sibling that runs the SAME wasb_infer.py natively
(no wsl, no 'conda activate' ? it invokes the wasb conda env python BY PATH, no /mnt path
translation) and emit a BYTE-IDENTICAL Frame,Visibility,X,Y CSV with the SAME filename
'{stem}__{vid12}.wasb.csv'.
- ACCEPTANCE GATE = byte-parity: same clip via WSL (Windows) vs native (Linux, device=cuda) ->
identical CSV + identical candidate windows. device default 'cuda' must keep wasb_infer's Hydra
overrides identical to before (parity).
- Selected via the detector_impl seam: indexing.detector_impl / RALLY_DETECTOR_IMPL = 'native'.
- Constraint: 'backend' must NOT import 'training' (worker shells out). The default trajectory
source stays 'synth' so the CPU e2e is unchanged.
- The code RUNS on Linux (GPU box) but the TESTS run on the dev's Windows machine; cross-
platform path handling matters.
- weights/repo/python/scratch come from env first
(WASB_WEIGHTS/WASB_REPO_DIR/WASB_PYTHON/WASB_SCRATCH) then indexing.wasb.* then default.
```

YOU ARE THE NO-WSL / CROSS-PLATFORM SKEPTIC. Confirm native_wasb_runner.py carries NO WSL/conda-activate/mnt plumbing (no WslCommandMixin, no 'wsl', no 'conda activate', no to_wsl_mnt_path) and find any leak. Then hunt cross-platform bugs: the code runs on LINUX but tests run on WINDOWS ? check os.path.expanduser, os.path.abspath, backslash normalization for the key, os.path.join for cwd/'src', shutil.copy/rmtree, subprocess list-form (no shell quoting needed). Will _repo_src()/cwd be correct on Linux? Will run_predict's os.path.abspath(video) mangle a real Linux path? Report concrete bugs.

2. assistant (2026-06-20T08:57:14.194Z)

I'll review this change as the no-WSL/cross-platform skeptic. Let me start by reading the key files.

3. user (2026-06-20T08:57:15.151Z)

```
1      """Linux-native WASB shuttle-trajectory runner (no WSL).
2
3      The de-risk goal the ``DetectorRunner`` ABC was built for (see ``base.py`` ?
4      "Optional Linux-native path"; ``docs/DECOUPLED_COMPUTE/04-BUILD-AND-TEST-PLAN.md``
M3.5).
5      It is the sibling of :class:`~backend.pipeline.detectors.wasb_runner.WasbRunner` for a
6      Linux GPU box (GCP / a SaaS worker): it runs the same verified ``wasb_infer.py``
core
7      that the WSL adapter runs, so the trajectory CSV is byte-identical ? but WITHOUT the WSL
8      transport:
9
10     * NO ``wslCommandMixin`` (no ``wsl -d <distro> bash -c``), NO ``conda activate`` (the
11     ``wasb`` env's python is invoked by path), NO ``/mnt/c`` path translation
12     (``to_wsl_mnt_path``). Paths are native; the video is already local on the box.
13     * ``device`` is ``cuda`` (default ? the WSL parity path), ``mps`` or ``cpu``; threaded
14     into ``wasb_infer.py --device`` (and so into the Hydra ``runner.device`` override).
15     * weights / repo / python / scratch come from the environment first
16     (``WASB_WEIGHTS`` / ``WASB_REPO_DIR`` / ``WASB_PYTHON`` / ``WASB_SCRATCH``), then
17     ``indexing.wasb.*`` config, then a default ? so a box needs no config edits.
18
19     It emits the identical ``Frame,Visibility,X,Y`` CSV (same ``{stem}__{vid12}_wasb.csv``
20     name as the WSL runner) and reuses the model-agnostic ``parse_trajectory_csv`` /
21     ``trajectory_to_action_windows`` from ``tracknet_runner``, so the trajectory hybrids and
22     the windowing brain are unchanged. Selected via the existing ``detector_impl`` seam
23     (``trajectory_hybrid._resolve_runner`` ? ``_build_native_runner``);
    ``detector_impl="native"``.
24
25     Acceptance gate (owner / GPU box): byte-parity ? the SAME clip through the WSL
runner
26     (Windows) and this runner (Linux, ``device=cuda``) yields an identical trajectory CSV
and
27     identical candidate windows. That parity is a hardware check (a real GPU + the ``wasb``
28     env), so it is verified by the owner on the box ? not in this offline, subprocess-mocked
29     test suite.
30     """
31     import os
32     import shutil
33     import logging
34     import subprocess
35     from dataclasses import dataclass
36     from typing import Any, Dict, List, Optional, Tuple
37
38     from backend.pipeline.detectors.base import DetectorRunner
39     # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical to
WSL/TrackNet.
40     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
41
42     logger = logging.getLogger(__name__)
43
44     # Path to the inference core we copy into the WASB repo's src/ (so it can import WASB's
45     # detectors/trackers/dataloaders + find configs/), exactly as the WSL adapter does.
46     _INFER_PY = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
47
48
49     @dataclass
50     class NativeWasbConfig:
51         """Resolved settings for a Linux-native WASB run.
```

```

53         Built via :meth:`from_indexing_cfg` with **environment overrides** taking precedence
54         over ``indexing.wasb.*`` config (the box sets env; no config edits needed).
55         ``device``
56         defaults to ``cuda`` (the WSL byte-parity path).
57         """
58         repo_dir: str = "~/models/WASB-SBDT" # native clone of nttcom/WASB-SBDT
59         python_bin: str = "python" # the `wasb` conda env python (invoked by
60         path, no activate)
61         weights_path: str = "" # REQUIRED (env WASB_WEIGHTS or
62         indexing.wasb.weights_path)
63         sport: str = "badminton"
64         device: str = "cuda" # cuda | mps | cpu
65         scratch_dir: str = "" # frame-extraction scratch (env); "" = next
66         to the output dir
67         timeout_sec: int = 1800 # 30 min; a hung GPU call is killed (0 = no
68         timeout)
69         keep_frames: bool = False # retain the decoded frame cache after
70         success (debug only)
71         stream_video: bool = False # fast path: decode in-process, no PNG
72         extraction (bit-identical)
73
74         @classmethod
75         def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "NativeWasbConfig":
76             w = (idx_cfg or {}).get("wasb", {}) or {}
77             env = os.environ.get
78             return cls(
79                 repo_dir=env("WASB_REPO_DIR") or w.get("repo_dir", "~/models/WASB-SBDT"),
80                 python_bin=env("WASB_PYTHON") or w.get("python_bin", "python"),
81                 weights_path=env("WASB_WEIGHTS") or w.get("weights_path", "") or "",
82                 sport=w.get("sport", "badminton"),
83                 device=env("WASB_DEVICE") or w.get("device", "cuda"),
84                 scratch_dir=env("WASB_SCRATCH") or env("RALLY_SCRATCH") or env("TMPDIR") or
85                 "",
86                 timeout_sec=int(w.get("timeout_sec", 1800)),
87                 keep_frames=bool(w.get("keep_frames", False)),
88                 stream_video=bool(w.get("stream_video", False)),
89             )
90
91         class NativeWasbRunner(DetectorRunner):
92             """WASB runner that executes natively on a Linux GPU box ? no WSL. See module
93             docstring."""
94
95             def __init__(self, cfg: NativeWasbConfig):
96                 self.cfg = cfg
97
98             # --- low-level native exec (the single place tests patch)
99             -----
100             def _run(self, argv: List[str], cwd: Optional[str] = None) ->
101             subprocess.CompletedProcess:
102                 logger.debug("native exec: %s (cwd=%s)", " ".join(argv), cwd)
103                 return subprocess.run(argv, cwd=cwd, capture_output=True, text=True,
104                                     timeout=(self.cfg.timeout_sec or None))
105
106             def _repo_src(self) -> str:
107                 return os.path.join(os.path.expanduser(self.cfg.repo_dir), "src")
108
109             def _copy_infer_script(self) -> bool:
110                 """Copy wasb_infer.py into the WASB repo src/ so it imports WASB modules + finds
111                 configs/."""
112                 repo_src = self._repo_src()
113                 try:
114                     os.makedirs(repo_src, exist_ok=True)
115                     shutil.copy(_INFER_PY, os.path.join(repo_src, "wasb_infer.py"))
116                 return True

```

```

106         except OSError as e:
107             logger.error("Failed to copy wasb_infer.py into %s: %s", repo_src, e)
108             return False
109
110     def _rm_rf(self, path: Optional[str]) -> None:
111         """Best-effort recursive delete of a native path (post-success cleanup; never
raises)."""
112         if path:
113             shutil.rmtree(path, ignore_errors=True)
114
115     def healthcheck(self) -> Tuple[bool, str]:
116         """Verify weights + repo present and the `wasb` python imports torch (and sees a
GPU on cuda)."""
117         if not self.cfg.weights_path:
118             return False, ("WASB weights path is empty ? set WASB_WEIGHTS (or
indexing.wasb.weights_path).")
119         weights = os.path.expanduser(self.cfg.weights_path)
120         if not os.path.exists(weights):
121             return False, f"WASB weights not found at {weights}"
122         repo_src = self._repo_src()
123         if not os.path.isdir(repo_src):
124             return False, (f"WASB-SBDT repo src/ not found at {repo_src} ? clone
nttcom/WASB-SBDT "
125                             f"(set WASB_REPO_DIR / indexing.wasb.repo_dir).")
126         code = "import torch; print('torch', torch.__version__, 'cuda',
torch.cuda.is_available())"
127         try:
128             res = self._run([self.cfg.python_bin, "-c", code])
129         except FileNotFoundError:
130             return False, f"python binary not found: {self.cfg.python_bin!r} (set
WASB_PYTHON)."
131         except subprocess.TimeoutExpired:
132             return False, "native torch healthcheck timed out."
133         out = (res.stdout or "").strip()
134         if res.returncode != 0:
135             return False, f"`{self.cfg.python_bin}` torch import failed: {(res.stderr or
out).strip()}"
136         if self.cfg.device == "cuda" and "cuda True" not in out:
137             return False, f"device=cuda but torch.cuda.is_available() is False ({out})"
138         return True, out
139
140     def run_predict(self, video_win_path: str, output_win_dir: str,
141                    video_id: Optional[str] = None) -> Optional[str]:
142         """Run WASB natively on a video. Returns the path to the trajectory CSV, or
None.
143
144         Output artifacts are namespaced by ``video_id`` (the file MD5) exactly as the
WSL
145         runner does, and the CSV is named ``{stem}__{vid12}_wasb.csv`` ? byte-for-byte
the
146         same downstream contract, so the parity comparison is apples-to-apples.
147         """
148         output_dir = os.path.abspath(output_win_dir)
149         os.makedirs(output_dir, exist_ok=True)
150         # Normalize for a cross-platform basename (tests pass Windows paths on Linux).
151         norm = str(video_win_path).replace("\\", "/")
152         stem, _ext = os.path.splitext(os.path.basename(norm))
153         # Content-derived key: same filename + different bytes -> different key (no
cache reuse).
154         key = f"{stem}__{video_id[:12]}" if video_id else (stem or "wasb")
155         out_csv = os.path.join(output_dir, f"{key}_wasb.csv")
156
157         if not self._copy_infer_script():
158             logger.error("Could not stage wasb_infer.py into the WASB repo src/.")
159             return None

```

```

160
161     weights = os.path.expanduser(self.cfg.weights_path)
162     argv = [self.cfg.python_bin, "wasb_infer.py",
163             "--video", os.path.abspath(str(video_win_path))]
164     frames_dir: Optional[str] = None
165     if self.cfg.stream_video:
166         # Fast path: decode in-process, no PNG extraction. Output is bit-identical
to disk.
167         argv.append("--stream-video")
168     else:
169         scratch = self.cfg.scratch_dir or output_dir
170         frames_dir = os.path.join(scratch, f"{key}_frames")
171         argv += ["--frames_out_dir", frames_dir]
172     argv += ["--weights", weights,
173             "--sport", self.cfg.sport,
174             "--device", self.cfg.device,
175             "--out", out_csv]
176
177     logger.info("Running native WASB inference (device=%s) on %s ...",
178               self.cfg.device, os.path.basename(norm))
179     try:
180         res = self._run(argv, cwd=self._repo_src())
181     except FileNotFoundError:
182         logger.error("python binary not found: %r (set WASB_PYTHON).",
self.cfg.python_bin)
183         return None
184     except subprocess.TimeoutExpired:
185         logger.error("WASB native inference timed out after %ss.",
self.cfg.timeout_sec)
186         return None
187     if res.returncode != 0:
188         logger.error("WASB native inference failed:\n%s", (res.stderr or res.stdout
or "")[-2000:])
189         return None
190     if not os.path.exists(out_csv):
191         logger.error("WASB native finished but expected CSV not found at %s",
out_csv)
192         return None
193     logger.info("WASB trajectory CSV: %s", out_csv)
194
195     # Success ? the CSV is the durable output; the decoded frame cache is a pure
intermediate.
196     # On any earlier failure we return above WITHOUT cleaning, so the cache survives
a resume.
197     # (Streaming never materializes frames, so there is nothing to drop on that
path.)
198     if not self.cfg.keep_frames and frames_dir:
199         logger.info("Cache hygiene: removing frame cache for %s "
200                   "(set indexing.wasb.keep_frames=true to retain).", key)
201         self._rm_rf(frames_dir)
202     return out_csv
203
204     # Trajectory parsing + windowing are inherited from tracknet_runner (model-
agnostic).
205     parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
206     trajectory_to_action_windows =
staticmethod(TrackNetRunner.trajectory_to_action_windows)
207

```

4. user (2026-06-20T08:57:15.577Z)

```

1     """WASB shuttle-trajectory runner (WSL2 subprocess adapter).
2
3     Windows-side counterpart to backend/detectors/wasb_infer.py. It:
4     1. stages the video into the WSL filesystem (fast local reads, not /mnt),

```

```

5         2. copies wasb_infer.py into the WASB repo's src/ dir,
6         3. runs it in the WSL `wasb` conda env -> a Frame,Visibility,X,Y trajectory CSV,
7         4. reuses the (model-agnostic) trajectory->action-window logic from tracknet_runner.
8
9     So the only WASB-specific bits live here; the windowing + CSV parsing are shared
10    with the TrackNet path. Verified inference core: wasb_infer.py (see its docstring).
11    """
12    import os
13    import shlex
14    import logging
15    import subprocess
16    from dataclasses import dataclass
17    from typing import Dict, Any, Optional, Tuple
18
19    # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical.
20    # parse_trajectory_csv / trajectory_to_action_windows are @staticmethods on
TrackNetRunner.
21    from backend.pipeline.detectors.tracknet_runner import to_wsl_mnt_path, TrackNetRunner
22    from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin,
wsl_tilde_quote
23
24    logger = logging.getLogger(__name__)
25
26    # Windows path to the inference wrapper we stage into WSL.
27    _INFER_WIN = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
28
29
30    @dataclass
31    class WasbConfig:
32        """Resolved settings for a WASB WSL run (built from config.json ->
indexing.wasb)."""
33        distro: str = "Ubuntu"
34        repo_dir: str = "~/models/WASB-SBDT"
35        conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
36        conda_env: str = "wasb"
37        weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
38        sport: str = "badminton"
39        wsl_stage_dir: str = "~/clips"
40        timeout_sec: int = 1800 # 30 min; a hung GPU/conda call is killed (0 = no timeout)
41        keep_frames: bool = False # retain staged video + frame cache after success (debug
only)
42        min_free_gb: float = 0.0 # warn if WSL free space below this before a run (0 =
off)
43        # FAST PATH (default OFF until verified): decode the staged video on the fly inside
44        # WSL and feed frames straight to the detector, skipping the slow per-frame PNG
45        # extraction that dominates a long clip. Output is bit-identical to the PNG path
46        # (PNG is lossless), but this stays opt-in until the owner's side-by-side confirms
it.
47        stream_video: bool = False
48
49    @classmethod
50    def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "WasbConfig":
51        w = (idx_cfg or {}).get("wasb", {}) or {}
52        return cls(
53            distro=w.get("wsl_distro", "Ubuntu"),
54            repo_dir=w.get("repo_dir", "~/models/WASB-SBDT"),
55            conda_sh=w.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
56            conda_env=w.get("conda_env", "wasb"),
57            weights_path=w.get("weights_path",
58                               "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"),
59            sport=w.get("sport", "badminton"),
60            wsl_stage_dir=w.get("wsl_stage_dir", "~/clips"),
61            timeout_sec=int(w.get("timeout_sec", 1800)),

```

```

62         keep_frames=bool(w.get("keep_frames", False)),
63         min_free_gb=float(w.get("min_free_gb", 0.0)),
64         stream_video=bool(w.get("stream_video", False)),
65     )
66
67
68 class WasbRunner(WslCommandMixin, DetectorRunner):
69     def __init__(self, cfg: WasbConfig):
70         self.cfg = cfg
71
72     def healthcheck(self) -> Tuple[bool, str]:
73         """Verify the WSL `wasb` env imports torch and sees a GPU. Returns (ok, msg)."""
74         cmd = (f"conda activate {self.cfg.conda_env} && "
75              f"python -c \"import torch; print('torch', torch.__version__, "
76              f"'cuda', torch.cuda.is_available())\"")
77         try:
78             res = self._wsl_bash(cmd)
79         except FileNotFoundError:
80             return False, "`wsl` not found ? Windows + WSL2 required."
81         except subprocess.TimeoutExpired:
82             return False, "WSL healthcheck timed out."
83         out = (res.stdout or "").strip()
84         if res.returncode != 0:
85             return False, f"WSL `wasb` env check failed: {(res.stderr or out).strip()}"
86         return True, out
87
88     def _stage_video(self, video_win_path: str, dest_name: str) -> Optional[str]:
89         src_mnt = to_wsl_mnt_path(video_win_path)
90         reason = self.wsl_source_unreadable_reason(src_mnt)
91         if reason:
92             logger.error("Cannot stage video into WSL: %s", reason)
93             return None
94         dest = f"{self.cfg.wsl_stage_dir}/{dest_name}"
95         cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)} "
96         {wsl_tilde_quote(dest)} && echo OK"
97         try:
98             res = self._wsl_bash(cmd)
99         except subprocess.TimeoutExpired:
100             logger.error("Staging video into WSL timed out.")
101             return None
102         if res.returncode != 0 or "OK" not in (res.stdout or ""):
103             logger.error("Failed to stage video into WSL: %s", (res.stderr or
104             res.stdout).strip())
105             return None
106         return dest
107
108     def _stage_infer_script(self) -> bool:
109         """Copy wasb_infer.py into the WASB repo src/ so it can import WASB modules."""
110         src_mnt = to_wsl_mnt_path(_INFER_WIN)
111         cmd = f"cp {shlex.quote(src_mnt)} {self.cfg.repo_dir}/src/wasb_infer.py && echo "
112         OK"
113         res = self._wsl_bash(cmd)
114         return res.returncode == 0 and "OK" in (res.stdout or "")
115
116     def run_predict(self, video_win_path: str, output_win_dir: str,
117                    video_id: Optional[str] = None) -> Optional[str]:
118         """Run WASB on a video. Returns the Windows path to the trajectory CSV, or None.
119
120         All WSL/cache/output artifacts are namespaced by ``video_id`` (the file MD5) so
121         two videos that share a filename (e.g. two ``match.mp4`` from different folders)
122         can never reuse each other's staged frames, resumable cache, or CSV.
123
124         """
125         # Resolve relative dirs (the hybrids pass "output/wasb") BEFORE the /mnt
126         # translation: to_wsl_mnt_path passes relative paths through unchanged, so
127         # WSL resolved them against the inference cwd (~/models/WASB-SBDT/src) and

```

```

124         # the CSV landed inside WSL while Windows polled the repo-relative path.
125         output_win_dir = os.path.abspath(output_win_dir)
126         os.makedirs(output_win_dir, exist_ok=True)
127         # Normalize for cross-platform basename (tests use Windows paths on Linux).
128         video_win_path = str(video_win_path).replace("\\", "/")
129         base = os.path.basename(video_win_path)
130         stem, ext = os.path.splitext(base)
131         # Unique, content-derived key: same filename + different bytes -> different key.
132         key = f"{stem}__{video_id[:12]}" if video_id else stem
133         wsl_out_dir = to_wsl_mnt_path(output_win_dir)
134         expected_csv_win = os.path.join(output_win_dir, f"{key}_wasb.csv")
135
136         if not self._stage_infer_script():
137             logger.error("Could not stage wasb_infer.py into WSL.")
138             return None
139         staged_video = self._stage_video(video_win_path, f"{key}{ext}")
140         if staged_video is None:
141             return None
142         frames_dir = f"{self.cfg.wsl_stage_dir}/{key}_frames"
143
144         # Disk guard: a 4K clip can extract ~100 GB of PNG frames. Warn early if the
145         # WSL stage filesystem is already low so the user can clean up before it fills.
146         # (Streaming never materializes the PNGs, so the guard only matters off the fast
147         path.)
148         if self.cfg.min_free_gb > 0 and not self.cfg.stream_video:
149             free = self._wsl_free_gb(self.cfg.wsl_stage_dir)
150             if free is not None and free < self.cfg.min_free_gb:
151                 logger.warning(
152                     "Low WSL disk: %.1f GB free at %s (< min_free_gb=%.1f). Frame
153                     extraction "
154                     "may fill the disk ? consider running tools/cleanup_caches.py.",
155                     free, self.cfg.wsl_stage_dir, self.cfg.min_free_gb)
156
157         # Fast path: stream-decode the video in-process (no PNG extraction). Output is
158         # bit-identical to the PNG round-trip but avoids writing ~46k PNGs for a 12-min
159         clip.
160         if self.cfg.stream_video:
161             frame_args = "--stream-video "
162         else:
163             frame_args = f"--frames_out_dir {wsl_tilde_quote(frames_dir)} "
164         cmd = (
165             f"conda activate {self.cfg.conda_env} && cd {self.cfg.repo_dir}/src && "
166             f"python wasb_infer.py "
167             f"--video {wsl_tilde_quote(staged_video)} "
168             f"{frame_args}"
169             f"--weights {wsl_tilde_quote(self.cfg.weights_path)} "
170             f"--sport {shlex.quote(self.cfg.sport)} "
171             f"--out {wsl_tilde_quote(wsl_out_dir + '/' + key + '_wasb.csv')}"
172         )
173         logger.info("Running WASB inference on %s ...", base)
174         try:
175             res = self._wsl_bash(cmd)
176         except subprocess.TimeoutExpired:
177             logger.error("WASB inference timed out after %ss.", self.cfg.timeout_sec)
178             return None
179         if res.returncode != 0:
180             logger.error("WASB inference failed:\n%s", (res.stderr or
181                 res.stdout)[-2000:])
182             return None
183         if not os.path.exists(expected_csv_win):
184             logger.error("WASB finished but expected CSV not found at %s",
185                 expected_csv_win)
186             return None
187         logger.info("WASB trajectory CSV: %s", expected_csv_win)

```

```

184         # Success ? the trajectory CSV is the durable output; the staged video copy and
185         # the (huge) decoded frame cache are now pure intermediates. Delete them unless
186         # explicitly retained. On any earlier failure we return above WITHOUT cleaning,
187         # so the frame cache + detector cache remain for a resumable re-run.
188         # (Streaming never writes a frame cache, so there's only the staged video to
drop.)
189         if not self.cfg.keep_frames:
190             stale = [staged_video] if self.cfg.stream_video else [frames_dir,
staged_video]
191             logger.info("Cache hygiene: removing staged video%s for %s "
192                         "(set indexing.wasb.keep_frames=true to retain).",
193                         "" if self.cfg.stream_video else " + frame cache", key)
194             self._wsl_rm_rf(stale)
195             return expected_csv_win
196
197         # Trajectory parsing + windowing are inherited from tracknet_runner (model-
agnostic).
198         parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
199         trajectory_to_action_windows =
staticmethod(TrackNetRunner.trajectory_to_action_windows)
200

```

5. assistant (2026-06-20T08:57:19.939Z)

Let me read the rest of the changed files in the working tree.

6. user (2026-06-20T08:57:20.905Z)

```

1     """Abstract base for detector runners (TrackNet, WASB, future native, etc.).
2
3     This completes the Detector Runner Abstraction (low-regret item 6 continuation / item 1
4     of the post-observability re-prioritization) to de-risk the WSL/conda dependency for
5     the high-accuracy shuttle trajectory substrate used by tracknet_hybrid / wasb_hybrid.
6
7     The ABC lets us:
8     - Keep the current WSL adapters working unchanged for Windows users.
9     - Swap in (or A/B test) a Linux-native implementation later with zero changes to
10        the hybrid segmenters or the model-agnostic windowing brain.
11     - Provide a single place to evolve the healthcheck / error contract and thread
12        detector readiness into per-video reports.
13
14     Contract (deliberately matches what the concrete runners + hybrids + tests already did):
15     - run_predict(video_win_path: str, output_win_dir: str) -> Optional[str]
16       Run detector, return the Windows-side path to the produced trajectory CSV
17       (e.g. ".../clip_predict.csv" or ".../clip_wasb.csv"), or None on any failure.
18       The caller (hybrid) is responsible for output dir; impls may stage the video
19       into WSL fs for perf.
20
21     - healthcheck() -> tuple[bool, str]
22       Return (ok, message). Never raises. "ok" decides whether to proceed to run_predict.
23       Message is logged (and can be surfaced to reports or UX in future).
24       Intended to be cheap (env existence, import, GPU visibility, weights present).
25
26     Existing behavior is 100% preserved. The only additions are the inheritance and
27     the explicit common type for future implementers.
28
29     How to add a new runner (for docs / future contributors):
30     1. Subclass DetectorRunner in a new or existing module under pipeline/detectors/.
31     2. Implement run_predict and healthcheck (return (bool, str) tuple).
32     3. If it produces the same CSV schema (Frame,Visibility,X,Y), reuse or re-export
33        TrackNetRunner.parse_trajectory_csv and .trajectory_to_action_windows (they are
34        deliberately static and model-agnostic). If the new detector needs different
35        windowing, it can implement its own or we generalize later.
36     4. Lazy-import heavy native deps inside the methods (so importing the segmenter
37        registry never pulls torch/tf on a machine that doesn't have them).

```

```

38 5. Update the two hybrid segmenters (or introduce a detector= choice) to accept
39     DetectorRunner instances (they already do the right thing via duck-typing + the
40     type hints we add here).
41 6. Add unit tests that exercise the contract (isinstance, healthcheck shape, run
42     returns str|None) using mocks for the external bits ? see test_tracknet_runner.py.
43 7. No change to eval/calibration code paths (they consume the CSV + the static
44     windowing helpers directly).
45
46 Optional Linux-native path (the main de-risk goal):
47 A LinuxNativeRunner (or ONNXRunner) would:
48 - Skip all wsl / _stage / conda activate.
49 - Load weights locally (torch or onnx) or call a native binary.
50 - Write the exact same CSV format so trajectory_to_action_windows works verbatim.
51 - healthcheck can just check "weights file exists and torch importable + CUDA?".
52 This removes the WSL requirement for Linux users / future SaaS workers while
53 keeping the exact same candidate quality characteristics (or better, once native
54 weights are available).
55
56 Error surfacing into reports:
57 The healthcheck message is already logged by the hybrids on failure. The reporting
58 harvest sees the outcome as "0 candidates" + detector name. When we wire richer
59 segmenter Results (or pass detector_health into emit_indexer_report context), the
60 (ok, msg) can be recorded explicitly under segmentation.details or a new detector
61 stage. The ABC makes that future change local to one place.
62
63 See also:
64 - backend/pipeline/segmenters/{tracknet_hybrid,wasb_hybrid}.py (the two callers)
65 - tests/test_tracknet_runner.py (contract + pure logic tests; hybrids mock the runner)
66 - docs/TRACKNET_WSL_SETUP.md and CODE_MAP.md (2.2 Detectors section)
67 """"
68
69 import logging
70 import shlex
71 import subprocess
72 from abc import ABC, abstractmethod
73 from typing import Any, Callable, List, Optional, Tuple
74
75 logger = logging.getLogger(__name__)
76
77
78 def wsl_tilde_quote(path: str) -> str:
79     """shlex.quote that still lets bash expand a leading `~/`.
80
81     ``shlex.quote("~/clips/x.mp4")`` single-quotes the whole path, so bash
82     treats ``~`` literally and ``cp``/``mkdir`` fail with "No such file or
83     directory" ? this broke live WSL staging for the default ``~/clips``
84     stage dir (found by the 2026-06-11 wasb_hybrid smoke run; the BACKLOG
85     "shlex-quote breaks ~ expansion" item). Quoting only the part after
86     ``~/`` keeps expansion AND protects spaces/specials in the remainder.
87     """
88     if path == "~":
89         return path
90     if path.startswith("~/"):
91         return "~/ " + shlex.quote(path[2:])
92     return shlex.quote(path)
93
94
95 class WslCommandMixin:
96     """Shared ``wsl -d <distro> bash -c 'source <conda_sh> && <cmd>'`` invocation.
97
98     Lives beside (not on) DetectorRunner: a future Linux-native runner must not
99     inherit WSL plumbing. Requires ``self.cfg`` with ``distro``, ``conda_sh``
100     and ``timeout_sec`` attributes (TrackNetConfig and WasbConfig both qualify).
101     ``timeout_sec == 0`` maps to None = wait forever (configs default 1800s).
102     """

```

```

103
104     cfg: Any
105
106     def _wsl_bash(self, inner_cmd: str) -> subprocess.CompletedProcess:
107         full = f"source {self.cfg.conda_sh} && {inner_cmd}"
108         argv = ["wsl", "-d", self.cfg.distro, "bash", "-c", full]
109         logger.debug("WSL exec: %s", full)
110         return subprocess.run(argv, capture_output=True, text=True,
111                               timeout=(self.cfg.timeout_sec or None))
112
113     # ---- cache hygiene helpers (shared by both WSL runners) -----
114     def _wsl_free_gb(self, wsl_path: str) -> Optional[float]:
115         """Best-effort free space (GB) on the WSL filesystem holding ``wsl_path``.
116
117         Returns None if it can't be determined (never raises). Used as a pre-run
118         disk guard so a long extraction doesn't fill the disk mid-flight.
119         """
120         try:
121             res = self._wsl_bash(f"df -P -B1 {wsl_tilde_quote(wsl_path)} | awk
122 'NR==2{{print $4}}'")
123             except (subprocess.TimeoutExpired, OSError):
124                 return None
125             try:
126                 return int((res.stdout or "").strip()) / (1024 ** 3)
127             except (ValueError, AttributeError):
128                 return None
129
130     def _wsl_rm_rf(self, wsl_paths: List[str]) -> None:
131         """Best-effort recursive delete of WSL paths (post-success cleanup; never
132 raises)."""
133         paths = [p for p in wsl_paths if p]
134         if not paths:
135             return
136         quoted = " ".join(wsl_tilde_quote(p) for p in paths)
137         try:
138             self._wsl_bash(f"rm -rf {quoted}")
139         except (subprocess.TimeoutExpired, OSError) as e:
140             logger.warning("WSL cache cleanup failed (non-fatal): %s", e)
141
142     def wsl_source_unreadable_reason(self, src_mnt: str) -> Optional[str]:
143         """Actionable message if ``src_mnt`` is NOT readable from inside WSL, else None.
144
145         Google Drive (``G:``/Drive File Stream), UNC and many network/virtual
146         filesystems are invisible to WSL's ``/mnt`` ? staging a video from such a path
147         fails with a cryptic ``cp: cannot stat`` and the whole run silently produces no
148         trajectory (i.e. "0 rallies", a wasted run that looks like a quality result).
149         A cheap ``test -r`` preflight turns that into one clear, fixable error BEFORE
150         any
151         frame extraction. Best-effort: a flaky/timed-out probe returns None so the real
152         ``cp`` still gets its chance (we never block a genuinely-readable source).
153         """
154         try:
155             res = self._wsl_bash(f"test -r {shlex.quote(src_mnt)} && echo READABLE")
156         except (subprocess.TimeoutExpired, OSError):
157             return None
158         if "READABLE" in (res.stdout or ""):
159             return None
160         return (
161             f"video source is not readable from WSL at '{src_mnt}'. Google Drive, UNC, "
162             f"and other network/virtual filesystems are invisible to WSL's /mnt ? copy "
163             f"the video to a local disk (e.g. C:\\ or the WSL filesystem) and retry."
164         )
165
166     class DetectorRunner(ABC):

```

```

165     """Common interface for shuttle/ball trajectory detectors (WSL or otherwise)."""
166
167     @abstractmethod
168     def run_predict(self, video_win_path: str, output_win_dir: str,
169                    video_id: Optional[str] = None) -> Optional[str]:
170         """Run inference and return path to trajectory CSV (or None on failure).
171
172         ``video_id`` (the file MD5) namespaces all staged/cached/output artifacts so two
173         videos that happen to share a filename cannot reuse each other's frames or CSV.
174         It is optional for back-compat; callers in the live pipeline always pass it.
175         """
176         pass
177
178     @abstractmethod
179     def healthcheck(self) -> Tuple[bool, str]:
180         """Quick check if the runner (env, weights, etc.) is usable.
181
182         Returns (ok, message). Never raises. Cheap to call before a long run.
183         """
184         pass
185
186     # De facto trajectory contract: the trajectory hybrids call these
187     # unconditionally after run_predict. Concrete runners reuse/re-export the
188     # model-agnostic TrackNetRunner staticmethods (same CSV schema + windowing);
189     # a runner with custom windowing supplies its own with the same shapes.
190     parse_trajectory_csv: Callable[..., Any]
191     trajectory_to_action_windows: Callable[..., Any]
192

```

7. user (2026-06-20T08:57:21.976Z)

```

1     """TrackNet shuttle-trajectory runner (WSL2 subprocess adapter).
2
3     This bridges the Windows-side indexer to a TensorFlow TrackNetV4 install living
4     inside WSL2 (ADR-001 in docs/DECISIONS.md). The flow is:
5
6     1. Translate the Windows video path to its WSL-visible form (and optionally
7       stage it into the WSL filesystem to avoid slow /mnt/c per-frame reads).
8     2. Invoke ``src/predict.py`` inside the TrackNetV4 conda env via ``wsl``.
9     3. Read back the per-frame ``Frame,Visibility,X,Y`` CSV it writes.
10    4. Convert that trajectory into high-motion "action windows" ? the same
11       candidate-rally shape HybridYoloSegmenter produces ? so the rest of the
12       pipeline (AI handoff, DB population) is unchanged.
13
14    Nothing here imports TensorFlow; all model code stays in WSL.
15    """
16
17    import os
18    import csv
19    import shlex
20    import logging
21    import subprocess
22    from dataclasses import dataclass
23    from typing import List, Dict, Any, Optional, Tuple
24
25    from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin,
wsl_tilde_quote
26
27    logger = logging.getLogger(__name__)
28
29
30    @dataclass
31    class TrackNetConfig:
32        """Resolved settings for a TrackNet WSL run (built from config.json ->
indexing.tracknet)."""

```

```

33     distro: str = "Ubuntu"
34     repo_dir: str = "~/models/TrackNetV4"           # WSL path to the cloned repo
35     conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
36     conda_env: str = "TrackNetV4"
37     weights_path: str = ""                           # WSL path to the .h5/.keras weights
(REQUIRED)
38     queue_length: int = 5
39     stage_in_wsl: bool = True                         # copy video into WSL fs first
(perf)
40     wsl_stage_dir: str = "~/clips"                   # where staged videos/outputs live
in WSL
41     timeout_sec: int = 1800                          # 30 min; kills a hung call (0 = no
timeout)
42     keep_frames: bool = False                       # retain staged video after success
(debug only)
43     min_free_gb: float = 0.0                        # warn if WSL free space below this
(0 = off)
44
45     @classmethod
46     def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "TrackNetConfig":
47         tn = (idx_cfg or {}).get("tracknet", {}) or {}
48         return cls(
49             distro=tn.get("wsl_distro", "Ubuntu"),
50             repo_dir=tn.get("repo_dir", "~/models/TrackNetV4"),
51             conda_sh=tn.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
52             conda_env=tn.get("conda_env", "TrackNetV4"),
53             weights_path=tn.get("weights_path", ""),
54             queue_length=int(tn.get("queue_length", 5)),
55             stage_in_wsl=bool(tn.get("stage_in_wsl", True)),
56             wsl_stage_dir=tn.get("wsl_stage_dir", "~/clips"),
57             timeout_sec=int(tn.get("timeout_sec", 1800)),
58             keep_frames=bool(tn.get("keep_frames", False)),
59             min_free_gb=float(tn.get("min_free_gb", 0.0)),
60         )
61
62
63     def to_wsl_mnt_path(win_path: str) -> str:
64         """Translate a Windows path (C:\\Users\\x) to its WSL /mnt form (/mnt/c/Users/x).
65
66         Already-POSIX paths (starting with / or ~) are returned unchanged so the
67         same helper is safe to call on either kind of path.
68         """
69         p = str(win_path)
70         if p.startswith("/") or p.startswith("~"):
71             return p
72         p = p.replace("\\", "/")
73         if len(p) >= 2 and p[1] == ":":
74             drive = p[0].lower()
75             return f"/mnt/{drive}{p[2:]}"
76         return p
77
78
79     @dataclass
80     class TrajectoryPoint:
81         frame: int
82         visible: bool
83         x: float
84         y: float
85
86
87     class TrackNetRunner(WSLCommandMixin, DetectorRunner):
88         """Runs TrackNet predict.py in WSL and turns the output CSV into action windows.
89
90         Implements the common DetectorRunner contract so a future Linux-native or
91         alternative trajectory runner can be substituted without touching the hybrids.

```

```

92     """
93
94     def __init__(self, cfg: TrackNetConfig):
95         self.cfg = cfg
96
97     def healthcheck(self) -> Tuple[bool, str]:
98         """Verify the WSL env is usable: conda env exists, TF imports, GPU visible.
99
100         Returns (ok, message). Cheap to call before a long run.
101         """
102         cmd = (
103             f"conda activate {self.cfg.conda_env} && "
104             f"python -c \"import tensorflow as tf; "
105             f"print('TF', tf.__version__, 'GPUs', "
106             f"len(tf.config.list_physical_devices('GPU')))\n"
107         )
108         try:
109             res = self._wsl_bash(cmd)
110             except FileNotFoundError:
111                 return False, "`wsl` not found ? is this running on Windows with WSL2
112                 installed?"
113             except subprocess.TimeoutExpired:
114                 return False, "WSL healthcheck timed out."
115             out = (res.stdout or "").strip()
116             if res.returncode != 0:
117                 return False, f"WSL env check failed: {(res.stderr or out).strip()}"
118             return True, out
119
120     # ----- inference
121
122     def run_predict(self, video_win_path: str, output_win_dir: str,
123                    video_id: Optional[str] = None) -> Optional[str]:
124         """Run predict.py on a video. Returns the Windows path to the produced CSV, or
125         None.
126
127         ``output_win_dir`` is a Windows directory (under the repo's output/) that
128         WSL writes into via its /mnt view, so the Windows process can read the CSV
129         back with no copy. ``video_id`` (file MD5) namespaces the staged video ? and
130         therefore predict.py's ``<stem>_predict.csv`` output ? so two videos that share
131         a filename cannot collide on the output CSV.
132         """
133         if not self.cfg.weights_path:
134             logger.error(
135                 "TrackNet weights_path is not set. Set indexing.tracknet.weights_path "
136                 "in config.json to the WSL path of your trained TrackNetV4 weights."
137             )
138             return None
139
140         # Resolve relative dirs BEFORE the /mnt translation ? to_wsl_mnt_path passes
141         # relative paths through unchanged, so WSL would resolve them against the
142         # predict.py cwd instead of the repo (same bug class as wasb_runner).
143         output_win_dir = os.path.abspath(output_win_dir)
144         os.makedirs(output_win_dir, exist_ok=True)
145         # Normalize separators so basename/splitext work when tests (or collector)
146         # pass Windows-style paths on a Linux host.
147         video_win_path = str(video_win_path).replace("\\", "/")
148         base = os.path.basename(video_win_path)
149         stem, ext = os.path.splitext(base)
150
151         # predict.py only accepts .avi/.mp4 and names the CSV "<stem>_predict.csv".
152         if ext.lower() not in (".mp4", ".avi"):
153             logger.error("TrackNet predict.py only supports .mp4/.avi (got %s)", ext)
154             return None
155
156         wsl_out = to_wsl_mnt_path(output_win_dir)

```

```

154         # Namespace by video_id so two same-filename videos don't collide on the CSV.
155         # predict.py derives its output name from the (staged) video stem, so staging
156         # under the namespaced name propagates into "<key>_predict.csv".
157         key = f"{stem}__{video_id[:12]}" if video_id else stem
158         expected_csv_win = os.path.join(output_win_dir, f"{key}_predict.csv")
159
160         # Resolve the video path WSL will read.
161         if self.cfg.stage_in_wsl:
162             staged = self._stage_video(video_win_path, f"{key}{ext}")
163             if staged is None:
164                 return None
165             wsl_video = staged
166         else:
167             # No staging: predict.py reads /mnt directly and writes
168             # We cannot rename its output, so fall back to the un-namespaced name.
169             wsl_video = to_wsl_mnt_path(video_win_path)
170             expected_csv_win = os.path.join(output_win_dir, f"{stem}_predict.csv")
171
172         if self.cfg.min_free_gb > 0:
173             free = self._wsl_free_gb(self.cfg.wsl_stage_dir)
174             if free is not None and free < self.cfg.min_free_gb:
175                 logger.warning("Low WSL disk: %.1f GB free at %s (< min_free_gb=%.1f); "
176                               "consider running tools/cleanup_caches.py.",
177                               free, self.cfg.wsl_stage_dir, self.cfg.min_free_gb)
178
179         cmd = (
180             f"conda activate {self.cfg.conda_env} && "
181             f"cd {self.cfg.repo_dir}/src && "
182             f"python predict.py "
183             f"--video_path {wsl_tilde_quote(wsl_video)} "
184             f"--model_weights {wsl_tilde_quote(self.cfg.weights_path)} "
185             f"--output_dir {wsl_tilde_quote(wsl_out)} "
186             f"--queue_length {self.cfg.queue_length}"
187         )
188         logger.info("Running TrackNet inference on %s ...", base)
189         try:
190             res = self._wsl_bash(cmd)
191         except subprocess.TimeoutExpired:
192             logger.error("TrackNet inference timed out after %ss.",
193                         self.cfg.timeout_sec)
194             return None
195
196         if res.returncode != 0:
197             logger.error("TrackNet predict.py failed:\n%s", (res.stderr or
198                     res.stdout)[-2000:])
199             return None
200
201         if not os.path.exists(expected_csv_win):
202             logger.error("TrackNet finished but expected CSV not found at %s",
203                         expected_csv_win)
204             return None
205         logger.info("TrackNet trajectory CSV: %s", expected_csv_win)
206
207         # Success ? the CSV is the durable output; drop the staged video copy (a pure,
208         # regenerable intermediate). On earlier failure we return above without
209         cleaning.
210         if self.cfg.stage_in_wsl and not self.cfg.keep_frames:
211             logger.info("Cache hygiene: removing staged video for %s "
212                         "(set indexing.tracknet.keep_frames=true to retain).", key)
213             self._wsl_rm_rf([wsl_video])
214         return expected_csv_win
215
216 def _stage_video(self, video_win_path: str, base: str) -> Optional[str]:
217     """Copy the input video into the WSL filesystem (avoids slow /mnt/c reads).

```

```

214
215         Returns the WSL path to the staged copy, or None on failure.
216         """
217         src_mnt = to_wsl_mnt_path(video_win_path)
218         reason = self.wsl_source_unreadable_reason(src_mnt)
219         if reason:
220             logger.error("Cannot stage video into WSL: %s", reason)
221             return None
222         dest = f"{self.cfg.wsl_stage_dir}/{base}"
223         cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}"
224         {wsl_tilde_quote(dest)} && echo OK"
225         try:
226             res = self._wsl_bash(cmd)
227         except subprocess.TimeoutExpired:
228             logger.error("Staging video into WSL timed out.")
229             return None
230         if res.returncode != 0 or "OK" not in (res.stdout or ""):
231             logger.error("Failed to stage video into WSL: %s", (res.stderr or
232 res.stdout).strip())
233             return None
234         return dest
235
236 # ----- CSV -> windows
237
238 @staticmethod
239 def parse_trajectory_csv(csv_win_path: str) -> List[TrajectoryPoint]:
240     """Parse a TrackNet predict CSV (columns: Frame,Visibility,X,Y)."""
241     points: List[TrajectoryPoint] = []
242     with open(csv_win_path, newline="") as f:
243         reader = csv.DictReader(f)
244         for row in reader:
245             try:
246                 points.append(TrajectoryPoint(
247                     frame=int(row["Frame"]),
248                     visible=int(row["Visibility"]) == 1,
249                     x=float(row["X"]),
250                     y=float(row["Y"]),
251                 ))
252             except (KeyError, ValueError):
253                 continue
254     points.sort(key=lambda p: p.frame)
255     return points
256
257 @staticmethod
258 def _active_frames(points: List[TrajectoryPoint], fps: float, frame_width: float,
259 velocity_thresh: float) -> List[Tuple[int, float]]:
260     """Per-frame ``(frame_idx, velocity)`` for frames where the shuttle is in
261 flight.
262     Computes each visible point's normalised velocity from the last *visible* point;
263 an absent shuttle breaks the trajectory. A frame qualifies when its velocity
264 exceeds
265 ``velocity_thresh``. ``fps``/``frame_width`` are expected to be already
266 defaulted by
267 the caller so identical floats feed the velocity math."""
268     active = [] # (frame_idx, velocity)
269     prev: Optional[TrajectoryPoint] = None
270     for p in points:
271         if not p.visible:
272             prev = None # break the trajectory; absent shuttle = not in flight
273             continue
274         if prev is not None:
275             dframes = p.frame - prev.frame
276             if dframes > 0:
277                 dist = ((p.x - prev.x) ** 2 + (p.y - prev.y) ** 2) ** 0.5

```

```

274         velocity = (dist / frame_width) * (fps / dframes)
275         if velocity > velocity_thresh:
276             active.append((p.frame, velocity))
277     prev = p
278     return active
279
280     @staticmethod
281     def trajectory_to_action_windows(
282         points: List[TrajectoryPoint],
283         fps: float,
284         frame_width: float,
285         chunk_start: float = 0.0,
286         velocity_thresh: float = 0.02,
287         merge_gap: float = 2.0,
288         min_window_duration: float = 2.0,
289         chunk_index: Optional[int] = None,
290         max_window_duration: float = 0.0,
291         min_split_gap: float = 0.5,
292         window_hard_cap: bool = False,
293         window_inactivity_end: float = 0.0,
294         inactivity_min_density: float = 0.34,
295         inactivity_rally_thresh: float = 0.08,
296         window_blob_fallback: bool = False,
297         window_blob_factor: float = 4.0,
298     ) -> List[Dict[str, Any]]:
299         """Convert a shuttle trajectory into candidate rally windows.
300
301         A frame is "active" when the shuttle is visible AND its inter-frame
302         displacement (normalised by frame width, per second) exceeds
303         ``velocity_thresh`` ? i.e. the shuttle is in flight. Active frames within
304         ``merge_gap`` seconds of each other are grouped into one window; short
305         windows are dropped. Mirrors HybridYoloSegmenter's windowing so the dict
306         shape feeding the AI-handoff phase is identical.
307
308         ``max_window_duration`` (0 = OFF, the default ? behaviour unchanged): a guard
309         against the *over-merge* failure where "shuttle moving" is conflated with "rally in
310         progress" and a single window swallows several rallies + the dead time between them. When
311         set, any window longer than this is recursively SPLIT at its largest internal inactivity
312         gap (the longest stretch with no in-flight shuttle ? the most likely rally boundary),
313         provided that gap is at least ``min_split_gap`` seconds. This is the bounded-windowing
314         fix from ``docs/RALLY_QUALITY_RESEARCH.md`` §6 (W1); it never merges, only cuts, so
315         recall cannot drop, and it is config-gated default-OFF until measured on the golden set.
316
317         ``window_hard_cap`` (default OFF) makes ``max_window_duration`` a TRUE cap: when
318         an over-long window has NO internal inactivity gap to split on (a continuously-
319         active trajectory ? the shuttle never stops moving, e.g. the real-footage
320         ``mahadevpura-1`` blob where the gap-splitter alone left a single 174 s window), it is recursively
321         hard-chopped at its temporal midpoint until every piece is ? the cap. Still only
322         cuts (never merges), so recall cannot drop; gated default-OFF until measured.
323
324         ``window_blob_fallback`` (default OFF ? the R5 last-resort blob splitter):
325         unlike ``window_hard_cap`` (which midpoint-chops BEFORE the R4 trim, so it fires on
326         every gap-less

```

```

326         over-cap window and over-segments normal clips), this runs AFTER the R4
inactivity trim and
327         force-chops ONLY a genuine *blob* ? a group still longer than
``window_blob_factor`` ×
328         ``max_window_duration`` once the gap-split and the principled rally-end trim
have both had
329         their chance (e.g. mahadevpura-1's ~65 s residual ? 11× a 6 s cap; a 7 s rally
is left
330         alone). The blob is midpoint-chopped down to ? the cap, those forced cuts are
logged, and
331         each resulting window is flagged ``forced_cut=True`` ? surfacing the clip as one
that needs
332         rally-STATE cues (net-crossings / two-sided motion), not a bigger cap. Only cuts
(recall
333         can't drop). Requires ``max_window_duration`` > 0; gated default-OFF until
measured.
334         """
335         if fps <= 0:
336             fps = 30.0
337         if frame_width <= 0:
338             frame_width = 1280.0
339
340         active = TrackNetRunner._active_frames(points, fps, frame_width,
velocity_thresh)
341
342         if not active:
343             return []
344
345         max_gap_frames = int(merge_gap * fps)
346
347         def _finalize(group: List[Tuple[int, float]]) -> Dict[str, Any]:
348             vels = [v for _, v in group]
349             return {
350                 "start": group[0][0] / fps + chunk_start,
351                 "end": group[-1][0] / fps + chunk_start,
352                 "active_frame_count": len(group),
353                 "peak_velocity": max(vels),
354                 "mean_velocity": sum(vels) / len(vels),
355                 "track_id_count": 1, # single shuttle; kept for window-shape parity
356                 "chunk_index": chunk_index,
357             }
358
359         groups: List[List[Tuple[int, float]]] = []
360         group = [active[0]]
361         for af in active[1:]:
362             if af[0] - group[-1][0] <= max_gap_frames:
363                 group.append(af)
364             else:
365                 groups.append(group)
366                 group = [af]
367         groups.append(group)
368
369         if max_window_duration > 0:
370             groups = TrackNetRunner._split_overlong_groups(
371                 groups, int(max_window_duration * fps), int(min_split_gap * fps),
372                 hard_cap=window_hard_cap)
373
374         # R4 ? rally-END inactivity trim (default OFF). Applied AFTER the cap split so
each rally
375         # piece has its own post-rally drift tail removed: the velocity windowing keeps
a window
376         # "active" while the shuttle drifts/gets-picked-up above threshold, over-
extending the END
377         # (the measured gap vs golden). Trim back to the last frame whose trailing
window is still

```

```

378         # dense with motion (rally on); a sustained low-density run = rally over. Only
cuts.
379         if window_inactivity_end > 0:
380             tw = int(window_inactivity_end * fps)
381             groups = [TrackNetRunner._trim_inactive_tail(g, tw, inactivity_min_density,
382                                                             inactivity_rally_thresh) for g
in groups]
383
384         # R5 ? blob-fallback (default OFF). LAST resort: after the gap-split AND the R4
trim, any
385         # group still longer than the cap is a gap-less, rally-end-signal-less blob;
force-chop it to
386         # ? the cap at the midpoint and flag every piece, so the degenerate single-
window outcome is
387         # avoided and the clip is surfaced (logged) as needing rally-STATE cues, not a
bigger cap.
388         forced_flags = [False] * len(groups)
389         if window_blob_fallback and max_window_duration > 0:
390             groups, forced_flags = TrackNetRunner._blob_fallback_chop(
391                 groups, int(max_window_duration * fps), window_blob_factor)
392
393         windows = []
394         for g, forced in zip(groups, forced_flags):
395             w = _finalize(g)
396             if forced:
397                 w["forced_cut"] = True
398             windows.append(w)
399         return [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
400
401     @staticmethod
402     def _trim_inactive_tail(group: List[Tuple[int, float]], trail_frames: int,
403                             min_density: float, rally_thresh: float) -> List[Tuple[int,
float]]:
404         """R4: trim the post-rally drift tail. After a rally the shuttle keeps moving
*slowly*
405         (picked up / walked / knock-up) above ``velocity_thresh``, so the velocity
windowing
406         over-extends the END. A frame is "fast" (real rally motion) iff its velocity >
``rally_thresh``
407         (higher than the active threshold). The rally is "on" at frame ``i`` while its
trailing
408         ``trail_frames`` window holds >= ``min_density`` fraction of fast frames; trim
back to the
409         LAST such frame ? everything after is sustained slow drift (rally over). Only
cuts (recall
410         can't rise); a window whose tail never goes slow is untouched, and if no fast-
dense point
411         exists at all the group is kept whole (fail-safe, no over-trim). O(n) two-
pointer; the START
412         is left alone (already ~Gemini-accurate per the n=2 boundary-error finding)."""
413         if trail_frames <= 0 or len(group) < 2:
414             return group
415         frames = [f for f, _ in group]
416         fast = [1 if v > rally_thresh else 0 for _, v in group]
417         lo = 0
418         fast_sum = 0
419         last_dense = -1
420         for i in range(len(frames)):
421             fast_sum += fast[i]
422             while frames[lo] <= frames[i] - trail_frames:
423                 fast_sum -= fast[lo]
424                 lo += 1
425             win_count = i - lo + 1
426             if win_count > 0 and fast_sum / win_count >= min_density:
427                 last_dense = i

```

```

428         return group[: last_dense + 1] if last_dense >= 0 else group
429
430     @staticmethod
431     def _midpoint_split_index(group: List[Tuple[int, float]]) -> int:
432         """Index of the active frame nearest the group's temporal midpoint (used to chop
433         a
434         gap-less group into two). Shared by the W1 hard-cap fallback and the R5 blob-
435         fallback."""
436         mid = (group[0][0] + group[-1][0]) / 2.0
437         split_i = min(range(1, len(group)), key=lambda i: abs(group[i][0] - mid))
438         return split_i
439
440     @staticmethod
441     def _split_overlong_groups(groups: List[List[Tuple[int, float]]], max_win_frames:
442     int,
443     min_split_frames: int,
444     hard_cap: bool = False) -> List[List[Tuple[int, float]]]:
445         """Recursively split any active-frame group longer than ``max_win_frames`` at
446         its largest
447         internal gap (? ``min_split_frames``) ? an inactivity-aware cut at the most
448         likely rally
449         boundary. Splitting only subdivides existing groups (never merges/extends), so
450         it cannot
451         invent or drop coverage; a group with no gap ? the floor is left intact (a
452         genuinely long
453         rally) UNLESS ``hard_cap`` is set, in which case such a group is hard-chopped at
454         its
455         temporal midpoint until every piece is ? ``max_win_frames`` (makes the cap a
456         true upper
457         bound on continuously-active trajectories). Iterative worklist to bound stack
458         depth."""
459         if max_win_frames <= 0:
460             return groups
461         out: List[List[Tuple[int, float]]] = []
462         work = list(groups)
463         while work:
464             g = work.pop()
465             if len(g) < 2 or (g[-1][0] - g[0][0]) <= max_win_frames:
466                 out.append(g)
467                 continue
468             best_i, best_gap = -1, -1
469             for i in range(1, len(g)):
470                 gap = g[i][0] - g[i - 1][0]
471                 if gap > best_gap:
472                     best_gap, best_i = gap, i
473             if best_i < 0 or best_gap < min_split_frames:
474                 # Too long but no real dead-time gap. Default: keep as one (a genuine
475                 long rally).
476                 # hard_cap: chop at the temporal midpoint so the cap is actually
477                 enforced.
478                 if hard_cap:
479                     split_i = TrackNetRunner._midpoint_split_index(g)
480                     work.append(g[:split_i])
481                     work.append(g[split_i:])
482                 else:
483                     out.append(g)
484                     continue
485                 work.append(g[:best_i])
486                 work.append(g[best_i:])
487             out.sort(key=lambda grp: grp[0][0])
488         return out
489
490     @staticmethod
491     def _blob_fallback_chop(groups: List[List[Tuple[int, float]]], max_win_frames: int,
492     blob_factor: float = 4.0

```

```

481         ) -> Tuple[List[List[Tuple[int, float]]], List[bool]]:
482     """R5 blob splitter ? the gap-less, R4-survived residual. After the W1 gap-split
AND the R4
483     inactivity trim have each had their chance, a group still longer than
484     ``blob_factor`` x
485     ``max_win_frames`` is a genuine continuously-active blob (no internal gap, no
rally-end
486     signal the principled rules could find ? the real-footage ``mahadevpura-1`` ~65
s residual ?
487     11x a 6 s cap). The factor is what keeps this SURGICAL: a normal long rally
488     (~1?2x the cap)
489     is left ALONE; only the degenerate blob qualifies. As a LAST RESORT the blob is
490     midpoint-chopped to ? the cap and every resulting piece is FLAGGED as a forced
cut, so the
491     degenerate single-window outcome is avoided and the clip is surfaced (logged) as
needing
492     rally-STATE cues ? net-crossings / two-sided motion ? not a bigger cap. Only
cuts (recall
493     can't drop). Returns ``(groups, forced_flags)`` aligned 1:1; ``forced_flags[i]``
is True iff
494     that piece came from a forced chop. Iterative worklist to bound stack depth."""
495     if max_win_frames <= 0:
496         return groups, [False] * len(groups)
497     blob_frames = max_win_frames * max(1.0, blob_factor)
498     out: List[List[Tuple[int, float]]] = []
499     flags: List[bool] = []
500     forced = 0
501     for g in groups:
502         if len(g) < 2 or (g[-1][0] - g[0][0]) <= blob_frames:
503             out.append(g)
504             flags.append(False)
505             continue
506         pieces: List[List[Tuple[int, float]]] = []
507         work = [g]
508         while work:
509             h = work.pop()
510             if len(h) < 2 or (h[-1][0] - h[0][0]) <= max_win_frames:
511                 pieces.append(h)
512                 continue
513                 split_i = TrackNetRunner._midpoint_split_index(h)
514                 work.append(h[:split_i])
515                 work.append(h[split_i:])
516             pieces.sort(key=lambda grp: grp[0][0])
517             out.extend(pieces)
518             flags.extend([True] * len(pieces))
519             forced += len(pieces) - 1
520         if forced:
521             logger.warning(
522                 "R5 blob-fallback: forced %d midpoint cut(s) on continuously-active
group(s) with "
523                 "no rally-end signal ? this clip needs rally-STATE cues (net-crossings /
two-sided "
524                 "motion), not a bigger cap", forced)
525     return out, flags

```

8. assistant (2026-06-20T08:57:24.114Z)

Now let me read the segmenter files and wasb_infer.py.

9. user (2026-06-20T08:57:25.077Z)

```

1     """Shared engine for trajectory-substrate hybrid segmenters.
2
3     tracknet_hybrid and wasb_hybrid were copy-paste twins ? CODE_MAP carried a

```

```

4      standing footgun: "any P3/P4 fix must be applied to BOTH". This base class is
5      now the single home for the whole pipeline shape:
6
7      healthcheck gate -> P2 trajectory -> candidate windows (+ persistence)
8      -> P3 AI-provider handoff over padded clips -> P4 DB population/linking
9
10     A concrete subclass supplies only its identity and runner wiring:
11     - ``family`` ? config section under ``indexing.*``, output subdir, and
12       the ``<family>-hybrid-<model>`` detector tag in metadata.
13     - ``display_label`` ? human label for log lines ("WASB", "TrackNet").
14     - ``name`` / ``description`` properties (the registry contract).
15     - ``_build_runner(idx_cfg)`` ? returns (DetectorRunner, detector-specific
16       detection_params extras such as weights_path).
17
18     This base is deliberately NOT registered; only concrete subclasses register.
19     """
20     import os
21     import time
22     import logging
23     import concurrent.futures
24     from typing import Any, Dict, List, Optional, Tuple
25
26     import cv2
27
28     from backend.pipeline.segmenters.base import BasePlaySegmenter
29     from backend.utils.video import get_video_duration, extract_video_chunk
30     from backend.sports import sport_registry
31     from backend.providers import provider_registry
32     from backend.pipeline.detectors.base import DetectorRunner
33
34     logger = logging.getLogger(__name__)
35
36
37     def _fmt_ts(seconds: float) -> str:
38         seconds = max(0, int(seconds))
39         h, rem = divmod(seconds, 3600)
40         m, s = divmod(rem, 60)
41         return f"{h:02d}:{m:02d}:{s:02d}"
42
43
44     class TrajectoryHybridSegmenter(BasePlaySegmenter):
45         """Trajectory-detector hybrid: precise candidate rallies + AI semantic
46         boundaries."""
47
48         #: config section under `indexing` (velocity_threshold lives there), the
49         #: output subdir for trajectories, and the metadata detector-tag prefix.
50         family: str = ""
51         #: human-readable label used in log lines.
52         display_label: str = ""
53
54         def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
55         Any]]:
56             """Return (runner, detector-specific detection_params extras) for the default
57             (WSL) path."""
58             raise NotImplementedError
59
60         def _build_native_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner,
61         Dict[str, Any]]:
62             """Return (runner, extras) for the Linux-native (no-WSL) path. Only families
63             with a
64             native runner override this; the base refuses with a clear message (M3.5: WASB
65             only)."""
66             raise NotImplementedError(
67                 f"detector_impl='native' is not supported for the "
68                 f"{self.display_label or self.family or 'trajectory'!r} family ? only WASB

```

```

has a "
63         f"Linux-native runner. Use detector_impl='auto' (WSL) or 'stub'.")
64
65     def _resolve_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner,
Dict[str, Any]]:
66         """Resolve the detector runner, honouring the CPU-stub and Linux-native
overrides.
67
68         ``RALLY_DETECTOR_IMPL`` env var (takes precedence) or
``indexing.detector_impl``:
69         ``"stub"`` swaps in a deterministic CPU runner (no GPU/WSL) so the whole
pipeline
70         is regression-testable without a GPU ? "mock a GPU with a CPU"; ``"native"``
swaps
71         in the Linux-native WASB runner (no WSL/conda-activate; GPU box) via
72         ``_build_native_runner``. Anything else (``"auto"``) delegates to
``_build_runner``
73         (the WSL runner) so the default production path is unchanged
74         (``docs/DECOUPLED_COMPUTE/``).
75         """
76         impl = (os.environ.get("RALLY_DETECTOR_IMPL") or idx_cfg.get("detector_impl") or
"auto").lower()
77         if impl == "stub":
78             from backend.pipeline.detectors.stub_runner import StubDetectorRunner,
StubConfig
79             logger.info("%s: detector_impl=stub ? CPU mock runner (no GPU/WSL).",
self.display_label or "trajectory")
80             return StubDetectorRunner(StubConfig.from_indexing_cfg(idx_cfg)),
{"detector_impl": "stub"}
81         if impl in ("native", "native_wasb", "wasb_native"):
82             logger.info("%s: detector_impl=native ? Linux-native runner (no WSL).",
self.display_label or "trajectory")
83             return self._build_native_runner(idx_cfg)
84         return self._build_runner(idx_cfg)
85
86     @staticmethod
87     def _probe_fps_width(video_path: str) -> tuple:
88         """Return (fps, frame_width) for a video, with sensible fallbacks."""
89         cap = cv2.VideoCapture(video_path)
90         fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
91         width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
92         cap.release()
93         return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)
94
95     @staticmethod
96     def _shot_count_from(segment: Dict[str, Any], duration: float) -> int:
97         """Provider-reported exchange count, else a duration-based estimate.
98
99         One canonical implementation ? the twins had drifted (wasb relied on
100         ``int(None)`` raising into the fallback; same result, by accident).
101         """
102         shot_count = segment.get("shuttle_exchange_count")
103         if shot_count is None:
104             return max(3, int(duration / 0.8))
105         try:
106             return int(shot_count)
107         except (ValueError, TypeError):
108             return max(3, int(duration / 0.8))
109
110     # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged)
111     -----
112     def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
frame_width: float, video_path: str,
113     idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
114         """Stats dict persisted into ``candidate_segments.stats`` for one window. The

```

```

BASE
117         returns exactly the 4 motion keys, so the trajectory twins persist byte-
identical
118         rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
119         features). ``points`` are the whole-video trajectory points
(TrajectoryPoint)."""
120         return {
121             "peak_velocity": cw["peak_velocity"],
122             "mean_velocity": cw["mean_velocity"],
123             "active_frame_count": cw["active_frame_count"],
124             "track_id_count": cw["track_id_count"],
125         }
126
127     def _select_for_handoff(self, windows: List[Dict[str, Any]],
128                           window_stats: List[Dict[str, Any]],
129                           idx_cfg: Dict[str, Any]) -> List[int]:
130         """Indices of the candidate windows that proceed to the AI handoff (and thus may
131         become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
132         ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
133         Persisted candidates are NOT affected ? they remain the full superset for
offline
134         re-scoring."""
135         return list(range(len(windows)))
136
137     def process_video(self, video_id: str, video_path: str, # type: ignore[override]
138                     player_pool: Optional[List[str]] = None,
139                     max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
140         # override-ignore: the ABC declares the Result contract (Success|Failure)
141         # but every live segmenter still returns a bare list ? the documented
142         # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
143         label = self.display_label
144         # --- Sport profile + AI provider wiring (identical across segmenters) ---
145         sport_name = self.config.get("sport", "badminton")
146         try:
147             sport_profile = sport_registry.get(sport_name)
148         except KeyError as e:
149             logger.error(str(e))
150         return []
151
152         idx_cfg = self.config.get("indexing", {})
153         # Fully-local, NO-AI mode (default OFF): the trajectory candidate windows become
the
154         # rallies directly ? Phase 3's AI handoff is skipped, so no provider / API key
is
155         # needed and nothing is uploaded. Used to run the local detector standalone
(e.g. to
156         # measure it against the Gemini path, or to avoid the cloud entirely). With
157         # FusionSegmenter the windows are still Gate-A/B-filtered via
`_select_for_handoff`.
158         skip_ai = bool(idx_cfg.get("skip_ai_handoff", False))
159         provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
160         if skip_ai:
161             model_name = "local-noai"
162             logger.info("%s in FULLY-LOCAL mode (skip_ai_handoff=True): candidate
windows will "
163                       "be emitted as rallies; no %s handoff, no API key needed.",
164                       label, provider_name)
165         else:
166             model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
167             api_key_env_var = f"{provider_name.upper()}_API_KEY"
168             api_key = os.environ.get(api_key_env_var)
169             if not api_key:
170                 from backend.pipeline.segmenters._keyhint import api_key_hint

```

```

171         logger.error(
172             f"{api_key_env_var} environment variable is not set! "
173             f"To run the {provider_name} handoff, set your API key. "
174             + api_key_hint(api_key_env_var)
175         )
176         return []
177     try:
178         provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
179     except KeyError as e:
180         logger.error(str(e))
181         return []
182
183     video_duration = get_video_duration(video_path)
184     if video_duration <= 0:
185         logger.error("Invalid video duration.")
186         return []
187     fps, frame_width = self._probe_fps_width(video_path)
188
189     # --- Runner config + windowing thresholds ---
190     # _resolve_runner honours the CPU-stub override (RALLY_DETECTOR_IMPL /
191     # indexing.detector_impl="stub") so the pipeline runs GPU/WSL-free; otherwise
192     # it delegates to the subclass _build_runner (the real WSL/native runner).
193     runner, runner_params = self._resolve_runner(idx_cfg)
194
195     velocity_thresh = idx_cfg.get(self.family, {}).get("velocity_threshold", 0.02)
196     merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
197     min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)
198     # Bounded-windowing over-merge fix (W1): 0 = OFF (unchanged). See
IndexingConfig.
199     max_window_duration = idx_cfg.get("max_window_duration", 0.0)
200     window_min_split_gap = idx_cfg.get("window_min_split_gap", 0.5)
201     window_hard_cap = idx_cfg.get("window_hard_cap", False)
202     window_inactivity_end = idx_cfg.get("window_inactivity_end", 0.0)
203     inactivity_rally_thresh = idx_cfg.get("inactivity_rally_thresh", 0.08)
204     inactivity_min_density = idx_cfg.get("inactivity_min_density", 0.34)
205     window_blob_fallback = idx_cfg.get("window_blob_fallback", False)
206     window_blob_factor = idx_cfg.get("window_blob_factor", 4.0)
207     handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)
208     ai_max_workers = idx_cfg.get("ai_max_workers", 5)
209     log_candidates = idx_cfg.get("log_yolo_candidates", True)
210     store_candidates = idx_cfg.get("store_yolo_candidates", True)
211
212     detection_params = {
213         "detector": self.name,
214         "velocity_thresh": velocity_thresh,
215         "merge_gap": merge_gap,
216         "min_action_window_duration": min_window_duration,
217         **runner_params,
218     }
219
220     # --- Phase 1: env healthcheck (fail fast with a clear message) ---
221     ok, msg = runner.healthcheck()
222     if not ok:
223         logger.error("%s detector environment not ready: %s", label, msg)
224         return []
225     logger.info("%s detector env OK: %s", label, msg)
226
227     # --- Phase 2: trajectory -> candidate rally windows ---
228     # Trajectory detectors process the whole video in one pass (they carry
229     # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
230     t_start = time.time()
231     out_dir = os.path.join("output", self.family)
232     csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
233     if not csv_path:

```

```

234         logger.error("%s produced no trajectory; aborting.", label)
235         return []
236
237     points = runner.parse_trajectory_csv(csv_path)
238     logger.info("Parsed %d trajectory points (%d visible).",
239               len(points), sum(1 for p in points if p.visible))
240
241     all_candidate_windows = runner.trajectory_to_action_windows(
242         points, fps=fps, frame_width=frame_width, chunk_start=0.0,
243         velocity_thresh=velocity_thresh, merge_gap=merge_gap,
244         min_window_duration=min_window_duration, chunk_index=0,
245         max_window_duration=max_window_duration, min_split_gap=window_min_split_gap,
246         window_hard_cap=window_hard_cap,
247         window_inactivity_end=window_inactivity_end,
248         inactivity_rally_thresh=inactivity_rally_thresh,
249         inactivity_min_density=inactivity_min_density,
250         window_blob_fallback=window_blob_fallback,
251         window_blob_factor=window_blob_factor,
252     )
253     logger.info("Phase 2 (%s) completed in %.1fs. Candidate windows: %d",
254               label, time.time() - t_start, len(all_candidate_windows))
255
256     if log_candidates:
257         for cw in all_candidate_windows:
258             logger.info(f"[{label} rally]
259             {_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
260             f"({cw['end'] - cw['start']:.1f}s) |
261             frames={cw['active_frame_count']} "
262             f"peak_v={cw['peak_velocity']:.3f}
263             mean_v={cw['mean_velocity']:.3f}")
264
265     # Per-window stats via the enrichment hook ? base = the 4 motion keys; the
266     fusion
267     # segmenter adds net_crossings + person-cue features. Computed once, reused for
268     both
269     # persistence and the handoff gate below.
270     window_stats = [
271         self._enrich_candidate_stats(cw, points, fps, frame_width, video_path,
272         idx_cfg)
273         for cw in all_candidate_windows
274     ]
275
276     # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
277     candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
278     if store_candidates:
279         for i, cw in enumerate(all_candidate_windows):
280             candidate_ids[i] = self.db.add_candidate_segment(
281                 video_id, detector=self.name,
282                 start_time=cw["start"], end_time=cw["end"],
283                 chunk_index=cw["chunk_index"], confidence=min(1.0,
284                 cw["peak_velocity"]),
285                 stats=window_stats[i], detection_params=detection_params,
286             )
287
288     if not all_candidate_windows:
289         return []
290
291     # --- Phase 3: AI provider handoff over padded candidate clips ---
292     # Which windows reach the handoff (base = all; fusion may apply Gate A/B).
293     Persisted
294     # candidates above stay the full superset ? only the served play_segments are
295     gated.
296     handoff_indices = self._select_for_handoff(all_candidate_windows, window_stats,
297     idx_cfg)
298

```

```

289         ai_segments: List[dict] = []
290     if skip_ai:
291         # Fully-local: the selected candidate windows ARE the rallies ? emit them
verbatim
292         # (no clip extraction, no upload). Boundaries are the local detector's; shot
count
293         # falls back to the duration-based estimate (no AI exchange count).
294         ai_segments = [
295             {
296                 "start_time": all_candidate_windows[wi]["start"],
297                 "end_time": all_candidate_windows[wi]["end"],
298                 "shuttle_exchange_count": None,
299                 "shot_count_confidence": "LocalNoAI",
300                 "_candidate_id": candidate_ids[wi],
301             }
302             for wi in handoff_indices
303         ]
304         logger.info("Phase 3 SKIPPED (fully-local): emitted %d candidate windows as
rallies "
305                     "(no AI handoff).", len(ai_segments))
306     else:
307         handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
308         os.makedirs(handoff_dir, exist_ok=True)
309         logger.info("Phase 3: %s validation on %d candidate windows...",
310                     provider_name, len(handoff_indices))
311
312         def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
313             w_start = max(0, window["start"] - handoff_padding)
314             w_end = min(video_duration, window["end"] + handoff_padding)
315             w_dur = w_end - w_start
316             w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
317             if not extract_video_chunk(video_path, w_start, w_dur, w_path,
reencode=True):
318                 return []
319             prompt = sport_profile.get_prompt(chunk_duration=w_dur)
320             segments, _ = provider.analyze_video(w_path, prompt,
chunk_duration=w_dur,
321                                                  label=f"Handoff {wi+1}")
322             valid = []
323             for seg in segments:
324                 try:
325                     seg["start_time"] = float(seg["start_time"]) + w_start
326                     seg["end_time"] = float(seg["end_time"]) + w_start
327                     seg["_candidate_id"] = candidate_ids[wi]
328                     valid.append(seg)
329                 except (ValueError, TypeError, KeyError) as e:
330                     logger.warning("Dropping segment with unparseable timestamps: %s
? %s", seg, e)
331                 try:
332                     os.remove(w_path)
333                 except OSError:
334                     pass
335             return valid
336
337         with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
executor:
338             futures = [executor.submit(process_candidate_window, wi,
all_candidate_windows[wi])
339                         for wi in handoff_indices]
340             for future in futures:
341                 ai_segments.extend(future.result())
342         try:
343             os.rmdir(handoff_dir)
344         except OSError:
345             pass

```

```

346         logger.info("Phase 3 completed. AI confirmed %d semantic play segments.",
len(ai_segments))
347
348         # --- Phase 4: DB population + candidate linking ---
349         segments_data = []
350         for segment in ai_segments:
351             try:
352                 start = float(segment["start_time"])
353                 end = float(segment["end_time"])
354                 dur = end - start
355                 metadata = {
356                     "shot_count": self._shot_count_from(segment, dur),
357                     "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
358                     "detector": f"{self.family}-hybrid-{model_name}",
359                 }
360                 segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
361                 if segment_id:
362                     candidate_id = segment.get("_candidate_id")
363                     if candidate_id is not None:
364                         self.db.link_candidate_to_segment(candidate_id, segment_id)
365                     segments_data.append({
366                         "id": segment_id, "start_time": start, "end_time": end,
367                         "duration": dur, "metadata": metadata,
368                     })
369             except Exception as e:
370                 logger.warning("Skipping invalid segment struct: %s", e)
371
372         return segments_data
373

```

10. user (2026-06-20T08:57:26.082Z)

```

1         """WASB shuttle-trajectory inference on an arbitrary frame sequence (runs in WSL).
2
3         WASB-SBDT only ships *dataset* evaluation; this is a thin wrapper that reuses its
4         own building blocks (ImageDataset transform + detector + online tracker) to run on
5         any folder of extracted frames and emit a trajectory CSV in the exact shape the
6         indexer's `tracknet_runner.parse_trajectory_csv` already consumes:
7
8         Frame,Visibility,X,Y
9
10        It must run INSIDE the WSL `wasb` conda env, from the WASB-SBDT `src/` dir (it
11        imports WASB's `detectors`/`trackers`/`dataloaders`). The Windows-side adapter
12        (`wasb_runner.py`) stages this file + the frames into WSL and invokes it.
13
14        Resumable across reboots
15        -----
16        The GPU detector pass (one forward per sliding window ? ~21k for a 6-min clip) is
17        the slow, expensive part; the CPU tracker pass that turns detections into a
18        trajectory is cheap and *stateful* (it must see every frame in order, so it can't
19        resume mid-stream). We exploit that split:
20
21        * The detector pass writes its per-window raw outputs incrementally to
22          ``<cache>/det_raw.jsonl`` (flushed+fsync'd every ``--flush-every`` windows) and
23          records progress in ``<cache>/manifest.json`` (atomic write). A reboot loses at
24          most ``--flush-every`` windows of GPU work instead of the whole pass.
25        * On restart we replay the cached windows, skip the DataLoader ahead to the first
26          un-cached window, and continue. Once every window is cached, the detector pass
27          is skipped entirely and we just re-run the (cheap, deterministic) tracker over
28          the full ordered cache -> trajectory CSV. So a lost trajectory CSV costs seconds,
29          not the GPU hour.
30
31        The cache is keyed by the output path (``<out>_wasbcache/`` by default), lives next

```

```

32     to the output (NOT in %TEMP%), and is invalidated automatically if the frames dir,
33     weights, sport, window size, or frame count change.
34
35     Usage (in WSL, from ~/models/WASB-SBDT/src):
36     python wasb_infer.py --frames_dir /path/frames --weights
37     ../pretrained_weights/wasb_badminton_best.pth.tar \
38     --out /path/traj.csv [--sport badminton] [--limit N] [--cache-dir DIR] [--flush-
every 1000] [--fresh]
39     """
40     import os
41     import os.path as osp
42     import csv
43     import glob
44     import json
45     import argparse
46     import logging
47     from collections import defaultdict
48     from contextlib import contextmanager
49
50     logger = logging.getLogger(__name__)
51
52     CACHE_VERSION = 1
53     _DET_FILE = "det_raw.jsonl"
54     _MANIFEST_FILE = "manifest.json"
55
56     def _build_cfg(weights: str, sport: str, device: str = "cuda"):
57         """Compose the WASB eval config via Hydra, overriding model/weights/device.
58
59         ``device`` is ``cuda`` (default ? the historical WSL behaviour, byte-parity
preserved),
60         ``mps`` (Apple) or ``cpu``. Only the single-GPU CUDA path requests
``runner.gpus=[0]``;
61         cpu/mps run with no GPU list so the detector places tensors on ``device``.
62         """
63         from hydra import compose, initialize
64         gpus = "[0]" if device == "cuda" else "[]" # single GPU on cuda; none for cpu/mps
65         with initialize(config_path="configs", version_base=None):
66             cfg = compose(config_name="eval", overrides=[
67                 f"dataset={sport}",
68                 "model=wasb",
69                 f"detector.model_path={weights}",
70                 f"runner.device={device}",
71                 f"runner.gpus={gpus}", # default config assumes multiple GPUs; pin to this
box
72             ])
73         return cfg
74
75
76     def _frame_id(path: str) -> int:
77         """Best-effort integer frame id from a frame filename (e.g. frame_000180.png ->
180)."""
78         stem = osp.splitext(osp.basename(path))[0]
79         digits = "".join(ch for ch in stem if ch.isdigit())
80         return int(digits) if digits else -1
81
82
83     # ----- #
84     # Resumable detector cache (pure-Python, no torch/numpy ? unit-testable)
85     # ----- #
86     def _cache_paths(cache_dir: str):
87         return osp.join(cache_dir, _DET_FILE), osp.join(cache_dir, _MANIFEST_FILE)
88
89
90     def default_cache_dir(out_csv: str) -> str:

```

```

91     """Stable cache dir next to the trajectory output (survives reboots; not %TEMP%)."""
92     d = osp.dirname(osp.abspath(out_csv))
93     stem = osp.splitext(osp.basename(out_csv))[0]
94     return osp.join(d, stem + "_wasbcache")
95
96
97 def _atomic_write_text(path: str, text: str) -> None:
98     """Write then rename so a crash mid-write can't leave a half-written file."""
99     tmp = path + ".tmp"
100    with open(tmp, "w") as f:
101        f.write(text)
102        f.flush()
103        os.fsync(f.fileno())
104    os.replace(tmp, path)
105
106
107 def _det_plain(det) -> list:
108     """A detector candidate dict {'xy': np.array([x,y]), 'score', 'scale'} -> JSON-able
109     list."""
110     xy = det["xy"]
111     return [float(xy[0]), float(xy[1]), float(det["score"]), int(det["scale"])]
112
113
114 def _dets_to_tracker(plain_list):
115     """Inverse of _det_plain: rebuild the dicts the tracker expects (xy MUST be a numpy
116     array,
117     the tracker does vector math / np.linalg.norm on it)."""
118     import numpy as np
119     return [{"xy": np.array([p[0], p[1]], dtype=float), "score": p[2], "scale": p[3]}
120             for p in plain_list]
121
122
123 def write_manifest(cache_dir: str, manifest: dict) -> None:
124     _, mpath = _cache_paths(cache_dir)
125     _atomic_write_text(mpath, json.dumps(manifest, indent=2))
126
127
128 def read_manifest(cache_dir: str):
129     _, mpath = _cache_paths(cache_dir)
130     if not osp.exists(mpath):
131         return None
132     try:
133         with open(mpath) as f:
134             return json.load(f)
135     except (json.JSONDecodeError, OSError):
136         return None
137
138
139 def manifest_compatible(manifest: dict, *, frames_dir: str, weights: str, sport: str,
140                        frames_in: int, frames_total: int, device: str = "cuda") ->
141 bool:
142     """A cache is reusable only if the run that produced it matches this run's inputs.
143     ``device`` defaults to ``cuda`` so caches written before device-keying (no
144     ``device``
145     field) stay compatible with a CUDA run ? but a cpu/mps run will NOT reuse a cuda
146     cache
147     (detections can differ numerically across devices), and vice-versa.
148     """
149     if not manifest or manifest.get("version") != CACHE_VERSION:
150         return False
151     return (
152         manifest.get("frames_dir") == osp.abspath(frames_dir)
153         and manifest.get("weights") == weights
154         and manifest.get("sport") == sport

```

```

151         and manifest.get("frames_in") == frames_in
152         and manifest.get("frames_total") == frames_total
153         and manifest.get("device", "cuda") == device
154     )
155
156
157 def reset_cache(cache_dir: str) -> None:
158     """Drop any existing cache so the next run starts clean."""
159     det_jsonl, mpath = _cache_paths(cache_dir)
160     for p in (det_jsonl, mpath):
161         if osp.exists(p):
162             os.remove(p)
163
164
165 def load_and_clean_cache(cache_dir: str):
166     """Read the longest *contiguous* (w == line index) valid prefix of det_raw.jsonl,
167     rebuild the per-frame detection accumulation, and rewrite the file to exactly that
168     prefix so a trailing half-written line from a crash can't corrupt future appends.
169
170     Returns (windows_done, det_by_fid) where det_by_fid maps frame_id -> list of
171     [x, y, score, scale] candidates (accumulated across every window the frame is in,
172     in window order ? identical to a non-resumed run).
173     """
174     det_jsonl, _ = _cache_paths(cache_dir)
175     det_by_fid = defaultdict(list)
176     valid: list = []
177     if osp.exists(det_jsonl):
178         with open(det_jsonl, "r") as f:
179             for line in f:
180                 s = line.strip()
181                 if not s:
182                     continue
183                 try:
184                     obj = json.loads(s)
185                 except json.JSONDecodeError:
186                     break # truncated trailing line (crash mid-
flush)
187                 if obj.get("w") != len(valid): # non-contiguous -> stop at the gap
188                     break
189                 valid.append(s)
190                 for fid, plain in obj["f"]:
191                     det_by_fid[int(fid)].extend([list(p) for p in plain])
192             # Guarantee a clean append boundary by rewriting only the good prefix.
193             _atomic_write_text(det_jsonl, ("\n".join(valid) + "\n") if valid else "")
194     return len(valid), det_by_fid
195
196
197 def _append_windows(fh, objs) -> None:
198     """Append window records as JSONL and durably flush (bounded loss on crash)."""
199     for o in objs:
200         fh.write(json.dumps(o, separators="," + ":") + "\n")
201     fh.flush()
202     os.fsync(fh.fileno())
203
204
205 def _atomic_write_trajectory(out_csv: str, rows) -> None:
206     os.makedirs(osp.dirname(osp.abspath(out_csv)), exist_ok=True)
207     tmp = out_csv + ".tmp"
208     with open(tmp, "w", newline="") as f:
209         w = csv.writer(f)
210         w.writerow(["Frame", "Visibility", "X", "Y"])
211         w.writerows(rows)
212         f.flush()
213         os.fsync(f.fileno())
214     os.replace(tmp, out_csv)

```

```

215
216
217 # ----- #
218 # Frame addressing + windowing (pure; no torch/cv2 ? unit-testable)
219 # ----- #
220 _STREAM_FRAME_FMT = "{:08d}.png"
221
222
223 def synthetic_frame_path(index: int) -> str:
224     """Stable, index-encoding frame name used by the streaming path.
225
226     It mirrors the on-disk names ``extract_frames`` writes (``{i:08d}.png``) so the
227     downstream integer-id parser (``_frame_id``) yields the SAME frame id whether the
228     frames came from disk or from the in-memory stream. The streaming image loader
229     inverts this name back to an index to fetch the decoded frame.
230     """
231     return _STREAM_FRAME_FMT.format(int(index))
232
233
234 def build_windows(frames: list, frames_in: int):
235     """Sliding windows of ``frames_in`` consecutive frame *paths* (the unit of GPU work
236     + checkpoint). Pure list math, identical for the disk and streaming paths.
237
238     Returns a list of windows, each a list of ``frames_in`` consecutive paths
239     (``[frames[i:i+frames_in] for i in range(len(frames) - frames_in + 1)]``). The
240     caller wraps each window into the ``{"frames", "annos"}`` sample shape WASB's
241     ``ImageDataset`` expects; keeping that out of here stays torch-free for unit tests.
242     """
243     return [frames[i:i + frames_in] for i in range(len(frames) - frames_in + 1)]
244
245
246 class SequentialFrameStore:
247     """In-memory sliding buffer over a forward-only frame reader, sized for the
248     detector's sliding-window access pattern (peak O(window) frames, not O(video)).
249
250     The detector DataLoader runs with ``shuffle=False`` and (in streaming mode)
251     ``num_workers=0``. ``ImageDataset.__getitem__(i)`` reads the frames of window ``i``
252     in order ? ``i, i+1, ..., i+window-1`` ? and samples are requested ``i = start,
253     start+1, ...``. So the global read sequence dips back by ``window-1`` at each window
254     boundary (``0,1,2, 1,2,3, 2,3,4, ...`` for ``window=3``); the highest index seen so
255     far never decreases. We keep only frames at or above ``max_seen - (window-1)`` (the
256     earliest index any not-yet-finished window can still ask for) and decode each source
257     frame exactly once via the forward-only ``reader(index)``.
258     """
259
260     def __init__(self, reader, total: int, window: int = 1, start_index: int = 0):
261         self._reader = reader
262         self._total = int(total)
263         self._window = max(1, int(window))
264         self._buf: dict = {}
265         # On a resumed run the detector's first needed frame is `start_index`; begin
266         # pulling there so the reader can skip (seek past) the already-cached prefix
267         # instead of decoding 0..start_index-1 just to throw them away.
268         self._next = int(start_index) # next index not yet pulled from the reader
269         self._max_seen = int(start_index) - 1
270         self._floor = int(start_index) # lowest index we still keep (never evict
below)
271
272     def get(self, index: int):
273         index = int(index)
274         if index < self._floor:
275             raise KeyError(
276                 f"frame {index} already evicted (below floor {self._floor}); the access
"
277                 f"pattern must stay within a window of the highest frame seen")

```

```

278         if index >= self._total:
279             raise KeyError(f"frame {index} out of range (total {self._total})")
280         # Pull forward from the reader until the requested index is buffered.
281         while self._next <= index:
282             self._buf[self._next] = self._reader(self._next)
283             self._next += 1
284         if index > self._max_seen:
285             self._max_seen = index
286         # Evict frames older than the earliest a still-running window could re-request:
287         # once we've seen index `m`, no future window starts before `m - (window-1)`.
288         new_floor = max(self._floor, self._max_seen - (self._window - 1))
289         for k in range(self._floor, new_floor):
290             self._buf.pop(k, None)
291         self._floor = new_floor
292         return self._buf[index]
293
294
295     def _index_from_synthetic_path(path: str) -> int:
296         """Invert ``synthetic_frame_path``: a frame path -> its integer source index.
297
298         Reuses ``_frame_id`` (digits-only parse) so the index and the cached frame-id stay
299         in lockstep with the on-disk naming scheme.
300         """
301         return _frame_id(path)
302
303
304     def _bgr_to_pil_rgb(frame_bgr):
305         """Convert an in-memory BGR uint8 frame (as ``cv2.VideoCapture`` yields) to the
306         EXACT
307         PIL RGB image WASB's disk path produces.
308
309         The disk path is ``frame_bgr -> cv2.imwrite(PNG) ->
310         Image.open(PNG).convert('RGB')``;
311         because PNG is lossless that round-trip equals ``cv2.cvtColor(frame_bgr,
312         COLOR_BGR2RGB)``
313         bit-for-bit (verified empirically, max|diff|=0). Returning that here makes the
314         streamed
315         frame indistinguishable from the on-disk one to everything downstream.
316         """
317         import cv2
318         from PIL import Image
319         return Image.fromarray(cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2RGB))
320
321
322     def _make_streamed_read_image(store, real_read_image):
323         """Build the ``read_image`` replacement used during a streaming detector pass.
324
325         A synthetic frame path -> the in-memory decoded frame for its index (as a PIL RGB
326         image,
327         matching ``read_image``'s real return type); any path that doesn't parse to an index
328         falls
329         through to ``real_read_image``. Pure ? imports no WASB module ? so it is unit-
330         testable
331         without the WSL-only ``dataloaders`` package.
332         """
333         def _read_image(path, *args, **kwargs):
334             idx = _index_from_synthetic_path(path)
335             if idx < 0:
336                 return real_read_image(path, *args, **kwargs)
337             return _bgr_to_pil_rgb(store.get(idx))
338         return _read_image
339
340
341     @contextmanager
342     def _streaming_read_image_patch(frame_loader, total: int, window: int = 1, start_index:

```

```

int = 0):
336     """Scope a shim over WASB's ``read_image`` so ``ImageDataset`` is fed in-memory
decoded
337     frames during the detector pass, byte-for-byte as it would over real PNGs.
338
339     WASB's ``ImageDataset.__getitem__`` reads each frame via ``read_image(path)`` ? NOT
340     ``cv2.imread``. ``read_image`` (``utils.utils``) does
``Image.open(path).convert('RGB')``,
341     returning a PIL **RGB** image, after an ``osp.exists(path)`` guard. We therefore
feed it a
342     PIL RGB image built from the streamed BGR frame (see ``_bgr_to_pil_rgb``), which is
343     byte-identical to the disk round-trip, and the shim never touches the filesystem so
the
344     ``osp.exists`` guard is sidestepped for synthetic stream paths.
345
346     We patch ``dataloaders.dataset_loader.read_image`` ? the binding the consumer
actually
347     calls (``dataset_loader`` does ``from utils import read_image``, so the name lives
in that
348     module's namespace; patching ``utils.read_image`` would NOT rebind the already-
imported
349     reference).
350
351     ``frame_loader(index) -> BGR uint8 ndarray`` is a forward-only sequential decoder
wrapped
352     in a ``SequentialFrameStore`` (sliding buffer sized to ``window`` = ``frames_in``);
on a
353     resume we start at ``start_index`` so the decoder seeks past already-cached windows.
The
354     original reader is restored on exit.
355     """
356     from dataloaders import dataset_loader as _dl
357     store = SequentialFrameStore(frame_loader, total, window=window,
start_index=start_index)
358     real_read_image = _dl.read_image
359     # Rebind read_image in the consuming module for the duration of the detector pass;
360     # restore on exit. (Plain assignment, not setattr-with-constant, to stay ruff
B010-clean.)
361     _dl.read_image = _make_streamed_read_image(store, real_read_image) # type:
ignore[assignment]
362     try:
363         yield store
364     finally:
365         _dl.read_image = real_read_image # type: ignore[assignment]
366
367
368     # ----- #
369     # Inference
370     # ----- #
371     def run(frames, weights: str, out_csv: str, sport: str = "badminton",
372            limit: int = 0, batch_size: int = 8, num_workers: int = 4,
373            log_every_batches: int = 50, cache_dir: "str | None" = None,
374            flush_every: int = 1000, fresh: bool = False,
375            frame_loader=None, source_key: "str | None" = None,
376            device: str = "cuda") -> str:
377         """Run WASB detection+tracking over an ordered frame source -> trajectory CSV.
378
379         Two equivalent ways to supply pixels (output is bit-identical between them):
380
381         * **frames-on-disk** (default): ``frames`` is a directory path string; frames are
382           globbed (``*.png``/``*.jpg``) and ``ImageDataset`` reads each file via
``cv2.imread``.
383         * **streaming** (fast path): ``frames`` is a pre-built ordered list of (synthetic)
384           frame paths and ``frame_loader(index) -> BGR uint8 ndarray`` supplies the decoded
385           pixels in-memory. We scope a shim over WASB's ``read_image`` (the actual per-frame

```

```

386         reader, which returns a PIL RGB image) over the DataLoader pass so every other
line of
387         ``ImageDataset.__getitem__`` runs UNCHANGED ? the tensor the detector sees is
identical
388         to the disk round-trip (``cv2.imwrite`` PNG -> ``Image.open().convert('RGB')`` ==
389         ``cvtColor(bgr, BGR2RGB)``), verified byte-identical). Streaming forces
``num_workers=0``
390         (the in-process shim + buffer can't cross worker processes).
391
392         ``source_key`` overrides the cache-keying identity (defaults to the abspath of the
393         frames dir); the streaming path passes a stable ``video:<abspath>`` key so a resume
394         after reboot still matches.
395         """
396         import time
397         import torch
398         from torch.utils.data import DataLoader
399         from detectors import build_detector
400         from trackers import build_tracker
401         from dataloaders import build_img_transforms, build_seq_transforms
402         from dataloaders.dataset_loader import ImageDataset
403         from utils import Center
404
405         streaming = frame_loader is not None
406
407         cfg = _build_cfg(weights, sport, device)
408         frames_in = int(cfg["model"]["frames_in"])
409         input_wh = (int(cfg["model"]["inp_width"]), int(cfg["model"]["inp_height"]))
410         output_wh = (int(cfg["model"]["out_width"]), int(cfg["model"]["out_height"]))
411
412         if streaming:
413             # `frames` is already the ordered list of (synthetic) frame paths.
414             if not isinstance(frames, (list, tuple)):
415                 raise SystemExit("streaming mode requires `frames` to be a list of frame
paths")
416             frames = list(frames)
417             frames_dir = source_key or "stream"
418         else:
419             frames_dir = frames
420             frames = sorted(glob.glob(osp.join(frames_dir, "*.png"))) +
glob.glob(osp.join(frames_dir, "*.jpg")))
421             if limit:
422                 frames = frames[:limit]
423             if len(frames) < frames_in:
424                 where = frames_dir if not streaming else "video stream"
425                 raise SystemExit(f"need >= {frames_in} frames, found {len(frames)} in {where}")
426
427             # Sliding windows of `frames_in` consecutive frames (the unit of GPU work +
checkpoint).
428             samples = []
429             for window in build_windows(frames, frames_in):
430                 annos = [{"frame_path": p, "center": Center(is_visible=False, x=-1.0, y=-1.0)}
for p in window]
431                 samples.append({"frames": window, "annos": annos})
432             windows_total = len(samples)
433
434             # ---- cache setup / resume decision ----- #
435             if cache_dir is None:
436                 cache_dir = default_cache_dir(out_csv)
437             os.makedirs(cache_dir, exist_ok=True)
438             det_jsonl, _ = _cache_paths(cache_dir)
439             cache_key = source_key if source_key is not None else osp.abspath(frames_dir)
440             manifest = None if fresh else read_manifest(cache_dir)
441             resume = manifest_compatible(
442                 manifest or {}, frames_dir=cache_key, weights=weights, sport=sport,
443                 frames_in=frames_in, frames_total=len(frames), device=device,

```

```

444         )
445         if resume:
446             windows_done, det_by_fid = load_and_clean_cache(cache_dir)
447             logger.info(f"resuming from cache: {windows_done}/{windows_total} windows
already detected")
448         else:
449             if manifest is not None:
450                 logger.info("cache incompatible with this run -> starting fresh")
451                 reset_cache(cache_dir)
452                 windows_done, det_by_fid = 0, defaultdict(list)
453
454         base_manifest = {
455             "version": CACHE_VERSION,
456             "frames_dir": cache_key,
457             "weights": weights,
458             "sport": sport,
459             "frames_in": frames_in,
460             "frames_total": len(frames),
461             "device": device,
462             "windows_total": windows_total,
463             "windows_done": windows_done,
464         }
465         write_manifest(cache_dir, base_manifest)
466
467         # ---- detector pass over the un-cached windows ----- #
468         remaining = samples[windows_done:]
469         if remaining:
470             _, transform_test = build_img_transforms(cfg)
471             try:
472                 _, seq_transform_test = build_seq_transforms(cfg)
473             except Exception:
474                 seq_transform_test = None
475             ds = ImageDataset(cfg, remaining, input_wh, output_wh,
476                             transform=transform_test, seq_transform=seq_transform_test,
is_train=False)
477             # Streaming forces single-process loading: the in-memory frame store + the
478             # cv2.imread shim live in THIS process and can't be pickled to DataLoader
workers.
479             loader_workers = 0 if streaming else num_workers
480             loader = DataLoader(ds, batch_size=batch_size, shuffle=False,
num_workers=loader_workers)
481             detector = build_detector(cfg)
482
483             from contextlib import ExitStack
484
485             t0 = time.time()
486             done = windows_done
487             win_idx = windows_done
488             log_every = max(1, log_every_batches)
489             flush_n = max(1, flush_every)
490             pending = []
491             logger.info(f"detecting on {len(remaining)} remaining windows "
492                        f"(total {windows_total}, batch={batch_size},
workers={loader_workers}, "
493                        f"source={'video-stream' if streaming else 'frames-dir'})...")
494             det_f = open(det_jsonl, "a")
495             try:
496                 with ExitStack() as stack:
497                     stack.enter_context(torch.no_grad())
498                     # In streaming mode, route ImageDataset's per-frame `cv2.imread(path)`
to the
499                     # in-memory decoded frame for that synthetic path; every other line of
500                     # __getitem__ runs unchanged so the tensor matches the PNG round-trip
exactly.
501                     # The detector's first needed frame is `windows_done` (window i starts

```

```

at
502         # frame i), so prime the store there for a resume.
503         if streaming:
504             stack.enter_context(
505                 _streaming_read_image_patch(frame_loader, len(frames),
506                                             window=frames_in,
start_index=windows_done))
507         for bi, batch in enumerate(loader):
508             imgs, _hms, trans = batch[0], batch[1], batch[2]
509             img_paths = [list(t) for t in batch[-1]] # [frame_in][batch] ->
path
510             batch_results, _ = detector.run_tensor(imgs, trans)
511             nb = imgs.shape[0]
512             for ib in range(nb):
513                 frames_payload = []
514                 for ie in sorted(batch_results[ib].keys()):
515                     p = img_paths[ie][ib]
516                     fid = _frame_id(p)
517                     plain = [_det_plain(d) for d in batch_results[ib][ie]]
518                     frames_payload.append([fid, plain])
519                     det_by_fid[fid].extend(plain)
520                 pending.append({"w": win_idx, "f": frames_payload})
521                 win_idx += 1
522             done += nb
523             if len(pending) >= flush_n or done >= windows_total:
524                 _append_windows(det_f, pending)
525                 pending = []
526                 base_manifest["windows_done"] = done
527                 write_manifest(cache_dir, base_manifest)
528             if (bi + 1) % log_every == 0 or done >= windows_total:
529                 el = time.time() - t0
530                 new_done = done - windows_done
531                 rate = new_done / el if el > 0 else 0.0
532                 eta = (windows_total - done) / rate if rate > 0 else 0.0
533                 logger.info(f"{done}/{windows_total} "
534                             f"({100 * done // max(1, windows_total)}%) |
{rate:.1f} win/s | "
535                             f"elapsed {el:.0f}s | ETA {eta:.0f}s")
536             if pending:
537                 _append_windows(det_f, pending)
538                 base_manifest["windows_done"] = done
539                 write_manifest(cache_dir, base_manifest)
540         finally:
541             det_f.close()
542             logger.info(f"detection done in {time.time() - t0:.0f}s; running tracker...")
543     else:
544         logger.info("detector cache already complete -> tracker-only re-run")
545
546     # ---- tracker re-run over the full ordered cache (cheap, deterministic) ---- #
547     tracker = build_tracker(cfg)
548     tracker.refresh()
549     rows = []
550     for fid in sorted(det_by_fid.keys()):
551         r = tracker.update(_dets_to_tracker(det_by_fid[fid]))
552         rows.append((fid, 1 if r.get("visi") else 0, r.get("x", -1), r.get("y", -1)))
553
554     _atomic_write_trajectory(out_csv, rows)
555     visible = sum(1 for _, v, *_ in rows if v)
556     logger.info(f"wrote {len(rows)} rows -> {out_csv}")
557     logger.info(f"visible-shuttle frames: {visible}/{len(rows)}")
558     return out_csv
559
560
561 def extract_frames(video_path: str, frames_dir: str) -> str:
562     """Decode a video to PNG frames named by source frame index (00000000.png ...).

```

```

563
564 Idempotent: if the dir already holds the expected number of frames (matching the
565 container's reported frame count) we skip re-decoding ? re-extraction after a
566 reboot is wasteful. NOTE: cv2 PNG extraction is slow; for big clips prefer
567 extracting with ffmpeg on the host and passing --frames_dir.
568 """
569 import cv2
570 os.makedirs(frames_dir, exist_ok=True)
571 cap = cv2.VideoCapture(video_path)
572 if not cap.isOpened():
573     raise SystemExit(f"cannot open video: {video_path}")
574 expected = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
575 existing = len(glob.glob(osp.join(frames_dir, "*.png")))
576 if expected > 0 and existing >= expected:
577     cap.release()
578     logger.info(f"reusing {existing} already-extracted frames -> {frames_dir}")
579     return frames_dir
580 i = 0
581 while True:
582     ok, frame = cap.read()
583     if not ok:
584         break
585     cv2.imwrite(osp.join(frames_dir, f"{i:08d}.png"), frame)
586     i += 1
587 cap.release()
588 logger.info(f"extracted {i} frames -> {frames_dir}")
589 return frames_dir
590
591
592 def _probe_frame_count(video_path: str) -> int:
593     """Container-reported frame count via cv2 (0 if unknown). Used to size the
594     stream."""
595     import cv2
596     cap = cv2.VideoCapture(video_path)
597     if not cap.isOpened():
598         raise SystemExit(f"cannot open video: {video_path}")
599     n = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
600     cap.release()
601     return n
602
603 def make_video_frame_loader(video_path: str):
604     """Return ``(frame_loader, count_hint, release)`` for output-preserving streaming.
605
606     ``frame_loader(index)`` -> BGR uint8 ndarray`` decodes the video forward-only with one
607     long-lived ``cv2.VideoCapture``. It MUST be called with non-decreasing indices (the
608     detector's sliding-window order); it reads sequentially and only uses a frame-seek
609     to
610     skip forward when a *resume* starts past the current position. Each ``cap.read()``
611     returns exactly the BGR uint8 array that ``cv2.imwrite``/``cv2.imread`` of a PNG
612     would
613     yield (PNG is lossless), so the detector sees identical pixels to the disk path.
614
615     Returns the count hint from the container so the caller can build the frame list;
616     ``release`` closes the capture.
617     """
618     import cv2
619     cap = cv2.VideoCapture(video_path)
620     if not cap.isOpened():
621         raise SystemExit(f"cannot open video: {video_path}")
622     count_hint = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
623     state = {"pos": 0} # next index cap.read() will return
624
625     def frame_loader(index: int):
626         index = int(index)

```

```

625         if index < state["pos"]:
626             raise RuntimeError(
627                 f"streaming decode is forward-only; got index {index} after
{state['pos']}")
628         if index > state["pos"]:
629             # Forward skip only happens when resuming past cached windows. Seek once.
630             cap.set(cv2.CAP_PROP_POS_FRAMES, index)
631             state["pos"] = index
632             ok, frame = cap.read()
633             if not ok:
634                 raise RuntimeError(f"failed to decode frame {index} from {video_path}")
635             state["pos"] += 1
636             return frame
637
638     def release():
639         cap.release()
640
641     return frame_loader, count_hint, release
642
643
644     def run_video_streaming(video_path: str, weights: str, out_csv: str, sport: str =
"badminton",
645                             limit: int = 0, batch_size: int = 8,
646                             log_every_batches: int = 50, cache_dir: str | None = None,
647                             flush_every: int = 1000, fresh: bool = False,
648                             device: str = "cuda") -> str:
649         """Fast path: decode ``video_path`` on the fly and feed frames straight to the
650         detector, WITHOUT extracting every frame to a PNG first (the disk round-trip that
651         dominates a long clip). Output is bit-identical to the frames-on-disk path; see
652         ``run``'s docstring for why.
653         """
654         frame_loader, count_hint, release = make_video_frame_loader(video_path)
655         try:
656             if count_hint <= 0:
657                 raise SystemExit(
658                     f"could not determine frame count for {video_path}; cannot stream "
659                     f"(fall back to frame extraction with --frames_dir)")
660             frame_list = [synthetic_frame_path(i) for i in range(count_hint)]
661             source_key = "video:" + osp.abspath(video_path)
662             return run(frame_list, weights, out_csv, sport, limit,
663                       batch_size=batch_size, num_workers=0,
664                       log_every_batches=log_every_batches, cache_dir=cache_dir,
665                       flush_every=flush_every, fresh=fresh,
666                       frame_loader=frame_loader, source_key=source_key, device=device)
667         finally:
668             release()
669
670
671     def main():
672         ap = argparse.ArgumentParser()
673         ap.add_argument("--frames_dir", help="folder of pre-extracted frames")
674         ap.add_argument("--video", help="video file; frames are extracted internally (in
WSL)")
675         ap.add_argument("--frames_out_dir", help="where to extract frames when --video is
used")
676         ap.add_argument("--stream-video", action="store_true",
677                         help="FAST PATH: decode --video on the fly and feed frames straight
to "
678                             "the detector (no PNG extraction). Output-equivalent to the
disk path.")
679         ap.add_argument("--weights", required=True)
680         ap.add_argument("--out", required=True)
681         ap.add_argument("--sport", default="badminton")
682         ap.add_argument("--device", default="cuda", choices=["cuda", "mps", "cpu"],
683                         help="torch device for the detector (default cuda = the WSL parity

```

```

path)")
684         ap.add_argument("--limit", type=int, default=0, help="cap #frames (for quick
tests)")
685         ap.add_argument("--batch-size", type=int, default=8)
686         ap.add_argument("--num-workers", type=int, default=4)
687         ap.add_argument("--log-every-batches", type=int, default=50, help="progress log
cadence")
688         ap.add_argument("--cache-dir", default=None,
689                         help="resumable detector cache dir (default: <out>_wasbcache next to
--out)")
690         ap.add_argument("--flush-every", type=int, default=1000,
691                         help="fsync the detector cache every N windows (bounds reboot
loss)")
692         ap.add_argument("--fresh", action="store_true", help="ignore any existing cache and
start over")
693         args = ap.parse_args()
694
695         if args.stream_video:
696             if not args.video:
697                 raise SystemExit("--stream-video requires --video")
698             run_video_streaming(args.video, args.weights, args.out, args.sport, args.limit,
699                               batch_size=args.batch_size,
700                               log_every_batches=args.log_every_batches,
cache_dir=args.cache_dir,
701                               flush_every=args.flush_every, fresh=args.fresh,
device=args.device)
702         return
703
704         frames_dir = args.frames_dir
705         if args.video:
706             frames_dir = extract_frames(args.video, args.frames_out_dir or (args.video +
"_frames"))
707         if not frames_dir:
708             raise SystemExit("provide --frames_dir or --video")
709         run(frames_dir, args.weights, args.out, args.sport, args.limit,
710           batch_size=args.batch_size, num_workers=args.num_workers,
711           log_every_batches=args.log_every_batches, cache_dir=args.cache_dir,
712           flush_every=args.flush_every, fresh=args.fresh, device=args.device)
713
714
715         if __name__ == "__main__":
716             main()
717

```

11. assistant (2026-06-20T08:57:28.545Z)

Now let me read the wasb_hybrid, fusion_hybrid, config models, and worker.

12. user (2026-06-20T08:57:30.083Z)

```

1      """WASB-based hybrid play segmenter.
2
3      Phase 2 uses **WASB** (nttcom/WASB-SBDT, MIT) shuttle-trajectory detection via
4      WSL2 (backend/pipeline/detectors/wasb_runner.py). WASB ships pretrained
5      badminton weights and runs today, so this is the live shuttle-motion substrate.
6
7      For the per-video *calibration* goal we mainly use Phase 2 (trajectory ->
8      candidate windows); the windowing thresholds (velocity_thresh / merge_gap /
9      min_window_duration) are what get tuned against golden rally labels.
10
11     The pipeline engine (healthcheck -> trajectory -> windows -> AI handoff -> DB)
12     lives in trajectory_hybrid.TrajectoryHybridSegmenter; this class only wires the
13     WASB runner and identity.
14     """
15     from typing import Any, Dict, Tuple

```

```

16
17 from backend.pipeline.segmenters.base import segmenter_registry
18 from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
19 from backend.pipeline.detectors.base import DetectorRunner
20 from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
21
22
23 class WasbHybridSegmenter(TrajectoryHybridSegmenter):
24     family = "wasb"
25     display_label = "WASB"
26
27     @property
28     def name(self) -> str:
29         return "wasb_hybrid"
30
31     @property
32     def description(self) -> str:
33         return ("Two-stage pipeline: WASB shuttle-trajectory detection (WSL2, pretrained
badminton "
34                 "weights) for precise candidate rallies + AI provider for semantic
boundaries.")
35
36     def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
37         cfg = WasbConfig.from_indexing_cfg(idx_cfg)
38         return WasbRunner(cfg), {"weights_path": cfg.weights_path}
39
40     def _build_native_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner,
Dict[str, Any]]:
41         # Lazy import: the native runner pulls no heavy deps, but keep parity with the
stub
42         # seam (imported only when detector_impl='native' is actually selected).
43         from backend.pipeline.detectors.native_wasb_runner import NativeWasbRunner,
NativeWasbConfig
44         cfg = NativeWasbConfig.from_indexing_cfg(idx_cfg)
45         return NativeWasbRunner(cfg), {"weights_path": cfg.weights_path,
46                                         "detector_impl": "native", "device": cfg.device}
47
48
49     segmenter_registry.register(WasbHybridSegmenter)
50

```

13. user (2026-06-20T08:57:30.539Z)

```

1     """Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
2
3     Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
4     TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
5     do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
6     shuttle-seeded window via two overridable seams the engine exposes:
7
8     - ``_enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,
9     GPU-free, computed from the trajectory we already have); and, only when the person cue
10    is explicitly enabled (``indexing.fusion.cue_flags.person``), the person-cue features
11    (``fusion_features``), persisting them into ``candidate_segments.stats`` for the learned
12    fusion (P3). The person path lazily builds ONE `PersonDetector` and only then loads
the
13    torchvision/supervision model ? so the default (person-OFF) path never loads a
detector
14    model nor touches the GPU. (torch/cv2 themselves are imported by the registry
regardless,
15    via the pre-existing yolo_hybrid ? person_detector chain; the lazy part is the model.)
16    - ``_select_for_handoff`` ? optional served Gate A/B: when
``indexing.fusion.min_crossings``
17    (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows

```



```

70         "queue_length": cfg.queue_length,
71         "fusion_substrate": "tracknet",
72     }
73     wcfg = WasbConfig.from_indexing_cfg(idx_cfg)
74     return WasbRunner(wcfg), {"weights_path": wcfg.weights_path, "fusion_substrate":
"wasb"}
75
76     def _build_native_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner,
Dict[str, Any]]:
77         # Native (no-WSL) path supports the WASB substrate only ? there is no native
TrackNet
78         # runner yet. Fusion's default substrate is WASB, so this covers the common
case.
79         substrate = idx_cfg.get("fusion", {}).get("substrate", "wasb")
80         if substrate != "wasb":
81             raise NotImplementedError(
82                 "detector_impl='native' supports the WASB substrate only ? set "
83                 "indexing.fusion.substrate='wasb' (no native TrackNet runner yet).")
84         from backend.pipeline.detectors.native_wasb_runner import NativeWasbRunner,
NativeWasbConfig
85         cfg = NativeWasbConfig.from_indexing_cfg(idx_cfg)
86         return NativeWasbRunner(cfg), {"weights_path": cfg.weights_path,
"detector_impl": "native",
87                                         "fusion_substrate": "wasb", "device": cfg.device}
88
89     def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
90                                frame_width: float, video_path: str,
91                                idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
92         stats = super()._enrich_candidate_stats(cw, points, fps, frame_width,
video_path, idx_cfg)
93         # Gate-A signal ? net-crossing count for this window. Pure, GPU-free, always
persisted
94         # (single service feed crosses ~once; a real rally crosses >=2x). axis/net auto-
estimated
95         # over the whole trajectory by rally_gate (camera-agnostic); reused across
windows.
96         gated = rally_gate.gate_windows(points, [(cw["start"], cw["end"])], fps,
97                                                min_crossings=0,
estimate=self._axis_estimate(points))
98         stats["net_crossings"] = float(gated[0].crossings) if gated else 0.0
99
100        fcfg = idx_cfg.get("fusion", {})
101        if fcfg.get("cue_flags", {}).get("person", False):
102            stats.update(self._person_features(cw, points, fps, frame_width, video_path,
fcfg))
103        return stats
104
105        # P3 (learned fusion) first-step landing note (per MULTI_SIGNAL_FUSION_PLAN +
NEXT_STEPS 2026-06-14):
106        # Person features (from _person_features when cue_flags.person) are now persisted
for the harness
107        # `compare` LOVO litmus on the 6 golden videos. Eager-person-compute optimisation
(compute only
108        # for shuttle-seeded windows + padding, not whole video) is already in place via the
lazy
109        # self._person + windowed detections_per_frame. Next: actual harness run + lift
measurement
110        # (owner/hardware) before any served promotion. Flag-gated (default person OFF). See
111        # docs/NEXT_STEPS.md "Rally-detection quality track" and EXPERIMENT_HARNESS.md for
discipline.
112
113        def _axis_estimate(self, points: List[Any]) -> tuple:
114            """Cache rally_gate's (axis, net, span) play-axis estimate per trajectory so
it's
115            computed once per video, not once per candidate window (it depends only on

```

```

points)."""
116         key = id(points)
117         if self._axis_cache_key != key or self._axis_cache is None:
118             self._axis_cache_key = key
119             self._axis_cache = rally_gate.estimate_play_axis(points)
120         est = self._axis_cache
121         assert est is not None
122         return est
123
124     def _person_features(self, cw: Dict[str, Any], points: List[Any], fps: float,
125                         frame_width: float, video_path: str,
126                         fcfg: Dict[str, Any]) -> Dict[str, float]:
127         """GPU path (person cue ON): run the person detector over this window's
ORIGINAL-video
128         frame range and compute the scale/translation-invariant person features. Lazily
builds
129         ONE PersonDetector. ? real-video verification is owner-side; this wiring is
mock-tested."""
130         # Lazy import so the default (person-OFF) path never pulls torch/cv2 through
this module.
131         from backend.pipeline.detectors.person_detector import PersonDetector
132         if self._person is None:
133             self._person = PersonDetector(self.config)
134
135         frame_skip = self.config.get("indexing", {}).get("yolo_frame_skip", 3)
136         # round (not truncate): int() would map e.g. 1.999s*30 to frame 59 not 60 ? a
137         # window-edge off-by-one that drops the boundary frame from the shuttle
alignment.
138         start_f, end_f = round(cw["start"] * fps), round(cw["end"] * fps)
139         det = self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
140         fw = det["frame_width"] or frame_width
141         fh = det["frame_height"]
142         # If we can't trust the frame dims, every normalised feature would silently
collapse
143         # to ~0 ? return an explicit all-zero person-feature row instead of a half-baked
one.
144         if fw <= 0 or fh <= 0:
145             return {k: 0.0 for k in ff.PERSON_FEATURE_NAMES}
146
147         poly = fcfg.get("court_polygon")
148         polygon = [(float(p[0]), float(p[1])) for p in poly] if poly else None
149         net = (fcfg.get("net_axis", "y"), float(fcfg.get("net_position", 0.5)))
150         shuttle_in = [p for p in points if start_f <= p.frame <= end_f]
151         return ff.compute_person_features(det["frames"], shuttle_in, fw, fh,
152                                         court_polygon=polygon, net=net)
153
154     def _select_for_handoff(self, windows: List[Dict[str, Any]],
155                           window_stats: List[Dict[str, Any]],
156                           idx_cfg: Dict[str, Any]) -> List[int]:
157         """Served Gate A/B over the (already-enriched) per-window stats. Default
thresholds
158         (0) keep every window ? identical to the substrate. Persisted candidates are
159         untouched (full superset).
160
161         NEVER-EMPTY safeguard: a video that had ?1 window must keep ?1 after gating. If
the
162         thresholds would drop EVERY window (e.g. a short, noisy or one-sided-feed
trajectory
163         where no segment clears ``min_crossings``), the gate falls back to the full
ungated
164         set rather than emit a video with zero rallies. This makes Gate A/B strictly
165         *pruning* ? it can shrink a candidate set but never zero it out ? so layering it
on
166         the substrate is "do no harm": the bounded-windowing / largest-internal-gap
split

```

```

167         below it always wins on an otherwise-empty video. Observed on real footage
168         (mahadevpura-1): a single mega-window split into pieces, none clearing the
crossings
169         gate ? without this guard the whole clip yielded 0 rallies (no reel)."""
170         fcfg = idx_cfg.get("fusion", {})
171         min_crossings = int(fcfg.get("min_crossings", 0))
172         min_players = int(fcfg.get("min_players", 0))
173         person_on = bool(fcfg.get("cue_flags", {}).get("person", False))
174
175         keep: List[int] = []
176         for i, st in enumerate(window_stats):
177             if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
178                 continue
179             if person_on and min_players > 0 and st.get("players_in_court", 0.0) <
min_players:
180                 continue
181             keep.append(i)
182
183         if not keep and window_stats:
184             # Gating emptied a non-empty video ? fall back to the ungated set.
185             return list(range(len(window_stats)))
186         return keep
187
188
189     segmenter_registry.register(FusionSegmenter)
190

```

14. assistant (2026-06-20T08:57:33.172Z)

Now let me read the worker `\_\_main\_\_.py` and the config models.

15. user (2026-06-20T08:57:34.122Z)

```

1     """Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.
2
3     python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
4         [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
5     python -m backend.worker train ...          # Gen-0 train-from-bucket (follow-up)
6
7     The URI scheme selects the storage backend via ``backend.storage`` (a bare path /
``local://``
8     is an identity passthrough; ``s3://`` / ``gcs://`` is an object store), so existing
local runs
9     are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
10    ``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini
key) ?
11    the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).
12
13    This is a thin orchestrator: it composes the SAME building blocks the ``backend.main``
CLI uses
14    (validator registry ? segmenter ? report), never a forked pipeline.
15    """
16    from __future__ import annotations
17
18    import argparse
19    import csv
20    import json
21    import os
22    import sys
23    import tempfile
24    from typing import Any, List
25
26    from backend import storage
27    from backend.config import load_config
28    from backend.infrastructure.database import Database

```

```

29     from backend.pipeline.segmenters.base import segmenter_registry
30     from backend.pipeline.validators.base import registry as validator_registry
31     from backend.reporting import emit_indexer_report
32     from backend.results import Failure
33     from backend.utils.hashing import compute_video_id
34
35
36     def _coerce(v: str) -> Any:
37         """Coerce a ``--set`` string value to bool/int/float where it round-trips, else
38         str."""
39         low = v.lower()
40         if low in ("true", "false"):
41             return low == "true"
42         for cast in (int, float):
43             try:
44                 return cast(v)
45             except ValueError:
46                 pass
47         return v
48
49     def _apply_overrides(config: Any, sets: List[str]) -> None:
50         """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
51         indexing.skip_ai_handoff=true)."""
52         for kv in sets or []:
53             key, sep, val = kv.partition("=")
54             if not sep:
55                 raise ValueError(f"--set expects k=v, got {kv!r}")
56             parts = key.split(".")
57             obj = config
58             for p in parts[:-1]:
59                 obj = getattr(obj, p)
60             setattr(obj, parts[-1], _coerce(val))
61
62     def _write_intervals_csv(segments: List[dict], path: str) -> None:
63         """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-
64         friendly
65         artifact (same schema as the rally-annotator labels)."""
66         with open(path, "w", newline="") as f:
67             w = csv.writer(f)
68             w.writerow(["start", "end", "shot_count", "ending_reason"])
69             for s in segments:
70                 w.writerow([
71                     f'{float(s.get("start_time", 0.0)):.3f}',
72                     f'{float(s.get("end_time", 0.0)):.3f}',
73                     s.get("shot_count", ""),
74                     s.get("ending_reason", s.get("shot_count_confidence", "")),
75                 ])
76
77     def _write_report_json(video_id: str, segments: List[dict], status: str, path: str) ->
78     None:
79         with open(path, "w") as f:
80             json.dump({
81                 "video_id": video_id,
82                 "status": status,
83                 "segment_count": len(segments),
84                 "segments": [{"start": float(s.get("start_time", 0.0)),
85                             "end": float(s.get("end_time", 0.0))} for s in segments],
86             }, f, indent=2)
87
88     def cmd_infer(args: argparse.Namespace) -> int:
89         config = load_config(args.config) if args.config else load_config()

```

```

90     _apply_overrides(config, args.set)
91     storage_cfg = config.storage.model_dump()
92
93     scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
94     os.makedirs(scratch, exist_ok=True)
95     config.output.output_dir = scratch
96
97     # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
to scratch.
98     local_video = storage.fetch(args.video_uri, os.path.join(scratch, "in.mp4"),
config=storage_cfg)
99     print(f"[worker] pulled {args.video_uri} -> {local_video}")
100
101     # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
byte-identical).
102     video_id = compute_video_id(local_video)
103
104     # 3. VALIDATE (same registry as the main CLI).
105     val_results, val_context = validator_registry.run_all_with_context(local_video,
config)
106     failures = [r for r in val_results if not r.passed]
107     db = Database(os.path.join(scratch, config.output.db_name))
108     db.add_video(video_id, local_video, "failed" if failures else "processed",
[r.model_dump() for r in val_results])
109
110
111     segments: List[dict] = []
112     segmenter_actual = None
113     segmentation_ran = False
114     status = "failed"
115     try:
116         if failures:
117             print("[worker] validation FAILED: "
+ "; ".join(f"{r.validator_name}: {r.message}" for r in failures))
118             return 2
119
120         seg_name = args.segmenter or config.indexing.default_segmenter
121         segmenter_cls = segmenter_registry.get(seg_name)
122         if not segmenter_cls:
123             print(f"[worker] unknown segmenter: {seg_name!r} "
f"(have {list(segmenter_registry.list_available())})")
124             return 2
125
126         segmenter_actual = seg_name
127         result = segmenter_cls(db, config).process_video(video_id, local_video,
max_frames=args.max_frames)
128         segmentation_ran = True
129         if isinstance(result, Failure):
130             print(f"[worker] segmenter FAILED: {result.message}")
131             return 3
132
133         segments = result.value if hasattr(result, "value") else result
134         status = "success"
135
136     finally:
137         # Best-effort per-video observability report (never raises) ? parity with the
main CLI.
138         emit_indexer_report(
139             db=db, video_id=video_id, video_path=local_video, config=config,
140             val_results=val_results, val_context=val_context,
141             segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
142             was_reingest=False, segmentation_ran=segmentation_ran,
143
144
145         # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
146         out_local = os.path.join(scratch, "outputs", video_id)
147         os.makedirs(out_local, exist_ok=True)
148         _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
149         _write_report_json(video_id, segments, status, os.path.join(out_local,
"report.json"))

```

```

148
149     out_prefix = args.out_uri.rstrip("/")
150     pushed = []
151     for fn in sorted(os.listdir(out_local)):
152         uri = f"{out_prefix}/outputs/{video_id}/{fn}"
153         storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
154         pushed.append(uri)
155
156     print(f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):")
157     for u in pushed:
158         print("    ", u)
159     return 0
160
161
162 def _resolve_train_detector(args: argparse.Namespace, config: Any):
163     """Build the trajectory DetectorRunner for `--trajectory-source detector`, honouring
164     detector_impl (RALLY_DETECTOR_IMPL / indexing.detector_impl: auto=WSL, native=GPU
box,
165     stub=CPU mock). Returns the runner, or prints an error + returns None.
166
167     Reuses the segmenter's `_resolve_runner` seam so the worker and the live CLI build
the
168     detector identically (one source of truth). A non-trajectory segmenter (e.g. yolo)
has
169     no shuttle detector and is rejected.
170     """
171     from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
172
173     seg_cls = segmenter_registry.get(args.segmenter)
174     if not seg_cls:
175         print(f"[worker] unknown segmenter: {args.segmenter!r} "
176               f"(have {list(segmenter_registry.list_available())})")
177         return None
178     seg = seg_cls(None, config)
179     if not isinstance(seg, TrajectoryHybridSegmenter):
180         print(f"[worker] --segmenter {args.segmenter!r} has no shuttle detector; "
181               f"use a trajectory hybrid (wasb_hybrid / tracknet_hybrid /
fusion_hybrid).")
182         return None
183     runner, _ = seg._resolve_runner(config.get("indexing", {}))
184     ok, msg = runner.healthcheck()
185     if not ok:
186         print(f"[worker] detector environment not ready: {msg}")
187         return None
188     print(f"[worker] detector ready ({args.segmenter}): {msg}")
189     return runner
190
191
192 def cmd_train(args: argparse.Namespace) -> int:
193     """Gen-0 train-from-bucket: pull labels (+ videos) -> trajectories -> manifest ->
shell out
194     to training.gen0.harness (backend must NOT import training) -> push the artifact
bundle.
195
196     Two trajectory sources (the train pipeline + harness are identical either way):
197     * ``synth`` (default) ? the CPU "mock GPU": deterministic, label-consistent
synthetic
198         shuttle paths (no GPU/WSL), so the whole pipeline is validated on a CPU box /
CI.
199     * ``detector`` ? REAL shuttle trajectories: pull each video and run the resolved
200     DetectorRunner (native WASB on a GPU box, or stub for a GPU-free wiring test).
This
201     is the M3.5 "first real training" path; only the trajectory source flips.
202     """

```

```

203     import subprocess
204
205     config = load_config(args.config) if args.config else load_config()
206     _apply_overrides(config, args.set)
207     storage_cfg = config.storage.model_dump()
208
209     scratch = args.scratch or tempfile.mkdtemp(prefix="rally-train-")
210     labels_dir = os.path.join(scratch, "labels")
211     traj_dir = os.path.join(scratch, "trajectories")
212     videos_dir = os.path.join(scratch, "videos")
213     os.makedirs(labels_dir, exist_ok=True)
214     os.makedirs(traj_dir, exist_ok=True)
215
216     corpus = args.corpus_uri.rstrip("/")
217     label_uris = sorted(u for u in storage.list_uris(f"{corpus}/labels/",
config=storage_cfg)
218                          if u.lower().endswith(".csv"))
219     if len(label_uris) < 2:
220         print(f"[worker] need >=2 labeled videos for leave-one-video-out; found
{len(label_uris)} "
221              f"under {corpus}/labels/")
222         return 2
223
224     # Real-detector source: resolve the runner once + index the bucket's videos/ by
stem.
225     runner = None
226     video_by_stem: dict = {}
227     if args.trajectory_source == "detector":
228         os.makedirs(videos_dir, exist_ok=True)
229         runner = _resolve_train_detector(args, config)
230         if runner is None:
231             return 2
232         for vuri in storage.list_uris(f"{corpus}/videos/", config=storage_cfg):
233             vname = storage.parse_uri(vuri).name
234             video_by_stem[os.path.splitext(vname)[0]] = vuri
235
236     print(f"[worker] trajectory source: {args.trajectory_source}")
237     specs: List[dict] = []
238     for uri in label_uris:
239         fname = storage.parse_uri(uri).name # e.g.
GX020094_proxy.rallies.csv
240         name = fname.replace(".rallies.csv", "").replace(".csv", "")
241         local_label = storage.fetch(uri, os.path.join(labels_dir, fname),
config=storage_cfg)
242         if args.trajectory_source == "detector":
243             vuri = video_by_stem.get(name)
244             if not vuri:
245                 print(f"[worker] no video for {name!r} under {corpus}/videos/ ?
skipping.")
246                 continue
247             local_video = storage.fetch(vuri, os.path.join(videos_dir,
storage.parse_uri(vuri).name),
248                                       config=storage_cfg)
249             video_id = compute_video_id(local_video)
250             # Real fps/width drive the trajectory frame->time mapping (synth fixes them
at 30/1280).
251             from backend.pipeline.segmenters.trajectory_hybrid import
TrajectoryHybridSegmenter
252             fps, frame_width = TrajectoryHybridSegmenter._probe_fps_width(local_video)
253             traj = runner.run_predict(local_video, traj_dir, video_id=video_id)
254             if not traj:
255                 print(f"[worker] detector produced no trajectory for {name!r} ?
skipping.")
256                 continue
257             specs.append({"name": name, "trajectory": traj, "labels": local_label,

```

```

258             "fps": fps, "frame_width": frame_width})
259         else:
260             from backend.pipeline.detectors.stub_runner import
synth_trajectory_from_labels
261             traj = os.path.join(traj_dir, f"{name}_traj.csv")
262             synth_trajectory_from_labels(local_label, traj, fps=args.fps,
frame_width=args.frame_width)
263             specs.append({"name": name, "trajectory": traj, "labels": local_label,
264                           "fps": args.fps, "frame_width": args.frame_width})
265
266         if len(specs) < 2:
267             print(f"[worker] need >=2 videos with trajectories for leave-one-video-out; got
{len(specs)}.")
268             return 2
269         manifest_path = os.path.join(scratch, "manifest.json")
270         with open(manifest_path, "w", encoding="utf-8") as f:
271             json.dump(specs, f, indent=2)
272         src_label = "real detector" if args.trajectory_source == "detector" else "CPU-mock
stub"
273         print(f"[worker] built manifest: {len(specs)} videos ({src_label} trajectories)")
274
275         out_dir = os.path.join(scratch, "artifacts")
276         cmd = [sys.executable, "-m", "training.gen0.harness",
277               "--manifest", manifest_path, "--out", out_dir, "--version", args.version]
278         if args.floor:
279             floor_local = (storage.fetch(args.floor, os.path.join(scratch, "floor.json"),
config=storage_cfg)
280                           if "://" in args.floor else args.floor)
281             cmd += ["--floor", floor_local]
282         print("[worker] training:", " ".join(cmd))
283         rc = subprocess.run(cmd).returncode
284         if rc != 0:
285             print(f"[worker] gen0 harness failed (rc={rc})")
286             return rc
287
288         # PUSH the versioned artifact bundle (weights.json + manifest.json) to the bucket.
289         bundle = os.path.join(out_dir, args.version)
290         out_prefix = args.out_uri.rstrip("/")
291         pushed = []
292         for fn in sorted(os.listdir(bundle)):
293             uri = f"{out_prefix}/weights/{args.version}/{fn}"
294             storage.put(os.path.join(bundle, fn), uri, config=storage_cfg)
295             pushed.append(uri)
296         print(f"[worker] train done: pushed {len(pushed)} artifact(s):")
297         for u in pushed:
298             print("    ", u)
299         return 0
300
301
302     def build_parser() -> argparse.ArgumentParser:
303         p = argparse.ArgumentParser(prog="python -m backend.worker",
304                                     description="Storage-aware worker: pull ? run pipeline ?
push.")
305         sub = p.add_subparsers(dest="cmd", required=True)
306
307         pi = sub.add_parser("infer", help="run inference on one video URI and push outputs")
308         pi.add_argument("--video-uri", required=True, help="local path / local:// / s3:// /
gcs://")
309         pi.add_argument("--out-uri", required=True, help="output prefix
(outputs/<video_id>/? is appended)")
310         pi.add_argument("--segmenter", default=None, help="default:
indexing.default_segmenter")
311         pi.add_argument("--config", default=None, help="config.json path (default:
./config.json)")
312         pi.add_argument("--scratch", default=None, help="local scratch dir (default: a temp

```

```

dir)")
313     pi.add_argument("--max-frames", type=int, default=None)
314     pi.add_argument("--set", action="append", default=[], metavar="k=v",
315                     help="config override, e.g. --set indexing.skip_ai_handoff=true")
316     pi.set_defaults(func=cmd_infer)
317
318     pt = sub.add_parser("train", help="Gen-0 train-from-bucket (labels [+videos] ->
trajectories -> harness)")
319     pt.add_argument("--corpus-uri", required=True, help="prefix containing labels/ (+
videos/ for "
320                     "--trajectory-source detector), e.g. s3://bucket or gcs://bucket")
321     pt.add_argument("--out-uri", required=True, help="output prefix (weights/<version>
is appended)")
322     pt.add_argument("--version", required=True, help="semver for the artifact bundle")
323     pt.add_argument("--trajectory-source", choices=["synth", "detector"],
default="synth",
324                     help="synth = CPU-mock label-consistent trajectories (default, no
GPU); "
325                     "detector = REAL shuttle trajectories via the resolved
DetectorRunner "
326                     "(needs videos/ in the corpus; honours detector_impl).")
327     pt.add_argument("--segmenter", default="wasb_hybrid",
328                     help="trajectory hybrid whose detector is used for --trajectory-
source detector")
329     pt.add_argument("--floor", default=None, help="floor baseline JSON (local path or
storage URI)")
330     pt.add_argument("--config", default=None)
331     pt.add_argument("--scratch", default=None)
332     pt.add_argument("--fps", type=float, default=30.0, help="synth trajectory fps
(detector probes the video)")
333     pt.add_argument("--frame-width", type=float, default=1280.0,
334                     help="synth trajectory frame width (detector probes the video)")
335     pt.add_argument("--set", action="append", default=[], metavar="k=v")
336     pt.set_defaults(func=cmd_train)
337     return p
338
339
340 def main(argv=None) -> int:
341     args = build_parser().parse_args(argv)
342     return args.func(args)
343
344
345 if __name__ == "__main__":
346     sys.exit(main())
347

```

16. user (2026-06-20T08:57:34.464Z)

```

1     """Pydantic models for the application configuration.
2
3     This module defines the canonical, validated, typed configuration for the
4     Badminton Highlight Indexer. Built-in defaults live here (single source of truth).
5
6     The models support graceful migration from old config.json files via extra="allow".
7     """
8
9     from __future__ import annotations
10
11     from typing import Any, List, Literal, Optional
12
13     from pydantic import BaseModel, Field, field_validator, model_validator
14
15     Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]
16
17

```

```

18     class ValidationRelevanceConfig(BaseModel):
19         court_green_lower: list[int] = Field(default=[35, 40, 40])
20         court_green_upper: list[int] = Field(default=[85, 255, 255])
21         court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)
22
23
24     class ValidationConfig(BaseModel):
25         min_resolution_width: int = 1280
26         min_resolution_height: int = 720
27         min_fps: float = 29.9
28         min_blur_threshold: float = 20.0
29         max_scene_cuts_per_minute: float = 0.5
30         relevance: ValidationRelevanceConfig =
Field(default_factory=ValidationRelevanceConfig)
31
32
33     class TrackNetConfig(BaseModel):
34         wsl_distro: str = "Ubuntu"
35         repo_dir: str = "~/models/TrackNetV4"
36         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
37         conda_env: str = "TrackNetV4"
38         weights_path: str = ""
39         queue_length: int = 5
40         stage_in_wsl: bool = True
41         wsl_stage_dir: str = "~/clips"
42         timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
43         velocity_threshold: float = 0.02
44         # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
45         # decoded frame cache) are deleted automatically ? they are pure intermediates,
46         # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
47         # true only for debugging. On FAILURE they are always kept so a re-run can resume.
48         keep_frames: bool = False
49         # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB.
50         # 0 = disabled.
51         min_free_gb: float = 0.0
52
53
54     class WasbIndexConfig(BaseModel):
55         """Typed config for the live WASB shuttle substrate (indexing.wasb).
56
57         Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
58         these knobs actually take effect ? previously this whole block was silently dropped
59         because nested config sections defaulted to extra='ignore'.
60         """
61         wsl_distro: str = "Ubuntu"
62         repo_dir: str = "~/models/WASB-SBDT"
63         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
64         conda_env: str = "wasb"
65         weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
66         sport: str = "badminton"
67         wsl_stage_dir: str = "~/clips"
68         timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
69         velocity_threshold: float = 0.02
70         # --- Linux-native runner only (detector_impl='native'); ignored by the WSL runner.
71         ---
72         # Env vars override these (WASB_DEVICE / WASB_PYTHON) for a config-free GPU box.
73         device: str = "cuda" # torch device for the native runner: cuda | mps | cpu
74         python_bin: str = "python" # the `wasb` conda env python (invoked by path, no
`conda activate`)
75         # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
76         # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates,
regenerable
77         # from the source. Set true only for debugging. On FAILURE they are kept so a re-run
resumes.

```

```

77     keep_frames: bool = False
78     # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB. 0 =
disabled.
79     min_free_gb: float = 0.0
80     # FAST PATH (default OFF until owner-verified side-by-side): decode the staged video
81     # on the fly and feed frames straight to the detector, skipping the slow per-frame
PNG
82     # extraction that dominates a long clip (~46k PNGs for a 12-min 1080p60 video, which
83     # overran the 30-min timeout). Output is bit-identical to the PNG path (PNG is
lossless),
84     # so this is purely a speed/disk win; kept opt-in until the side-by-side confirms
it.
85     stream_video: bool = False
86
87
88     class FusionConfig(BaseModel):
89         """Config for the multi-signal `fusion_hybrid` segmenter (MULTI-SIGNAL_FUSION_PLAN
P2).
90
91         Every default reproduces control behaviour: the fusion segmenter persists the
92         shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']``
is
93         true ? the person-cue features, but it does NOT gate the served handoff unless a
94         threshold is raised (``min_crossings``/``min_players`` default 0 = OFF). The court
mask
95         is optional: ``court_polygon=None`` ? Gate B counts every detection (player-count-
only);
96         supplied ? off-court persons (spectators / adjacent courts) are excluded.
97         """
98         # Which trajectory substrate seeds the windows (shuttle stays primary).
99         substrate: Literal["wasb", "tracknet"] = "wasb"
100        # Windowing velocity threshold for the fusion family (the engine reads
101        # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
102        # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
103        velocity_threshold: float = 0.02
104        # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
105        # enabled ? so the default path never imports torch / touches the GPU.
106        cue_flags: dict[str, bool] = Field(default_factory=dict)
107        # Served Gate A: drop windows whose shuttle net-crossings < this from the AI
handoff.
108        # 0 = OFF (no gating; persisted candidates are always the full superset for offline
re-scoring).
109        min_crossings: int = Field(default=0, ge=0)
110        # Served Gate B: when the person cue is on, drop windows with median in-court
players
111        # < this. 0 = OFF.
112        min_players: int = Field(default=0, ge=0)
113        # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
114        court_polygon: Optional[list[list[float]]] = None
115        # Net line for the person two-sided-motion split (person features only ? the shuttle
116        # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
117        net_axis: Literal["x", "y"] = "y"
118        net_position: float = Field(default=0.5, ge=0.0, le=1.0)
119
120
121        class IndexingConfig(BaseModel):
122            default_segmenter: str = "yolo_hybrid"
123            use_gpu: bool = True
124            # Detector backend for the trajectory hybrids (wasb/tracknet). "auto" = the real
125            # WSL runner (default, production); "stub" = a deterministic CPU mock (no GPU/WSL)
126            # so the whole pipeline is regression-testable on a CPU box / CI; "native" = the
127            # Linux-native WASB runner (no WSL/conda-activate; a GPU box ? M3.5). The
128            # RALLY_DETECTOR_IMPL env var overrides this. See
pipeline/detectors/{stub,native_wasb}_runner.py.
129            detector_impl: str = "auto"

```

```

130     motion_threshold: float = 0.08
131     min_segment_duration: float = 3.0
132     dead_time_merge_gap: float = 2.0
133     chunk_duration: float = 90.0
134     chunk_overlap: float = 10.0
135     detector_model: str = "fasterrcnn_resnet50_fpn"
136     detector_score_threshold: float = 0.5
137     yolo_velocity_threshold: float = 0.15
138     yolo_frame_skip: int = 3
139     yolo_tracker: str = "bytetrack.yaml"
140     yolo_prefetch_workers: int = 2
141     min_action_window_duration: float = 2.0
142     # BOUNDED WINDOWING (over-merge fix W1, RALLY_QUALITY_RESEARCH.md §6). 0 = OFF; > 0
= a window
143     # longer than this many seconds is recursively split at its largest internal
inactivity gap (?)
144     # ``window_min_split_gap`` s ? the most likely rally boundary), so one window can't
swallow
145     # several rallies + the dead time between them. Never merges, only cuts (recall
can't drop).
146     # *** R1 MILESTONE DEFAULT-ON (cap=6): the smallest safe local-windowing flip.
Measured n=13
147     # golden, R2 tolF1: baseline?milestone (cap=6 + R4 below) ?+0.222, sign 12/0,
p=0.0005; cap=6
148     # beats cap=12 ?+0.110, sign 12/0; net over-seg guard PASS (mean merge_rate
0.651?0.161). ***
149     max_window_duration: float = 6.0
150     window_min_split_gap: float = 0.5 # min internal gap (s) that qualifies as a split
point
151     # HARD-CAP fallback for W1: when True, an over-long window with NO internal
inactivity gap
152     # (a continuously-active trajectory ? e.g. the real-footage mahadevpura-1 174s blob)
is
153     # chopped at its midpoint until ? max_window_duration, making the cap a true upper
bound.
154     # Only cuts (recall can't drop). OFF by default until measured/promoted.
155     window_hard_cap: bool = False
156     # R4 rally-END inactivity trim (s, 0 = OFF). After a rally the shuttle keeps moving
slowly
157     # (picked up / walked) above velocity_thresh, so the window over-extends its END
(the measured
158     # [|?end| gap vs golden). Trim the post-rally tail back to the last frame whose
trailing
159     # window stays dense with FAST motion (velocity > inactivity_rally_thresh). Only
cuts.
160     # *** R1 MILESTONE DEFAULT-ON (1.5/0.20/0.30): on top of cap=6, R4 adds a small but
significant
161     # end-precision gain ? ?tolF1 +0.040, sign 9/1/3, p=0.022, [|?end| 4.96?3.31s (n=13).
The W1 cap
162     # is the dominant lever; R4 is the marginal increment. See QUALITY_ITERATIONS.md
"R1". ***
163     window_inactivity_end: float = 1.5
164     inactivity_rally_thresh: float = 0.20
165     inactivity_min_density: float = 0.30
166     # R5 blob-fallback (default OFF). LAST-RESORT splitter for the continuously-active
blob: after
167     # the W1 gap-split AND the R4 inactivity trim have each had their chance, a group
STILL longer
168     # than window_blob_factor × max_window_duration (no internal gap, no rally-end
signal ? e.g.
169     # mahadevpura-1's ~65s residual ? 11× a 6s cap) is midpoint-chopped to ? the cap as
a FORCED cut
170     # (logged + flagged on the window) so the degenerate single-window outcome is
avoided and the
171     # clip is surfaced as needing rally-STATE cues, not a bigger cap. Differs from

```

```

window_hard_cap
172     # (which chops BEFORE R4 and over-segments normal clips): blob-fallback runs AFTER
R4 and only
173     # force-cuts the genuine blob (the factor gate). Only cuts (recall can't drop).
174     window_blob_fallback: bool = False
175     # Magnitude gate that keeps R5 surgical: only a group longer than this ×
max_window_duration is a
176     # blob (a normal long rally ~1-2× the cap is left alone; the mahadevpura-1 residual
is ~11×).
177     # Default 4.0 is do-no-harm on the golden corpus (rescues only the continuously-
active
178     # clips, every other clip a no-op within noise) ? see docs/QUALITY_ITERATIONS.md
"R5".
179     window_blob_factor: float = 4.0
180     log_yolo_candidates: bool = True
181     store_yolo_candidates: bool = True
182     ai_handoff_padding: float = 2.0
183     ai_max_workers: int = 5
184     # Width (px) the gemini segmenter re-encodes chunks to before upload. Stream-copied
185     # chunks of a high-bitrate (4K) source blow past the provider upload cap
186     # (backend/providers/gemini.py::MAX_UPLOAD_MB) and are refused ? observed live as
187     # "0 segments / success". Re-encoding also makes chunk boundaries frame-accurate
188     # (stream copy cuts at the nearest keyframe). 0 = legacy stream-copy at source res.
189     # Matches gemini_refine's --clip-scale-width default.
190     ai_chunk_scale_width: int = 1280
191     # FULLY-LOCAL mode (default OFF): when true, the trajectory-hybrid segmenters
192     # (wasb_hybrid / tracknet_hybrid / fusion_hybrid) skip the Phase-3 AI handoff
entirely and
193     # emit their local candidate windows AS the rallies ? no provider, no API key,
nothing
194     # uploaded. Lets the local detector run standalone (e.g. to benchmark it against the
Gemini
195     # path, or to operate with zero cloud dependency). The pure `gemini` segmenter
ignores this.
196     skip_ai_handoff: bool = False
197     # Optional debug knob: trim every video to the first N seconds before processing.
198     debug_duration: Optional[float] = None
199
200     tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
201     wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
202     fusion: FusionConfig = Field(default_factory=FusionConfig)
203
204     @field_validator("default_segmenter")
205     @classmethod
206     def segmenter_must_be_registered(cls, v: str) -> str:
207         # Registry check happens at runtime in main/segmenter selection.
208         # Here we just allow any string for forward compatibility.
209         return v
210
211     @model_validator(mode="after")
212     def _chunk_step_must_be_positive(self) -> "IndexingConfig":
213         # The chunked segmenters advance by (chunk_duration - chunk_overlap); a non-
positive
214         # step makes `while t < duration: t += step` loop forever, growing memory before
any
215         # GPU work. Reject the bad config at load time instead.
216         if self.chunk_overlap >= self.chunk_duration:
217             raise ValueError(
218                 f"indexing.chunk_overlap ({self.chunk_overlap}) must be < chunk_duration
"
219                 f"({self.chunk_duration}); otherwise chunking never advances."
220             )
221         return self
222
223

```

```

224 class OutputConfig(BaseModel):
225     default_highlights_name: str = "highlights.mp4"
226     db_name: str = "sports_indexer.db"
227     # Directory where the DB, highlight reels, and processed clips are written.
228     # The CLI's --output-dir overrides this at startup (see backend/main.py::main).
229     output_dir: str = "output"
230
231
232 class WatermarkConfig(BaseModel):
233     """Branding overlay burned into each highlight clip during the per-clip re-encode
234     (backend/pipeline/stitcher/compiler.py). **On by default** ? set `enabled: false`
235     (under `stitching.watermark` in config.json) to turn it off. The text is fully
236     configurable. The concat stage stays stream-copy (-c copy)."""
237     enabled: bool = True
238     text: str = "Generated by Khelsutra.guru"
239     logo_path: str = "" # optional transparent PNG overlay; "" = no logo
240     font_path: str = "" # optional .ttf; "" = use the fontconfig `font` family
below
241     font: str = "Sans" # fontconfig family used when font_path is empty
242     font_size_ratio: float = Field(default=0.035, gt=0.0, le=0.5) # text height as a
fraction of frame height
243     opacity: float = Field(default=0.85, ge=0.0, le=1.0)
244     box: bool = True # faint backing box behind the text for legibility
245     margin: int = Field(default=24, ge=0) # px inset from the chosen corner
246     position: Literal["bottom-right", "bottom-left", "top-right", "top-left"] = "bottom-
right"
247
248
249 class StitchingConfig(BaseModel):
250     begin_padding: float = 0.5
251     end_padding: float = 0.5
252     use_cache: bool = False
253     min_shot_count: int = 1
254     # Long-rally reel filter (seconds; 0 = OFF). Drop detected rally pairs whose
duration
255     # (end ? start) is below this from the highlight reel, BEFORE compiling. A cleaner
256     # reel-selection knob than `min_shot_count`: shot_count is unlabeled / "Unknown" on
the
257     # fully-local no-AI path, whereas duration is derived from the rally's own
start/end. The
258     # local detector over-fires and its false positives are mostly SHORT (motion blips,
259     # background-court fragments), so this keeps the reel to the eye-catching long
rallies.
260     # OFF by default ? the detector output is unchanged, only which rallies reach the
reel.
261     min_rally_duration: float = Field(default=0.0, ge=0.0)
262     watermark: WatermarkConfig = Field(default_factory=WatermarkConfig)
263
264
265 class LoggingConfig(BaseModel):
266     """Logging knobs for the run entrypoints (see backend/utils/logging_setup.py)."""
267     # Third-party HTTP/RPC client loggers (httpx, httpcore, urllib3, google-genai,
268     # google.auth, grpc) emit one INFO line per request ? dozens per Gemini run, which
269     # buries our own [rally]/[compile] lines in run.log during manual QA. Default raises
270     # them to WARNING; set true to restore full chatter for request-flow debugging.
271     verbose_http: bool = False
272
273
274 class SecurityConfig(BaseModel):
275     # When set, /api/process video_path must resolve under this directory (hosted/tenant
276     # mode). Default None preserves the single-user local 'any local file' workflow.
277     allowed_video_root: Optional[str] = None
278
279     # --- Chunked upload (B2, PR-2) ---
280     # Landing zone for browser uploads (assembled from parts). Per-user subdirectories

```

```

281     # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
282     # contain upload_dir or /api/process will reject the uploaded paths.
283     upload_dir: str = "output/uploads"
284     # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
285     # free-tier edge proxies cap request bodies there (HOSTING_PLAN $2).
286     max_upload_mb: int = 4096
287     max_part_mb: int = 95
288     allowed_upload_extensions: List[str] = ["mp4", "mov"]
289
290
291     class StorageConfig(BaseModel):
292         """Pluggable storage backend (docs/DECOUPLED_COMPUTE/). Default ``local`` = today's
293         behaviour (filesystem paths, byte-for-byte). ``s3`` covers AWS + Cloudflare R2 + DO
Spaces
294         + MinIO (via ``endpoint_url``); ``gcs`` = Google Cloud Storage; ``gdrive`` = Google
Drive
295         (follow-up). Per-backend settings are loose dicts passed to the backend constructor
?
296         **secrets are never stored here**, only endpoints / regions / file paths / source-
selectors
297         (boto3/google auth chains supply credentials). See ``backend/storage/``. """
298         backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
299         # Output root for produced artifacts. "" => fall back to output.output_dir (non-
breaking).
300         output_root: str = ""
301         local: dict[str, Any] = Field(default_factory=dict)
302         s3: dict[str, Any] = Field(default_factory=dict)
303         gcs: dict[str, Any] = Field(default_factory=dict)
304         gdrive: dict[str, Any] = Field(default_factory=dict)
305
306
307     class AppConfig(BaseModel):
308         """Root configuration object.
309
310         All runtime configuration should be accessed via an instance of this class
311         (or the loader) rather than raw dict lookups.
312         """
313
314         sport: Sport = "badminton"
315         ai_provider: str = "gemini"
316         # Auto-tracking alias: 2.5-flash is legacy now that 3.5-flash is the stable
flagship;
317         # the alias never deprecates underneath us. Eval reports record the run date
alongside.
318         ai_model: str = "gemini-flash-latest"
319
320         validation: ValidationConfig = Field(default_factory=ValidationConfig)
321         indexing: IndexingConfig = Field(default_factory=IndexingConfig)
322         output: OutputConfig = Field(default_factory=OutputConfig)
323         stitching: StitchingConfig = Field(default_factory=StitchingConfig)
324         security: SecurityConfig = Field(default_factory=SecurityConfig)
325         logging: LoggingConfig = Field(default_factory=LoggingConfig)
326         storage: StorageConfig = Field(default_factory=StorageConfig)
327
328         # Allow extra keys during transition so existing config.json files do not break.
329         model_config = {
330             "extra": "allow",
331             "validate_assignment": True,
332         }
333
334         # Temporary migration shim for legacy dict-style access (config.get("section",
{})).get(...)).
335         # Will be removed once all call sites (validators, segmenters, etc.) move to typed
336         # attribute access / the typed config. A nested *section* is returned as a plain
dict

```

```

337         # via model_dump, so legacy chained access keeps working ? returning the Pydantic
338         # section model instead breaks it (a model has no .get), which bit every validator
on
339         # real runs. Scalar leaves are returned as-is.
340         def get(self, key: str, default: Any = None) -> Any:
341             _MISSING = object()
342             val: Any = _MISSING
343             if key in type(self).model_fields or hasattr(self, key):
344                 val = getattr(self, key, _MISSING)
345             if val is _MISSING:
346                 # Legacy convenience: find a leaf key inside any nested section. Sections
are
347                 # discovered dynamically from the model fields, so a future top-level
section
348                 # (e.g. `audio`, `jobs`) is covered automatically without editing a
hardcoded list.
349                 for fname in type(self).model_fields:
350                     section = getattr(self, fname, None)
351                     if isinstance(section, BaseModel) and hasattr(section, key):
352                         val = getattr(section, key)
353                         break
354             if val is _MISSING:
355                 return default
356             if isinstance(val, BaseModel):
357                 return val.model_dump(mode="json")
358             return val
359
360         def __getitem__(self, key: str) -> Any:
361             val = self.get(key)
362             if val is None and key not in type(self).model_fields:
363                 # Re-raise like a real dict for unknown top-level
364                 raise KeyError(key)
365             return val
366
367
368         # Convenience type alias for callers that want to annotate
369         Config = AppConfig
370

```

17. assistant (2026-06-20T08:57:37.930Z)

Now let me read the new tests and check the git diff to confirm exactly what changed.

18. user (2026-06-20T08:57:38.888Z)

```

1         """NativeWasbRunner coverage ? the Linux-native (no-WSL) WASB detector path (M3.5).
2
3         The native subprocess is mocked (no torch/conda/GPU here). These tests lock the contract
4         the GPU-box byte-parity gate then relies on: it IS a DetectorRunner, it is NOT a WSL
runner
5         (no WslCommandMixin / wsl invocation), it builds a native argv (no `wsl`, no `conda
6         activate`, no `/mnt` translation) that runs the SAME wasb_infer.py with the SAME output
CSV
7         name as the WSL runner, threads the device through, and cleans the frame cache on
success.
8         """
9         import os
10        import types
11        from unittest.mock import patch
12
13        import pytest
14
15        from backend.pipeline.detectors.native_wasb_runner import NativeWasbRunner,
NativeWasbConfig
16        from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin

```

```

17
18
19     def _proc(rc=0, stdout="", stderr=""):
20         return types.SimpleNamespace(returncode=rc, stdout=stdout, stderr=stderr)
21
22
23     # --- identity / no-WSL guarantee
24     -----
25     def test_is_a_detector_runner():
26         assert isinstance(NativeWasbRunner(NativeWasbConfig()), DetectorRunner)
27
28     def test_is_not_a_wsl_runner():
29         """The whole point of M3.5: the native runner must carry NO WSL plumbing."""
30         r = NativeWasbRunner(NativeWasbConfig())
31         assert not isinstance(r, WslCommandMixin)
32         assert not hasattr(r, "_wsl_bash")
33
34
35     def test_reuses_model_agnostic_windowing():
36         from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
37         assert NativeWasbRunner.parse_trajectory_csv is TrackNetRunner.parse_trajectory_csv
38         assert NativeWasbRunner.trajectory_to_action_windows is
39         TrackNetRunner.trajectory_to_action_windows
40
41     # --- config resolution (env overrides win over config over default)
42     -----
43     def test_from_indexing_cfg_defaults():
44         cfg = NativeWasbConfig.from_indexing_cfg({})
45         assert cfg.device == "cuda" and cfg.python_bin == "python"
46         assert cfg.weights_path == "" and cfg.repo_dir == "~/models/WASB-SBDT"
47         assert cfg.stream_video is False and cfg.keep_frames is False
48
49     def test_from_indexing_cfg_reads_config_block():
50         cfg = NativeWasbConfig.from_indexing_cfg({"wasb": {
51             "weights_path": "/m/w.pth.tar", "device": "cpu", "python_bin":
52             "/envs/wasb/bin/python",
53             "repo_dir": "/m/WASB-SBDT", "stream_video": True, "keep_frames": True, "sport":
54             "tennis"}}})
55         assert cfg.weights_path == "/m/w.pth.tar" and cfg.device == "cpu"
56         assert cfg.python_bin == "/envs/wasb/bin/python" and cfg.repo_dir == "/m/WASB-SBDT"
57         assert cfg.stream_video is True and cfg.keep_frames is True and cfg.sport ==
58         "tennis"
59
60     def test_env_overrides_beat_config(monkeypatch):
61         monkeypatch.setenv("WASB_WEIGHTS", "/env/w.pth.tar")
62         monkeypatch.setenv("WASB_DEVICE", "mps")
63         monkeypatch.setenv("WASB_PYTHON", "/env/py")
64         monkeypatch.setenv("WASB_REPO_DIR", "/env/repo")
65         monkeypatch.setenv("WASB_SCRATCH", "/env/scratch")
66         cfg = NativeWasbConfig.from_indexing_cfg({"wasb": {
67             "weights_path": "/cfg/w", "device": "cpu", "python_bin": "/cfg/py", "repo_dir":
68             "/cfg/repo"}}})
69         assert cfg.weights_path == "/env/w.pth.tar" and cfg.device == "mps"
70         assert cfg.python_bin == "/env/py" and cfg.repo_dir == "/env/repo"
71         assert cfg.scratch_dir == "/env/scratch"
72
73     # --- healthcheck
74     -----
75     def _ok_cfg(tmp_path, **kw):
76         w = tmp_path / "w.pth.tar"

```

```

74     w.write_text("weights")
75     (tmp_path / "repo" / "src").mkdir(parents=True)
76     return NativeWasbConfig(weights_path=str(w), repo_dir=str(tmp_path / "repo"), **kw)
77
78
79     def test_healthcheck_ok(tmp_path):
80         r = NativeWasbRunner(_ok_cfg(tmp_path, device="cuda"))
81         with patch.object(r, "_run", return_value=_proc(0, stdout="torch 1.11.0 cuda
True")):
82             ok, msg = r.healthcheck()
83             assert ok and "torch" in msg
84
85
86     def test_healthcheck_empty_weights():
87         ok, msg = NativeWasbRunner(NativeWasbConfig(weights_path="")).healthcheck()
88         assert not ok and "weights" in msg.lower()
89
90
91     def test_healthcheck_weights_missing(tmp_path):
92         (tmp_path / "repo" / "src").mkdir(parents=True)
93         cfg = NativeWasbConfig(weights_path=str(tmp_path / "nope.pth"),
repo_dir=str(tmp_path / "repo"))
94         ok, msg = NativeWasbRunner(cfg).healthcheck()
95         assert not ok and "not found" in msg.lower()
96
97
98     def test_healthcheck_repo_missing(tmp_path):
99         w = tmp_path / "w.pth.tar"
100        w.write_text("x")
101        cfg = NativeWasbConfig(weights_path=str(w), repo_dir=str(tmp_path / "absent"))
102        ok, msg = NativeWasbRunner(cfg).healthcheck()
103        assert not ok and "repo" in msg.lower()
104
105
106     def test_healthcheck_torch_import_fails(tmp_path):
107         r = NativeWasbRunner(_ok_cfg(tmp_path))
108         with patch.object(r, "_run", return_value=_proc(1, stderr="ModuleNotFoundError:
torch")):
109             ok, msg = r.healthcheck()
110             assert not ok and "failed" in msg.lower()
111
112
113     def test_healthcheck_cuda_unavailable_when_device_cuda(tmp_path):
114         r = NativeWasbRunner(_ok_cfg(tmp_path, device="cuda"))
115         with patch.object(r, "_run", return_value=_proc(0, stdout="torch 1.11.0 cuda
False")):
116             ok, msg = r.healthcheck()
117             assert not ok and "cuda" in msg.lower()
118
119
120     def test_healthcheck_cpu_device_ok_without_cuda(tmp_path):
121         r = NativeWasbRunner(_ok_cfg(tmp_path, device="cpu"))
122         with patch.object(r, "_run", return_value=_proc(0, stdout="torch 1.11.0 cuda
False")):
123             ok, msg = r.healthcheck()
124             assert ok # cpu run does not require a GPU
125
126
127     def test_healthcheck_python_not_found(tmp_path):
128         r = NativeWasbRunner(_ok_cfg(tmp_path, python_bin="/no/such/python"))
129         with patch.object(r, "_run", side_effect=FileNotFoundError()):
130             ok, msg = r.healthcheck()
131             assert not ok and "python" in msg.lower()
132
133

```

```

134     # --- run_predict
-----
135     def _drive_success(cfg, tmp_path, vid="abc123abc123", **patches):
136         """Drive run_predict to success with the subprocess mocked; returns (runner, out,
rm_spy, argv, cwd)."""
137         r = NativeWasbRunner(cfg)
138         key = f"clip_{vid[:12]}"
139         (tmp_path / f"{key}_wasb.csv").write_text("Frame,Visibility,X,Y\n") # success = CSV
exists
140         with patch.object(r, "_copy_infer_script", return_value=True), \
141             patch.object(r, "_run", return_value=_proc(0)) as run_spy, \
142             patch.object(r, "_rm_rf") as rm_spy:
143             out = r.run_predict(r"C:\v\clip.mp4", str(tmp_path), video_id=vid)
144             argv = run_spy.call_args[0][0]
145             cwd = run_spy.call_args.kwargs.get("cwd")
146             return r, out, rm_spy, argv, cwd, key
147
148
149     def test_run_predict_builds_native_argv_no_wsl(tmp_path):
150         _, out, rm, argv, cwd, key =
_drive_success(NativeWasbConfig(weights_path="/m/w.pth", device="cuda"), tmp_path)
151         assert out == os.path.join(os.path.abspath(str(tmp_path)), f"{key}_wasb.csv")
152         # No WSL / conda plumbing anywhere in the command.
153         assert "wsl" not in argv
154         assert not any("conda" in a or "/mnt/" in a for a in argv)
155         # It runs the SAME inference core, device-threaded, writing the SAME CSV name as
WSL.
156         assert argv[0] == "python" and argv[1] == "wasb_infer.py"
157         assert "--device" in argv and argv[argv.index("--device") + 1] == "cuda"
158         assert "--weights" in argv and argv[argv.index("--weights") + 1] == "/m/w.pth"
159         assert argv[argv.index("--out") + 1] == out
160         # Disk mode extracts frames + cleans them on success.
161         assert "--frames_out_dir" in argv and "--stream-video" not in argv
162         assert cwd.endswith(os.path.join("src"))
163         rm.assert_called_once()
164
165
166     def test_run_predict_stream_mode_no_frames_dir(tmp_path):
167         _, out, rm, argv, _cwd, _key = _drive_success(
168             NativeWasbConfig(weights_path="/m/w.pth", stream_video=True), tmp_path)
169         assert out is not None
170         assert "--stream-video" in argv and "--frames_out_dir" not in argv
171         rm.assert_not_called() # streaming never materializes a frame cache
172
173
174     def test_run_predict_device_threaded_cpu(tmp_path):
175         _, _out, _rm, argv, _cwd, _key = _drive_success(
176             NativeWasbConfig(weights_path="/m/w.pth", device="cpu"), tmp_path)
177         assert argv[argv.index("--device") + 1] == "cpu"
178
179
180     def test_run_predict_keep_frames_skips_cleanup(tmp_path):
181         _, _out, rm, _argv, _cwd, _key = _drive_success(
182             NativeWasbConfig(weights_path="/m/w.pth", keep_frames=True), tmp_path)
183         rm.assert_not_called()
184
185
186     def test_run_predict_none_when_copy_infer_fails(tmp_path):
187         r = NativeWasbRunner(NativeWasbConfig(weights_path="/m/w.pth"))
188         with patch.object(r, "_copy_infer_script", return_value=False):
189             assert r.run_predict(r"C:\v\clip.mp4", str(tmp_path), video_id="abc123abc123")
is None
190
191
192     def test_run_predict_none_when_subprocess_fails(tmp_path):

```

```

193     r = NativeWasbRunner(NativeWasbConfig(weights_path="/m/w.pth"))
194     with patch.object(r, "_copy_infer_script", return_value=True), \
195         patch.object(r, "_run", return_value=_proc(1, stderr="CUDA error")):
196         assert r.run_predict(r"C:\v\clip.mp4", str(tmp_path), video_id="abc123abc123")
is None
197
198
199     def test_run_predict_none_when_csv_absent(tmp_path):
200         # Subprocess "succeeds" (rc 0) but writes no CSV -> treated as failure.
201         r = NativeWasbRunner(NativeWasbConfig(weights_path="/m/w.pth"))
202         with patch.object(r, "_copy_infer_script", return_value=True), \
203             patch.object(r, "_run", return_value=_proc(0)):
204             assert r.run_predict(r"C:\v\clip.mp4", str(tmp_path), video_id="abc123abc123")
is None
205
206
207     def test_run_predict_times_out_returns_none(tmp_path):
208         import subprocess
209         r = NativeWasbRunner(NativeWasbConfig(weights_path="/m/w.pth"))
210         with patch.object(r, "_copy_infer_script", return_value=True), \
211             patch.object(r, "_run", side_effect=subprocess.TimeoutExpired(cmd="x",
timeout=1)):
212             assert r.run_predict(r"C:\v\clip.mp4", str(tmp_path), video_id="abc123abc123")
is None
213
214
215     def test_csv_name_matches_wsl_contract(tmp_path):
216         """The native CSV name must equal the WSL runner's so downstream + the parity diff
line up."""
217         _, out, _rm, _argv, _cwd, key =
_drive_success(NativeWasbConfig(weights_path="/m/w.pth"), tmp_path)
218         assert os.path.basename(out) == f"{key}_wasb.csv"
219
220
221     # --- the detector_impl='native' seam (parallels the stub seam)
-----
222     def test_resolve_runner_native_selects_native_runner_for_wasb(monkeypatch):
223         monkeypatch.delenv("RALLY_DETECTOR_IMPL", raising=False)
224         from backend.pipeline.segmenters.wasb_hybrid import WasbHybridSegmenter
225         runner, params = WasbHybridSegmenter(None, {}). _resolve_runner({"detector_impl":
"native"})
226         assert isinstance(runner, NativeWasbRunner)
227         assert params.get("detector_impl") == "native"
228
229
230     def test_resolve_runner_native_env_overrides_config(monkeypatch):
231         monkeypatch.setenv("RALLY_DETECTOR_IMPL", "native")
232         from backend.pipeline.segmenters.wasb_hybrid import WasbHybridSegmenter
233         runner, _ = WasbHybridSegmenter(None, {}). _resolve_runner({"detector_impl": "auto"})
234         assert isinstance(runner, NativeWasbRunner)
235
236
237     def test_resolve_runner_native_supported_for_fusion_wasb(monkeypatch):
238         monkeypatch.delenv("RALLY_DETECTOR_IMPL", raising=False)
239         from backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter
240         runner, params = FusionSegmenter(None, {}). _resolve_runner({"detector_impl":
"native"})
241         assert isinstance(runner, NativeWasbRunner)
242         assert params.get("fusion_substrate") == "wasb"
243
244
245     def test_resolve_runner_native_refused_for_fusion_tracknet(monkeypatch):
246         monkeypatch.delenv("RALLY_DETECTOR_IMPL", raising=False)
247         from backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter
248         with pytest.raises(NotImplementedError):

```

```

249         FusionSegmenter(None, {})._resolve_runner(
250             {"detector_impl": "native", "fusion": {"substrate": "tracknet"}})
251
252
253     def test_resolve_runner_native_refused_for_tracknet(monkeypatch):
254         monkeypatch.delenv("RALLY_DETECTOR_IMPL", raising=False)
255         from backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
256         with pytest.raises(NotImplementedError):
257             TrackNetHybridSegmenter(None, {})._resolve_runner({"detector_impl": "native"})
258

```

19. user (2026-06-20T08:57:39.878Z)

warning: in the working copy of 'backend/pipeline/segmenters/wasb_hybrid.py', LF will be replaced by CRLF the next time Git touches it

```

backend/config/models.py | 9 +-
backend/pipeline/detectors/wasb_infer.py | 43 ++++++---
backend/pipeline/segmenters/fusion_hybrid.py | 13 +++
backend/pipeline/segmenters/trajectory_hybrid.py | 28 +++++-
backend/pipeline/segmenters/wasb_hybrid.py | 8 ++
backend/worker/__main__.py | 106 ++++++-----
tests/test_worker.py | 51 ++++++
7 files changed, 222 insertions(+), 36 deletions(-)
---BRANCH---
feat/native-wasb-runner

```

20. assistant (2026-06-20T08:57:43.985Z)

The diff stat doesn't list `native\_wasb\_runner.py` or `test\_native\_wasb\_runner.py` ? those are untracked (new files). Let me verify they show as untracked, and check `\_resolve\_runner` for the `idx\_cfg.get` call on a dict vs config object. Let me also examine the worker's `\_resolve\_train\_detector` more closely, since it calls `config.get("indexing", {})`.

21. user (2026-06-20T08:57:45.107Z)

```

M backend/config/models.py
M backend/pipeline/detectors/wasb_infer.py
M backend/pipeline/segmenters/fusion_hybrid.py
M backend/pipeline/segmenters/trajectory_hybrid.py
M backend/pipeline/segmenters/wasb_hybrid.py
M backend/worker/__main__.py
M tests/test_worker.py
?? artifacts/
?? backend/pipeline/detectors/native_wasb_runner.py
?? docs/KHELSUTRA_PRODUCT_OVERVIEW.md
?? tests/test_native_wasb_runner.py
---
diff --git a/backend/pipeline/segmenters/trajectory_hybrid.py
b/backend/pipeline/segmenters/trajectory_hybrid.py
index 9b1f202..f7edb9d 100644
--- a/backend/pipeline/segmenters/trajectory_hybrid.py
+++ b/backend/pipeline/segmenters/trajectory_hybrid.py
@@ -51,17 +51,27 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
     display_label: str = ""

     def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str, Any]]:
-        """Return (runner, detector-specific detection_params extras)."""
+        """Return (runner, detector-specific detection_params extras) for the default (WSL)
+path."""
+        raise NotImplementedError

+    def _build_native_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
+        """Return (runner, extras) for the Linux-native (no-WSL) path. Only families with a

```

```

+         native runner override this; the base refuses with a clear message (M3.5: WASB
only)."""
+         raise NotImplementedError(
+             f"detector_impl='native' is not supported for the "
+             f"{self.display_label or self.family or 'trajectory'!r} family ? only WASB has a "
+             f"Linux-native runner. Use detector_impl='auto' (WSL) or 'stub'."
+         )
+
+     def _resolve_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
-         """Resolve the detector runner, honouring the CPU-stub override.
+         """Resolve the detector runner, honouring the CPU-stub and Linux-native overrides.

            ``RALLY_DETECTOR_IMPL`` env var (takes precedence) or ``indexing.detector_impl``:
            ``stub`` swaps in a deterministic CPU runner (no GPU/WSL) so the whole pipeline
-         is regression-testable without a GPU ? "mock a GPU with a CPU"
-         (``docs/DECOUPLED_COMPUTE/``). Anything else delegates to ``_build_runner`` (the
-         real WSL/native runner) so the default production path is unchanged.
+         is regression-testable without a GPU ? "mock a GPU with a CPU"; ``native`` swaps
+         in the Linux-native WASB runner (no WSL/conda-activate; GPU box) via
+         ``_build_native_runner``. Anything else (``auto``) delegates to ``_build_runner``
+         (the WSL runner) so the default production path is unchanged
+         (``docs/DECOUPLED_COMPUTE/``).
            """
            impl = (os.environ.get("RALLY_DETECTOR_IMPL") or idx_cfg.get("detector_impl") or
"auto").lower()
            if impl == "stub":
@@ -69,6 +79,10 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
            logger.info("%s: detector_impl=stub ? CPU mock runner (no GPU/WSL).",
                self.display_label or "trajectory")
            return StubDetectorRunner(StubConfig.from_indexing_cfg(idx_cfg)), {"detector_impl":
"stub"}
+         if impl in ("native", "native_wasb", "wasb_native"):
+             logger.info("%s: detector_impl=native ? Linux-native runner (no WSL).",
+                 self.display_label or "trajectory")
+             return self._build_native_runner(idx_cfg)
            return self._build_runner(idx_cfg)

        @staticmethod
@@ -206,9 +220,9 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
            # --- Phase 1: env healthcheck (fail fast with a clear message) ---
            ok, msg = runner.healthcheck()
            if not ok:
-             logger.error("%s WSL environment not ready: %s", label, msg)
+             logger.error("%s detector environment not ready: %s", label, msg)
            return []
-             logger.info("%s WSL env OK: %s", label, msg)
+             logger.info("%s detector env OK: %s", label, msg)

            # --- Phase 2: trajectory -> candidate rally windows ---
            # Trajectory detectors process the whole video in one pass (they carry

```

22. assistant (2026-06-20T08:57:51.185Z)

Now I have a full picture. Let me focus on the key concern: cross-platform path handling and the no-WSL guarantee. Let me trace a few critical things.

First, the no-WSL audit of `native\_wasb\_runner.py` ? confirmed it imports `DetectorRunner` and `TrackNetRunner` only (not `WslCommandMixin`, `to\_wsl\_mnt\_path`, or `wsl\_tilde\_quote`). No `wsl`, `conda activate`, or `/mnt` in the argv construction. That part is clean.

Now let me dig into the cross-platform bugs. The critical issue is `run\_predict`'s `os.path.abspath(str(video\_win\_path))` on line 163. Let me trace what happens with a real Linux path vs what the test passes. Let me also check the `\_resolve\_train\_detector` path where `config.get("indexing", {})` returns a dict.

23. user (2026-06-20T08:57:56.287Z)

```
<string>:13: SyntaxWarning: invalid escape sequence '\c'
--- On LINUX (posixpath), a REAL linux video path ---
abspath: /home/ubuntu/videos/clip.mp4
basename of norm: clip.mp4
splitext: ('clip', '.mp4')

--- On LINUX (posixpath), a WINDOWS path passed by a TEST ---
repr: 'C:\x0b\\clip.mp4'
abspath (posix): C:\Users\avidu\Projects\badminton-highlight-indexer/C:
\clip.mp4
basename of norm: clip.mp4
```

24. assistant (2026-06-20T08:57:58.854Z)

The escape mangled it. Let me redo that test cleanly with a raw Windows path string to confirm the real behavior on Linux.

25. user (2026-06-20T08:58:04.523Z)

```
<stdin>:11: SyntaxWarning: invalid escape sequence '\c'
=== LINUX production: real Linux video path ===
os.path.abspath(video): /data/videos/clip.mp4

=== LINUX test machine running the WINDOWS-style path the test passes ===
raw: 'C:\x0b\\clip.mp4'
os.path.abspath(video) on linux: 'C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer/C:\x0b\\clip.mp4'
norm: C:
/clip.mp4
basename(norm): clip.mp4

=== But the argv --video uses os.path.abspath(str(video_win_path)) (NOT norm) ===
argv --video value (linux): 'C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer/C:\x0b\\clip.mp4'
```

26. assistant (2026-06-20T08:58:10.929Z)

The `\\v` is being interpreted as a vertical tab even in the heredoc because Python parses it. The test uses `r"C:\\v\\clip.mp4"` (raw). The key insight holds regardless: `os.path.abspath` of a Windows-style path on Linux mangles it. But this only matters for the `**test**` path, not the production Linux path. On production Linux, `video_win_path` will be a real Linux path which `abspath` handles fine. The test mocks `_run`, so it never inspects the `argv --video` value for correctness of the path? it only checks structural things. So this is benign for production but worth noting.

Let me now look at the more important real-Linux correctness concerns. Let me verify the `cwd` `_repo_src()` and the worker's `config.get("indexing", {})` returning a dict (since `AppConfig.get` returns `model_dump`). Let me confirm what type `_resolve_runner` receives in the worker train path.

27. user (2026-06-20T08:58:16.242Z)

```
type(idx): <class 'dict'>
detector_impl: auto
type(wasb): <class 'dict'>
device: cuda python_bin: python weights_path: ~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar
```

28. assistant (2026-06-20T08:58:23.290Z)

Important finding: the default `WasbIndexConfig.weights_path` is `~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar` (non-empty), but `NativeWasbConfig` default

`weights\_path` is `""`. When the worker/segmenter builds the native config via `from\_indexing\_cfg`, the typed config supplies the WSL default weights path. Let me verify the native config picks that up and whether it's the correct WASB path. This is actually fine ? it inherits the WSL weights default. But note the env-first logic: `env("WASB\_WEIGHTS")` or `w.get("weights\_path", "")` or `""`. Since `w.get("weights\_path")` returns the typed default, that's fine.

Now let me check a subtle but real cross-platform concern: the `scratch\_dir` resolution and `frames\_dir` join, plus whether `\_repo\_src()` produces a correct path on Linux. Let me also verify the worker train path passes the right `idx\_cfg` and check `test\_worker.py`.

29. user (2026-06-20T08:58:24.031Z)

```
1      """Worker dispatcher: pull (storage) ? run pipeline ? push (storage).
2
3      `cmd_infer` is exercised end-to-end over the LOCAL storage backend with the validator +
4      segmenter mocked, so it needs no real video/GPU/network. The real S3 round-trip is
validated
5      on the droplet against MinIO.
6      """
7      import json
8
9      import pytest
10
11     from backend.config import load_config
12     from backend.worker import __main__ as wmain
13
14
15     # ----- pure helpers
16
17     @pytest.mark.parametrize("raw,expected", [
18         ("true", True), ("False", False), ("3", 3), ("2.5", 2.5), ("wasb_hybrid",
"wasb_hybrid"),
19     ])
20     def test_coerce(raw, expected):
21         assert wmain._coerce(raw) == expected and type(wmain._coerce(raw)) is type(expected)
22
23
24     def test_apply_overrides_on_typed_config():
25         cfg = _fresh_config()
26         wmain._apply_overrides(cfg, ["indexing.skip_ai_handoff=true",
"indexing.min_segment_duration=1.5",
27                                     "sport=tennis"])
28         assert cfg.indexing.skip_ai_handoff is True
29         assert cfg.indexing.min_segment_duration == 1.5
30         assert cfg.sport == "tennis"
31
32
33     def test_apply_overrides_rejects_bad_format():
34         cfg = _fresh_config()
35         with pytest.raises(ValueError):
36             wmain._apply_overrides(cfg, ["no_equals_sign"])
37
38
39     def test_write_intervals_csv(tmp_path):
40         segs = [{"start_time": 2.0, "end_time": 5.0, "shot_count": 4,
"shot_count_confidence": "LocalNoAI"},
41                {"start_time": 9.0, "end_time": 12.5}]
42         out = tmp_path / "intervals.csv"
43         wmain._write_intervals_csv(segs, str(out))
44         lines = out.read_text().splitlines()
45         assert lines[0] == "start,end,shot_count,ending_reason"
46         assert lines[1].startswith("2.000,5.000,4,")
47         assert lines[2].startswith("9.000,12.500,,")
48
```

```

49
50 def test_write_report_json(tmp_path):
51     out = tmp_path / "report.json"
52     wmain._write_report_json("vid1", [{"start_time": 1.0, "end_time": 2.0}], "success",
str(out))
53     data = json.loads(out.read_text())
54     assert data["video_id"] == "vid1" and data["segment_count"] == 1
55     assert data["segments"][0] == {"start": 1.0, "end": 2.0}
56
57
58 def test_parser_infer_accepts_set_and_uris():
59     args = wmain.build_parser().parse_args([
60         "infer", "--video-uri", "s3://b/v.mp4", "--out-uri", "s3://b",
61         "--set", "indexing.skip_ai_handoff=true", "--set", "sport=tennis"])
62     assert args.cmd == "infer" and args.video_uri == "s3://b/v.mp4"
63     assert args.set == ["indexing.skip_ai_handoff=true", "sport=tennis"]
64
65
66     # ----- cmd_infer (local,
mocked)
67
68     class _OKResult:
69         passed = True
70         validator_name = "fake"
71         message = "ok"
72
73         def model_dump(self):
74             return {"validator_name": "fake", "passed": True, "message": "ok"}
75
76
77     class _FailResult(_OKResult):
78         passed = False
79         message = "too blurry"
80
81
82     class _FakeSeg:
83         def __init__(self, db, config):
84             pass
85
86         def process_video(self, video_id, path, max_frames=None):
87             return [
88                 {"start_time": 2.0, "end_time": 5.0, "shot_count": 4,
"shot_count_confidence": "LocalNoAI"},
89                 {"start_time": 9.0, "end_time": 12.0, "shot_count": 6,
"shot_count_confidence": "LocalNoAI"},
90             ]
91
92
93     def _fresh_config():
94         # Load defaults without touching any cwd config.json.
95         import tempfile
96         p = tempfile.NamedTemporaryFile("w", suffix=".json", delete=False)
97         p.write("{}")
98         p.close()
99         return load_config(p.name)
100
101
102 @pytest.fixture
103 def mocked_pipeline(monkeypatch):
104     monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vid123")
105     monkeypatch.setattr(wmain.validator_registry, "run_all_with_context",
lambda path, cfg: ([_OKResult()], {}))
106     monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name: _FakeSeg)
107     monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
108
109

```

```

110
111 def test_cmd_infer_local_roundtrip(tmp_path, mocked_pipeline):
112     video = tmp_path / "in.mp4"
113     video.write_bytes(b"FAKEVIDEO")
114     cfg = tmp_path / "config.json"
115     cfg.write_text("{}")
116     out = tmp_path / "bucket"
117     scratch = tmp_path / "scratch"
118
119     rc = wmain.main([
120         "infer", "--video-uri", str(video), "--out-uri", str(out),
121         "--config", str(cfg), "--scratch", str(scratch),
122         "--segmenter", "wasb_hybrid", "--set", "indexing.skip_ai_handoff=true",
123     ])
124     assert rc == 0
125     intervals = out / "outputs" / "vid123" / "intervals.csv"
126     report = out / "outputs" / "vid123" / "report.json"
127     assert intervals.exists() and report.exists()
128     rows = intervals.read_text().splitlines()
129     assert len(rows) == 3 # header + 2 rallies
130     assert json.loads(report.read_text())["segment_count"] == 2
131
132
133 def test_cmd_infer_validation_failure_returns_2(tmp_path, monkeypatch):
134     monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidF")
135     monkeypatch.setattr(wmain.validator_registry, "run_all_with_context",
136         lambda path, cfg: ([_FailResult()], {}))
137     monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
138     video = tmp_path / "in.mp4"
139     video.write_bytes(b"X")
140     cfg = tmp_path / "config.json"
141     cfg.write_text("{}")
142     rc = wmain.main(["infer", "--video-uri", str(video), "--out-uri", str(tmp_path /
143 "o"),
144         "--config", str(cfg), "--scratch", str(tmp_path / "s"))]
145
146     assert rc == 2
147
148
149 # --- cmd_train: pull labels -> synth trajectories -> manifest -> harness -> push bundle
150
151
152 def test_cmd_train_orchestration(tmp_path, monkeypatch):
153     import os
154     import subprocess
155
156     labels_dir = tmp_path / "corpus" / "labels"
157     labels_dir.mkdir(parents=True)
158     header = "rally_number,start_time,end_time,ending_reason,sport,shots_count\n"
159     (labels_dir / "v1.rallies.csv").write_text(header +
160 "1,2.0,5.0,winner,badminton,\n2,9.0,12.0,let,badminton,\n")
161     (labels_dir / "v2.rallies.csv").write_text(header + "1,3.0,6.0,winner,badminton,\n")
162     out_dir = tmp_path / "out"
163     scratch = tmp_path / "scratch"
164     cfg = tmp_path / "config.json"
165     cfg.write_text("{}")
166
167     # Mock the harness subprocess: write a fake versioned bundle at --out/--version.
168     def fake_run(cmd, *a, **k):
169         out = cmd[cmd.index("--out") + 1]
170         ver = cmd[cmd.index("--version") + 1]
171         bdir = os.path.join(out, ver)
172         os.makedirs(bdir, exist_ok=True)
173         for fn in ("weights.json", "manifest.json"):
174             with open(os.path.join(bdir, fn), "w") as f:
175                 f.write("{}")

```

```

172         return subprocess.CompletedProcess(cmd, 0)
173     monkeypatch.setattr(subprocess, "run", fake_run)
174
175     rc = wmain.main(["train", "--corpus-uri", str(tmp_path / "corpus"), "--out-uri",
176                     str(out_dir),
177                     "--version", "0.0.1", "--config", str(cfg), "--scratch",
178                     str(scratch)])
179     assert rc == 0
180     # bundle pushed to <out-uri>/weights/<version>/
181     assert (out_dir / "weights" / "0.0.1" / "weights.json").exists()
182     assert (out_dir / "weights" / "0.0.1" / "manifest.json").exists()
183     # the manifest fed to the harness enumerated both labeled videos with a trajectory
184     each
185     specs = json.loads((scratch / "manifest.json").read_text())
186     assert len(specs) == 2
187     assert all("trajectory" in s and "labels" in s and "fps" in s for s in specs)
188
189     def test_parser_train_detector_args():
190         args = wmain.build_parser().parse_args([
191             "train", "--corpus-uri", "gcs://b", "--out-uri", "gcs://b", "--version",
192             "1.0.0",
193             "--trajectory-source", "detector", "--segmenter", "fusion_hybrid"])
194         assert args.trajectory_source == "detector" and args.segmenter == "fusion_hybrid"
195
196     def test_cmd_train_detector_source_runs_real_extract_via_stub(tmp_path, monkeypatch):
197         """--trajectory-source detector pulls videos + runs the resolved DetectorRunner.
198         With
199         RALLY_DETECTOR_IMPL=stub the *wiring* is exercised GPU-free (the stub IS the
200         runner)."""
201         import os
202         import subprocess
203
204         monkeypatch.setenv("RALLY_DETECTOR_IMPL", "stub") # resolve a CPU stub runner (no
205         GPU/WSL)
206         corpus = tmp_path / "corpus"
207         labels_dir = corpus / "labels"
208         videos_dir = corpus / "videos"
209         labels_dir.mkdir(parents=True)
210         videos_dir.mkdir(parents=True)
211         header = "rally_number,start_time,end_time,ending_reason,sport,shots_count\n"
212         (labels_dir / "v1.rallies.csv").write_text(header + "1,2.0,5.0,winner,badminton,\n")
213         (labels_dir / "v2.rallies.csv").write_text(header + "1,3.0,6.0,winner,badminton,\n")
214         (videos_dir / "v1.mp4").write_bytes(b"FAKEVIDE01")
215         (videos_dir / "v2.mp4").write_bytes(b"FAKEVIDE02")
216         out_dir = tmp_path / "out"
217         scratch = tmp_path / "scratch"
218         cfg = tmp_path / "config.json"
219         cfg.write_text("{}")
220
221         def fake_run(cmd, *a, **k):
222             out = cmd[cmd.index("--out") + 1]
223             ver = cmd[cmd.index("--version") + 1]
224             bdir = os.path.join(out, ver)
225             os.makedirs(bdir, exist_ok=True)
226             for fn in ("weights.json", "manifest.json"):
227                 with open(os.path.join(bdir, fn), "w") as f:
228                     f.write("{}")
229             return subprocess.CompletedProcess(cmd, 0)
230         monkeypatch.setattr(subprocess, "run", fake_run)
231
232         rc = wmain.main(["train", "--corpus-uri", str(corpus), "--out-uri", str(out_dir),
233                         "--version", "0.0.2", "--config", str(cfg), "--scratch",
234                         str(scratch),

```

```

229         "--trajectory-source", "detector", "--segmenter", "wasb_hybrid"])
230     assert rc == 0
231     specs = json.loads((scratch / "manifest.json").read_text())
232     assert len(specs) == 2
233     # Real-extract path: each spec points at an on-disk trajectory the runner actually
wrote.
234     assert all(os.path.exists(s["trajectory"]) for s in specs)
235     assert all(s["trajectory"].endswith("_stub.csv") for s in specs) # the stub
runner's output
236

```

30. assistant (2026-06-20T08:58:26.623Z)

Now let me run the new tests and the broader suite to confirm green, plus run ruff/mypy on the new files per the pre-LGTM gate.

31. user (2026-06-20T08:58:35.994Z)

```

..... [100%]
43 passed in 5.99s

```

32. user (2026-06-20T08:58:44.159Z)

```

All checks passed!
=== MYPY ===
backend\sports\_init__.py:7: note: By default the bodies of untyped functions are not checked,
consider using --check-untyped-defs [annotation-unchecked]
backend\pipeline\validators\base.py:36: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
backend\providers\_init__.py:7: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
Success: no issues found in 2 source files

```

33. assistant (2026-06-20T08:58:53.263Z)

Clean. Now let me focus on hunting the real cross-platform bugs. The key concern from the prompt is the `\_resolve\_train\_detector` worker path. Let me look more carefully: in `cmd\_train` detector source, the runner is built once and reused across videos. But `run\_predict` writes to `traj\_dir` which is the SAME directory for every video. The native runner uses `frames\_dir = os.path.join(scratch, f"{key}\_frames")` where `scratch = self.cfg.scratch\_dir` or `output\_dir`. With env `WASB\_SCRATCH` unset, scratch falls back to `output\_dir` = `traj\_dir` (the abspath of it). That's fine.

But there's a subtler issue: the worker train passes `traj\_dir` (relative path `scratch/trajectories`) to `run\_predict`, which does `os.path.abspath`. Good.

Let me now examine a genuine concern: `os.path.expanduser` of the default `repo_dir="~/models/WASB-SBDT"` on the GPU box, and how the `_copy_infer_script` interacts with the resumable cache keying. Let me check the cache-key consistency issue between disk mode and what the native runner does. Let me also verify the `make_video_frame_loader` count check and the device-keyed manifest don't break parity.

The biggest correctness question per the acceptance gate is `byte-parity`. Let me compare the exact argv the WSL runner builds vs the native runner, paying attention to `--frames_out_dir` and the cache dir defaults.

34. user (2026-06-20T08:59:02.839Z)

```

device default parity: WSL omits --device (main default cuda); native passes --device cuda ->
SAME effective override
frames_out_dir handling OK on Linux

```

35. assistant (2026-06-20T08:59:06.766Z)

Now let me dig into a real parity concern I noticed: the ****device-keyed manifest** in ``manifest_compatible`` and a potential parity break. The acceptance gate says "device default 'cuda' must keep wasb_infer's Hydra overrides identical to before (parity)." Let me check the ``_build_cfg`` change carefully ? the prior version. Let me look at the git diff for `wasb_infer.py`.

36. user (2026-06-20T08:59:07.474Z)

```
diff --git a/backend/pipeline/detectors/wasb_infer.py b/backend/pipeline/detectors/wasb_infer.py
index 7b589d5..5df5939 100644
```

```
--- a/backend/pipeline/detectors/wasb_infer.py
+++ b/backend/pipeline/detectors/wasb_infer.py
@@ -53,16 +53,22 @@ _DET_FILE = "det_raw.jsonl"
 _MANIFEST_FILE = "manifest.json"
```

```
-def _build_cfg(weights: str, sport: str):
-    """Compose the WASB eval config via Hydra, overriding model/weights/device."""
+def _build_cfg(weights: str, sport: str, device: str = "cuda"):
+    """Compose the WASB eval config via Hydra, overriding model/weights/device.
+
+    ``device`` is ``cuda`` (default ? the historical WSL behaviour, byte-parity preserved),
+    ``mps`` (Apple) or ``cpu``. Only the single-GPU CUDA path requests ``runner.gpus=[0]``;
+    cpu/mps run with no GPU list so the detector places tensors on ``device``.
+
+    """
+
+    from hydra import compose, initialize
+    gpus = "[0]" if device == "cuda" else "[]" # single GPU on cuda; none for cpu/mps
+    with initialize(config_path="configs", version_base=None):
+        cfg = compose(config_name="eval", overrides=[
+            f"dataset={sport}",
+            "model=wasb",
+            f"detector.model_path={weights}",
+            "runner.device=cuda",
+            "runner.gpus=[0]", # this box has a single GPU; default config assumes more
+            f"runner.device={device}",
+            f"runner.gpus={gpus}", # default config assumes multiple GPUs; pin to this box
+        ])
+    return cfg
```

```
@@ -129,8 +135,13 @@ def read_manifest(cache_dir: str):
```

```
def manifest_compatible(manifest: dict, *, frames_dir: str, weights: str, sport: str,
-    frames_in: int, frames_total: int) -> bool:
-    """A cache is reusable only if the run that produced it matches this run's inputs."""
+    frames_in: int, frames_total: int, device: str = "cuda") -> bool:
+    """A cache is reusable only if the run that produced it matches this run's inputs.
+
+    ``device`` defaults to ``cuda`` so caches written before device-keying (no ``device``
+    field) stay compatible with a CUDA run ? but a cpu/mps run will NOT reuse a cuda cache
+    (detections can differ numerically across devices), and vice-versa.
+
+    """
+
+    if not manifest or manifest.get("version") != CACHE_VERSION:
+        return False
+    return (
```

```
@@ -139,6 +150,7 @@ def manifest_compatible(manifest: dict, *, frames_dir: str, weights: str,
sport:
```

```
    and manifest.get("sport") == sport
    and manifest.get("frames_in") == frames_in
    and manifest.get("frames_total") == frames_total
+    and manifest.get("device", "cuda") == device
+
+)
```

```
@@ -360,7 +372,8 @@ def run(frames, weights: str, out_csv: str, sport: str = "badminton",
```

```

        limit: int = 0, batch_size: int = 8, num_workers: int = 4,
        log_every_batches: int = 50, cache_dir: "str | None" = None,
        flush_every: int = 1000, fresh: bool = False,
-       frame_loader=None, source_key: "str | None" = None) -> str:
+       frame_loader=None, source_key: "str | None" = None,
+       device: str = "cuda") -> str:
    """Run WASB detection+tracking over an ordered frame source -> trajectory CSV.

    Two equivalent ways to supply pixels (output is bit-identical between them):
@@ -391,7 +404,7 @@ def run(frames, weights: str, out_csv: str, sport: str = "badminton",

    streaming = frame_loader is not None

-   cfg = _build_cfg(weights, sport)
+   cfg = _build_cfg(weights, sport, device)
    frames_in = int(cfg["model"]["frames_in"])
    input_wh = (int(cfg["model"]["inp_width"]), int(cfg["model"]["inp_height"]))
    output_wh = (int(cfg["model"]["out_width"]), int(cfg["model"]["out_height"]))
@@ -427,7 +440,7 @@ def run(frames, weights: str, out_csv: str, sport: str = "badminton",
    manifest = None if fresh else read_manifest(cache_dir)
    resume = manifest_compatible(
        manifest or {}, frames_dir=cache_key, weights=weights, sport=sport,
-       frames_in=frames_in, frames_total=len(frames),
+       frames_in=frames_in, frames_total=len(frames), device=device,
    )
    if resume:
        windows_done, det_by_fid = load_and_clean_cache(cache_dir)
@@ -445,6 +458,7 @@ def run(frames, weights: str, out_csv: str, sport: str = "badminton",
    "sport": sport,
    "frames_in": frames_in,
    "frames_total": len(frames),
+   "device": device,
    "windows_total": windows_total,
    "windows_done": windows_done,
}
@@ -630,7 +644,8 @@ def make_video_frame_loader(video_path: str):
def run_video_streaming(video_path: str, weights: str, out_csv: str, sport: str = "badminton",
    limit: int = 0, batch_size: int = 8,
    log_every_batches: int = 50, cache_dir: "str | None" = None,
-   flush_every: int = 1000, fresh: bool = False) -> str:
+   flush_every: int = 1000, fresh: bool = False,
+   device: str = "cuda") -> str:
    """Fast path: decode ``video_path`` on the fly and feed frames straight to the
    detector, WITHOUT extracting every frame to a PNG first (the disk round-trip that
    dominates a long clip). Output is bit-identical to the frames-on-disk path; see
@@ -648,7 +663,7 @@ def run_video_streaming(video_path: str, weights: str, out_csv: str, sport:
str
        batch_size=batch_size, num_workers=0,
        log_every_batches=log_every_batches, cache_dir=cache_dir,
        flush_every=flush_every, fresh=fresh,
-       frame_loader=frame_loader, source_key=source_key)
+       frame_loader=frame_loader, source_key=source_key, device=device)

    finally:
        release()

@@ -664,6 +679,8 @@ def main():
    ap.add_argument("--weights", required=True)
    ap.add_argument("--out", required=True)
    ap.add_argument("--sport", default="badminton")
+   ap.add_argument("--device", default="cuda", choices=["cuda", "mps", "cpu"],
+   help="torch device for the detector (default cuda = the WSL parity path)")
    ap.add_argument("--limit", type=int, default=0, help="cap #frames (for quick tests)")
    ap.add_argument("--batch-size", type=int, default=8)
    ap.add_argument("--num-workers", type=int, default=4)
@@ -681,7 +698,7 @@ def main():

```

```

        run_video_streaming(args.video, args.weights, args.out, args.sport, args.limit,
                             batch_size=args.batch_size,
                             log_every_batches=args.log_every_batches, cache_dir=args.cache_dir,
-                             flush_every=args.flush_every, fresh=args.fresh)
+                             flush_every=args.flush_every, fresh=args.fresh, device=args.device)

    return

    frames_dir = args.frames_dir
@@ -692,7 +709,7 @@ def main():
    run(frames_dir, args.weights, args.out, args.sport, args.limit,
        batch_size=args.batch_size, num_workers=args.num_workers,
        log_every_batches=args.log_every_batches, cache_dir=args.cache_dir,
-        flush_every=args.flush_every, fresh=args.fresh)
+        flush_every=args.flush_every, fresh=args.fresh, device=args.device)

if __name__ == "__main__":

```

37. assistant (2026-06-20T08:59:11.761Z)

The wasb_infer.py changes preserve cuda parity: `device="cuda"` default ? `gpus="[0]"` ? identical Hydra overrides as before. The manifest `device` key uses `.get("device", "cuda")` so old caches stay compatible. Good.

Now let me verify there are no existing tests for wasb_infer.py that might break with the device-keyed manifest, and run the full detector test suite.

38. user (2026-06-20T08:59:23.431Z)

```

..... [ 39%]
..... [ 78%]
..... [100%]
184 passed, 807 deselected in 8.56s

```

39. assistant (2026-06-20T08:59:26.454Z)

Now let me run the full suite quickly to confirm nothing else regressed, then I'll finalize the cross-platform analysis.

40. user (2026-06-20T09:00:51.192Z)

```

obsreport (optional reporting dep): available
=====
..... [ 7%]
..... [ 14%]
..... [ 21%]
..... [ 29%]
..... [ 36%]
..... [ 43%]
..... [ 50%]
..... [ 58%]
..... [ 65%]
..... [ 72%]
..... [ 79%]
..... [ 87%]
..... [ 94%]
..... [100%]
991 passed in 81.06s (0:01:21)
=====
RESULT: PASS - all tests green
=====

```

41. assistant (2026-06-20T09:00:56.481Z)

Full suite is green (991 passed). Now let me complete the targeted cross-platform bug hunt. Let me re-examine the specific items the prompt flagged, focusing on real Linux production correctness.

The one genuine concern I want to verify is the **frame cache vs resumable detector cache** interaction in the native runner. On a failure, the native runner returns without cleaning ``frames_dir``, but the resumable detector cache (``<out>_wasbcache/``) lives next to the CSV in ``output_dir``. On success, it cleans ``frames_dir`` but NOT the ``_wasbcache`` dir. Let me check whether the WSL runner cleans the wasbcache ? if there's an asymmetry that leaks disk on the native path.

42. user (2026-06-20T09:00:58.073Z)

```
backend\pipeline\detectors\native_wasb_runner.py:110:    def _rm_rf(self, path: Optional[str])
-> None:
backend\pipeline\detectors\native_wasb_runner.py:201:        self._rm_rf(frames_dir)
backend\pipeline\detectors\base.py:129:    def _wsl_rm_rf(self, wsl_paths: List[str]) -> None:
backend\pipeline\detectors\wasb_infer.py:31:The cache is keyed by the output path
(`<out>_wasbcache/` by default), lives next
backend\pipeline\detectors\wasb_infer.py:86:def _cache_paths(cache_dir: str):
backend\pipeline\detectors\wasb_infer.py:87:    return osp.join(cache_dir, _DET_FILE),
osp.join(cache_dir, _MANIFEST_FILE)
backend\pipeline\detectors\wasb_infer.py:90:def default_cache_dir(out_csv: str) -> str:
backend\pipeline\detectors\wasb_infer.py:94:    return osp.join(d, stem + "_wasbcache")
backend\pipeline\detectors\wasb_infer.py:121:def write_manifest(cache_dir: str, manifest: dict)
-> None:
backend\pipeline\detectors\wasb_infer.py:122:    _, mpath = _cache_paths(cache_dir)
backend\pipeline\detectors\wasb_infer.py:126:def read_manifest(cache_dir: str):
backend\pipeline\detectors\wasb_infer.py:127:    _, mpath = _cache_paths(cache_dir)
backend\pipeline\detectors\wasb_infer.py:157:def reset_cache(cache_dir: str) -> None:
backend\pipeline\detectors\wasb_infer.py:159:    det_jsonl, mpath = _cache_paths(cache_dir)
backend\pipeline\detectors\wasb_infer.py:165:def load_and_clean_cache(cache_dir: str):
backend\pipeline\detectors\wasb_infer.py:174:    det_jsonl, _ = _cache_paths(cache_dir)
backend\pipeline\detectors\wasb_infer.py:373:        log_every_batches: int = 50, cache_dir:
"str | None" = None,
backend\pipeline\detectors\wasb_infer.py:435:    if cache_dir is None:
backend\pipeline\detectors\wasb_infer.py:436:        cache_dir = default_cache_dir(out_csv)
backend\pipeline\detectors\wasb_infer.py:437:    os.makedirs(cache_dir, exist_ok=True)
backend\pipeline\detectors\wasb_infer.py:438:    det_jsonl, _ = _cache_paths(cache_dir)
backend\pipeline\detectors\wasb_infer.py:440:    manifest = None if fresh else
read_manifest(cache_dir)
backend\pipeline\detectors\wasb_infer.py:446:        windows_done, det_by_fid =
load_and_clean_cache(cache_dir)
backend\pipeline\detectors\wasb_infer.py:451:        reset_cache(cache_dir)
backend\pipeline\detectors\wasb_infer.py:465:    write_manifest(cache_dir, base_manifest)
backend\pipeline\detectors\wasb_infer.py:527:        write_manifest(cache_dir,
base_manifest)
backend\pipeline\detectors\wasb_infer.py:539:        write_manifest(cache_dir,
base_manifest)
backend\pipeline\detectors\wasb_infer.py:646:        log_every_batches: int =
50, cache_dir: "str | None" = None,
backend\pipeline\detectors\wasb_infer.py:664:    log_every_batches=log_every_batches, cache_dir=cache_dir,
backend\pipeline\detectors\wasb_infer.py:689:        help="resumable detector cache
dir (default: <out>_wasbcache next to --out)")
backend\pipeline\detectors\wasb_infer.py:700:    log_every_batches=args.log_every_batches, cache_dir=args.cache_dir,
backend\pipeline\detectors\wasb_infer.py:711:        log_every_batches=args.log_every_batches,
cache_dir=args.cache_dir,
backend\pipeline\detectors\wasb_runner.py:194:        self._wsl_rm_rf(stale)
backend\pipeline\detectors\tracknet_runner.py:209:        self._wsl_rm_rf([wsl_video])
```

43. assistant (2026-06-20T09:01:10.259Z)

Confirmed: neither the WSL runner nor the native runner cleans the ``<out>_wasbcache/`` dir on

success ? both only clean the frame cache. So that's symmetric (pre-existing behavior, not a regression introduced here). Not a finding for this change.

Now, the critical cross-platform parity concern. The WSL runner writes frames to `~/clips/<key>_frames`` (the WSL stage dir), and the resumable cache lands next to `--out`` (the output dir). The native runner writes frames to `scratch/<key>_frames``. The `--out`` CSV path is the same dir contract for both. The `**cache key**` in `manifest_compatible`` for disk mode is `osp.abspath(frames_dir)`` ? which differs between WSL (`~/clips/...``) and native (`scratch/...``). But since these are different machines, no clash. Parity of the OUTPUT CSV is what matters, and that's driven by pixels + weights + device, not the frames_dir path. Good.

Let me verify one last real bug candidate: the native runner's `_run`` uses `cwd=self._repo_src()`` and runs `python wasb_infer.py`` (relative). The WSL runner does `cd repo/src && python wasb_infer.py``. `wasb_infer's _build_cfg`` uses `initialize(config_path="configs", version_base=None)`` ? a `**relative**` Hydra config path resolved against the `**caller's file location**` (`wasb_infer.py` is copied into `repo/src/``), not `cwd`. Let me confirm Hydra's `config_path`` is relative to the `.py` file, which means `cwd` matters less but the script must be in `src/``. The native runner copies it there via `_copy_infer_script``. Good ? parity preserved.

Let me do a final check on a potential real Linux issue: `os.path.abspath(str(video_win_path))`` when the worker passes a path. In the worker train path, `local_video`` is `os.path.join(videos_dir, name)`` ? a real local path. On Linux, `abspath`` is correct. Fine.

Let me verify the one thing that could actually bite on Linux: whether `shutil.copy(_INFER_PY, ...)`` and the `_INFER_PY`` resolution work when the package is installed/run from the GPU box.

44. user (2026-06-20T09:01:13.388Z)

`wasb_infer.py` sibling exists: True

45. assistant (2026-06-20T09:01:26.832Z)

I have completed my analysis. Let me compile my findings.

Summary of analysis

****No-WSL / no-conda-activate / no-mnt guarantee: CLEAN.**** `native_wasb_runner.py`` imports only `DetectorRunner`` (the ABC) and `TrackNetRunner`` (for static windowing helpers). It does NOT import or inherit `WslCommandMixin``, and does NOT import `to_wsl_mnt_path`` or `wsl_tilde_quote``. The argv is pure list-form (`[python_bin, "wasb_infer.py", "--video", ...]``) with no shell, no `wsl``, no `conda activate``, no `/mnt`` translation. `test_is_not_a_wsl_runner`` locks this. No leak found.

****Parity (cuda default): PRESERVED.**** `_build_cfg(device="cuda")`` ? `gpus="[0]"`` ? byte-identical Hydra overrides to the old hardcoded `runner.device=cuda`` / `runner.gpus=[0]``. The native runner passes `--device cuda`` explicitly, which equals `wasb_infer.main()``'s default. The device-keyed manifest uses `.get("device", "cuda")`` so pre-existing caches stay compatible.

****Cross-platform (runs on Linux, tested on Windows): correct for production.**** On real Linux paths, `os.path.abspath``, `expanduser``, `os.path.join(repo,"src")`` all behave correctly. The `norm = video.replace("\\", "/")`` only drives the basename/key (so Windows test paths yield a clean key on Linux); the production `--video`` value uses `os.path.abspath(video)`` on a real Linux path, which is fine. I confirmed `config.get("indexing")`` returns a dict through the worker, and `NativeWasbConfig.from_indexing_cfg`` reads it correctly.

****Verification:**** full suite 991 passed; native+worker tests 43 passed; ruff clean; mypy clean on the new files.

The findings below are minor/nit ? no blockers or majors. The change is sound for its acceptance gate (which is owner-side byte-parity on the GPU box).

46. user (2026-06-20T09:01:48.989Z)

Structured output provided successfully

47. assistant (2026-06-20T09:02:05.378Z)

I have completed the no-WSL / cross-platform skeptic review of the M3.5 native WASB runner change.

Verdict

****No blockers or majors.**** The change is sound against its stated contracts. Findings are all nits.

No-WSL guarantee: CLEAN (the core concern)

`backend/pipeline/detectors/native\_wasb\_runner.py` carries zero WSL plumbing:
- Imports only `DetectorRunner` (ABC) and `TrackNetRunner` (static windowing helpers) ? NOT `WslCommandMixin`, NOT `to\_wsl\_mnt\_path`, NOT `wsl\_tilde\_quote`.
- `argv` is pure list-form: `[python\_bin, "wasb\_infer.py", "--video", abspath, ...]` ? no shell, no `wsl`, no `conda activate`, no `/mnt` translation. The `wasb` env python is invoked by path (`python\_bin`).
- `test\_is\_not\_a\_wsl\_runner` locks `not isinstance(r, WslCommandMixin)` and `not hasattr(r, "\_wsl\_bash")`.

Parity (cuda default): PRESERVED (the acceptance gate)

- `\_build\_cfg(device="cuda")` ? `gpus="[0]"` ? byte-identical Hydra overrides to the old hardcoded `runner.device=cuda` / `runner.gpus=[0]`.
- Native passes `--device cuda` explicitly = `wasb\_infer.main()`'s default.
- Device-keyed manifest uses `.get("device", "cuda")`, so pre-device caches stay CUDA-compatible.
- Same CSV name `{stem}\_\_{vid12}\_wasb.csv`, same `default\_cache\_dir` (next to `--out`) ? downstream + the owner's byte-parity diff line up.

Cross-platform: correct for production Linux

Verified `os.path.abspath`/`expanduser`/`os.path.join(repo,"src")` all behave on real Linux paths; `norm = video.replace("\\", "/")` only feeds the basename/key so Windows test paths yield clean keys on Linux; `config.get("indexing")` correctly returns a dict through the worker train path into `NativeWasbConfig.from\_indexing\_cfg`.

Verification performed (pre-LGTM gate)

- `python run\_all\_tests.py` ? 991 passed.
- `tests/test\_native\_wasb\_runner.py` + `tests/test\_worker.py` ? 43 passed.
- `ruff check` on all 7 changed/new files ? clean.
- `mypy` on `native\_wasb\_runner.py` + `worker/\_\_main\_\_.py` ? clean.

The three nits (raw-vs-normalized `--video` path symmetry at `native\_wasb\_runner.py:163`;
`<out>\_wasbcache` not reclaimed on success ? pre-existing parity behavior, not a regression;
`scratch\_dir` env-only with no config knob) are detailed in the structured output. None affect the GPU-box byte-parity acceptance gate.

Files reviewed (all absolute): `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\detectors\native\_wasb\_runner.py`, `...\wasb\_infer.py`, `...\wasb\_runner.py`, `...\base.py`, `...\tracknet\_runner.py`, `...\segmenters\trajectory\_hybrid.py`, `...\wasb\_hybrid.py`, `...\fusion\_hybrid.py`, `...\config\models.py`, `...\worker\\_\_main\_\_.py`, `...\tests\test\_native\_wasb\_runner.py`, `...\tests\test\_worker.py`.